

DESARROLLO DE LA METODOLOGÍA SCRUM

Milagros Llauce

ÍNDICE DE CONTENIDO

PRODUCT BACKLOG	125
PRODUCT BACKLOG PRIORIZADO	128
PLANIFICACIÓN DE LANZAMIENTO	132
HISTORIAS DE USUARIO	133
TAREAS POR HISTORIA DE USUARIO	144
CRONOGRAMA DEL SPRINT	145
SPRINT 1	147
ANÁLISIS DE REQUERIMIENTOS SPRINT 1	148
FUNCIONALIDADES DEL PRIMER SPRINT	149
DISEÑO DEL PRIMER SPRINT	153
ENTIDADES DEL PRIMER SPRINT	165
DIAGRAMA LÓGICO DEL PRIMER SPRINT	166
DIAGRAMA FÍSICO DEL PRIMER SPRINT	166
IMPLEMENTACIÓN DEL PRIMER SPRINT	167
PRUEBAS DE CAJA NEGRA DEL PRIMER SPINRT	188
ACTAS DE PLANIFICACIÓN Y REVISIÓN DEL SPRINT 1	194
SPRINT 2	196
ANÁLISIS DEL SEGUNDO SPRINT	196
FUNCIONALIDADES DEL SEGUNDO SPRINT	197
DISEÑO DEL SEGUNDO SPRINT	199
ENTIDADES DEL SEGUNDO SPRINT:	207
DIAGRAMA ENTIDAD / RELACIÓN DEL SEGUNDO SPRINT	208
DIAGRAMA LÓGICO DEL SEGUNDO SPRINT	209
DIAGRAMA FÍSICO DEL SEGUNDO SPRINT	209
IMPLEMENTACIÓN DEL SEGUNDO SPRINT	210
PRUEBAS DE CAJA NEGRA DEL SEGUNDO SPRINT	225
ACTAS DE PLANIFICACIÓN Y REVISIÓN DEL SPRINT 2	229
SPRINT 3	231
ANÁLISIS DEL TERCER SPRINT	231
FUNCIONALIDADES DEL TERCER SPRINT	231
DISEÑO DEL SPRINT 3	233
ENTIDADES DEL TERCER SPRINT	238
DIAGRAMA ENTIDAD/ RELACIÓN DEL TERCER SPRINT	238
DIAGRAMA LÓGICO DEL TERCER SPRINT	239
IMPLEMENTACIÓN DEL TERCER SPRINT	240
PRUEBA DE CAJA NEGRA DEL TERCER SPRINT	249

ACTAS DE PLANIFICACIÓN Y REVISIÓN DEL SPRINT 3.....	251
SPRINT 4	253
ANÁLISIS DEL SPRINT 4.....	253
FUNCIONALIDADES DEL CUARTO SPRINT	253
DISEÑO DEL CUARTO SPRINT	254
ENTIDADES DEL CUARTO SPRINT	258
DIAGRAMA ENTIDAD/ RELACIÓN DEL CUARTO SPRINT	259
DIAGRAMA LÓGICO DEL CUARTO SPRINT	259
DIAGRAMA FÍSICO DEL CUARTO SPRINT	260
IMPLEMENTACIÓN DEL CUARTO SPRINT	261
PRUEBA CAJA NEGRA DEL CUARTO SPRINT	269
ACTAS DE PLANIFICACIÓN Y REVISIÓN DEL SPRINT 4.....	271

INDICE DE TABLAS

Tabla 1.	Product Backlog	125
Tabla 2.	Nivel de prioridad según MoSCoW	128
Tabla 3.	Backlog priorizado	128
Tabla 4.	Resumen historias de usuario.....	133
Tabla 5.	Tareas por historia de usuario.....	144
Tabla 6.	Requerimientos del Sprint 1	148
Tabla 7.	Funcionalidades del primer sprint	149
Tabla 8.	Retrospectiva del sprint 1	192
Tabla 9.	Sprint 2.....	196
Tabla 10.	Funcionalidades del sprint 2	197
Tabla 11.	Pruebas HU 07	225
Tabla 12.	Pruebas HU 08	226
Tabla 13.	Retrospectiva del sprint 2	228
Tabla 14.	Sprint 3.....	231
Tabla 15.	Funcionalidades del sprint 3	231
Tabla 16.	Retrospectiva del sprint 3	250
Tabla 17.	Sprint 4.....	253
Tabla 18.	Funcionalidades del sprint 4	253
Tabla 19.	Retrospectiva del sprint 4	270

INDICE DE FIGURAS

Figura 1.	Cronograma de la planificación de lanzamiento.	132
Figura 2.	Historia de usuario N°1	134
Figura 3.	Historia de usuario N°2	135
Figura 4.	Historia de usuario N°3	136
Figura 5.	Historia de Usuario N°4	137
Figura 6.	Historia de Usuario N°5	138
Figura 7.	Historia de Usuario N°6	139
Figura 8.	Historia de Usuario N°7	140
Figura 9.	Historia de Usuario N°8	141
Figura 10.	Historia de Usuario N°9	142
Figura 11.	Historia de Usuario N°10	143
Figura 12.	Cronograma del sprint 1	146
Figura 13.	Cronograma del sprint 2	146
Figura 14.	Cronograma del sprint 3	147
Figura 15.	Cronograma del sprint 4	147
Figura 16.	Prototipo para el requerimiento RF-01.....	153
Figura 17.	Login.....	154
Figura 18.	Prototipo para el requerimiento RF-02.....	155
Figura 19.	Dashboard – productos más vendidos	155
Figura 20.	Prototipo para el requerimiento RF-03.....	156
Figura 21.	Productos más vendidos por categoría	156
Figura 22.	Prototipo para el requerimiento RF-05.....	157
Figura 23.	Tabla informativa de stock.....	157
Figura 24.	Prototipo para el requerimiento RF-06.....	158
Figura 25.	Crear nuevo usuario	158
Figura 26.	Prototipo para el requerimiento RF-10.....	159
Figura 27.	Buscar productos	159
Figura 28.	Prototipo para el requerimiento RF-11.....	160
Figura 29.	Crear productos.....	160
Figura 30.	Prototipo para el requerimiento RF-12.....	161
Figura 31.	Editar productos	161
Figura 32.	Prototipo para el requerimiento RF-13.....	162
Figura 33.	Eliminar producto	162
Figura 34.	Prototipo para el requerimiento RF-14.....	163
Figura 35.	Crear categoría	163

Figura 36.	Prototipo para el requerimiento RF-15.....	164
Figura 37.	Editar categoría existente	164
Figura 38.	Entidades del sprint 1.....	165
Figura 39.	Diagrama entidad relación del sprint 1	165
Figura 40.	Diagrama Lógico del Sprint 1	166
Figura 41.	Diagrama físico del sprint 1	167
Figura 42.	Diagrama de la base de datos del Sprint 1.....	168
Figura 43.	Modelo del Requerimiento Funcional 01.....	169
Figura 44.	Vista del Requerimiento Funcional 01.....	170
Figura 45.	Controlador del Requerimiento Funcional 01.....	170
Figura 46.	Vista del requerimiento funcional 02.....	171
Figura 47.	Vista del modelo del requerimiento funcional 02	171
Figura 48.	Modelo del requerimiento 03	172
Figura 49.	Vista del requerimiento 03	173
Figura 50.	Controlador del requerimiento 03.....	173
Figura 51.	Modelo del requerimiento 04	174
Figura 52.	Vista del requerimiento 04	175
Figura 53.	Modelo del requerimiento 06	176
Figura 54.	Vista del requerimiento 06	177
Figura 55.	Controlador del requerimiento 6.....	178
Figura 56.	Modelo del requerimiento 10	178
Figura 57.	Vista del requerimiento 10	180
Figura 58.	Modelo del requerimiento 12	180
Figura 59.	Vista del requerimiento 12	181
Figura 60.	Controlador del requerimiento 12.....	182
Figura 61.	Modelo del requerimiento 13	182
Figura 62.	Vista del requerimiento 13	183
Figura 63.	Controlador del requerimiento 13.....	183
Figura 64.	Modelo del requerimiento 14	184
Figura 65.	Vista del requerimiento 14	185
Figura 66.	Controlador del requerimiento 14.....	185
Figura 67.	Modelo del requerimiento 15	186
Figura 68.	Vista del requerimiento 15	187
Figura 69.	Controlador del requerimiento 15.....	187
Figura 70.	Pruebas del HU01	188
Figura 71.	Prueba del HU02	189
Figura 72.	Prueba del HU03	190

Figura 73.	Prueba del HU05	191
Figura 74.	Prueba del HU06	192
Figura 75.	Acta de planificación Sprint 1	194
Figura 76.	Acta de revisión del sprint 1	195
Figura 77.	Prototipo para el requerimiento RF-16.....	199
Figura 78.	Realizar venta.....	200
Figura 79.	Prototipo para el requerimiento RF-17.....	200
Figura 80.	Generar comprobante	201
Figura 81.	Prototipo para el requerimiento RF-18.....	202
Figura 82.	Cancelar venta	202
Figura 83.	Prototipo para el requerimiento RF-19.....	203
Figura 84.	Realizar abastecimiento	203
Figura 85.	Prototipo para el requerimiento RF-25.....	204
Figura 86.	Reporte rotación de inventarios	204
Figura 87.	Prototipo para el requerimiento RF-26.....	205
Figura 88.	Reporte porcentaje de inventario sin rotación	205
Figura 89.	Prototipo para el requerimiento RF-27.....	206
Figura 90.	Reporte duración del inventario	206
Figura 91.	Prototipo para el requerimiento RF-28.....	207
Figura 92.	Reporte Valor económico del inventario.....	207
Figura 93.	Entidades del Sprint 2	208
Figura 94.	Diagrama Entidad / Relación del Sprint 2	208
Figura 95.	Diagrama Lógico del Sprint 2	209
Figura 96.	Diagrama Fisico del Sprint 2.....	210
Figura 97.	Diagrama de la base de datos del Sprint 2	211
Figura 98.	Modelo del requerimiento 16	211
Figura 99.	Vista del requerimiento 16	212
Figura 100.	Controlador del requerimiento 16.....	212
Figura 101.	Modelo del requerimiento 17	213
Figura 102.	Vista del requerimiento 17	214
Figura 103.	Controlador del requerimiento 17.....	214
Figura 104.	Modelo del requerimiento 18	215
Figura 105.	Vista del requerimiento 18	216
Figura 106.	Controlador del requerimiento 18.....	216
Figura 107.	Modelo del requerimiento 19	217
Figura 108.	Vista del requerimiento 19	218
Figura 109.	Controlador del requerimiento 19.....	218

Figura 110.	Modelo del requerimiento 25	219
Figura 111.	Vista del requerimiento 25	219
Figura 112.	Controlador del requerimiento 25.....	220
Figura 113.	Modelo del requerimiento 26	220
Figura 114.	Vista del requerimiento 26	221
Figura 115.	Controlador del requerimiento 26.....	221
Figura 116.	Modelo del requerimiento 27	222
Figura 117.	Vista del requerimiento 27	222
Figura 118.	Controlador del requerimiento 27.....	223
Figura 119.	Modelo del requerimiento 28	224
Figura 120.	Vista del requerimiento 28	224
Figura 121.	Controlador del requerimiento 28.....	224
Figura 122.	Pruebas HU 10	227
Figura 123.	Acta de planificación sprint 2	229
Figura 124.	Acta de revisión del sprint 2	230
Figura 125.	Prototipo para el requerimiento RF-05.....	233
Figura 126.	Búsqueda de usuario por carácter o rol	234
Figura 127.	Prototipo para el requerimiento RF-07.....	234
Figura 128.	Editar usuario.....	235
Figura 129.	Prototipo para el requerimiento RF-08.....	235
Figura 130.	Eliminar usuario.....	236
Figura 131.	Prototipo para el requerimiento RF-23.....	236
Figura 132.	Reporte de ventas realizadas.....	237
Figura 133.	Prototipo para el requerimiento RF-24.....	237
Figura 134.	Reportes abastecimientos realizados	238
Figura 135.	Entidades del Sprint 3	238
Figura 136.	Diagrama Entidad / Relación del Sprint 3	239
Figura 137.	Diagrama Lógico del Sprint 3	239
Figura 138.	Diagrama Físico del Sprint 3.....	240
Figura 139.	Diagrama de la base de datos del Sprint 3	240
Figura 140.	Modelo del requerimiento 05	241
Figura 141.	Vista del requerimiento 05	241
Figura 142.	Controlador del requerimiento 05.....	242
Figura 143.	Modelo del requerimiento 07	243
Figura 144.	Vista del requerimiento 07	243
Figura 145.	Controlador del requerimiento 07.....	244
Figura 146.	Modelo del requerimiento 08	245

Figura 147.	Vista del requerimiento 08	245
Figura 148.	Controlador del requerimiento 08.....	245
Figura 149.	Modelo del requerimiento 23	246
Figura 150.	Vista del requerimiento 23	246
Figura 151.	Controlador del requerimiento 23.....	247
Figura 152.	Modelo del requerimiento 24	248
Figura 153.	Vista del requerimiento 24	248
Figura 154.	Controlador del requerimiento 24.....	249
Figura 155.	Pruebas HU3	250
Figura 156.	Pruebas HU9	250
Figura 157.	Acta de planificación Sprint 3.....	252
Figura 158.	Acta de revisión del sprint 3	253
Figura 159.	Prototipo para el requerimiento RF-09.....	255
Figura 160.	Crear sucursal.....	255
Figura 161.	Ventas por producto.....	256
Figura 162.	Prototipo para el requerimiento RF-20.....	256
Figura 163.	Reporte de ventas por vendedor	257
Figura 164.	Prototipo para el requerimiento RF-21.	257
Figura 165.	Detalle stock de producto	258
Figura 166.	Prototipo para el requerimiento RF-22.....	258
Figura 167.	Entidades del Sprint 4	259
Figura 168.	Diagrama Entidad / Relación del Sprint 4	259
Figura 169.	Diagrama Lógico del Sprint 4	260
Figura 170.	Diagrama Físico del Sprint 4.....	261
Figura 171.	Diagrama de la base de datos del Sprint 4	262
Figura 172.	Modelo del requerimiento 09	262
Figura 173.	Vista del requerimiento 09	263
Figura 174.	Controlador del requerimiento 09.....	263
Figura 175.	Modelo del requerimiento 20	264
Figura 176.	Vista del requerimiento 20	264
Figura 177.	Controlador del requerimiento 20.....	265
Figura 178.	Modelo del requerimiento 21	265
Figura 179.	Vista del requerimiento 21	266
Figura 180.	Controlador del requerimiento 21.....	266
Figura 181.	Modelo del requerimiento 22	267
Figura 182.	Vista del requerimiento 22	267
Figura 183.	Controlador del requerimiento 22.....	268

Figura 184.	Pruebas HU9	269
Figura 185.	Pruebas HU	270
Figura 186.	Acta de planificación del sprint 4	271
Figura 187.	Acta de revisión del sprint 4	272

PRODUCT BACKLOG

En la **tabla N1°** se evidencia la lista del Product Backlog, especificando la priorización y tiempo estimado.

Tabla 1. Product Backlog

N°	Requerimiento	Tiempo estimado
RF - 01	El sistema debe tener una ventana que permita el inicio de sesión por medio de un usuario y contraseña	2
RF - 02	El sistema debe contar con un dashboard en donde se visualice un gráfico estadístico de productos no vendidos.	2
RF - 03	El sistema debe contar con un dashboard en donde se visualice un gráfico estadístico de productos vendidos según categorías.	1
RF - 04	El sistema debe contar con un dashboard en donde se visualice un gráfico estadístico de productos con bajo stock.	1
RF - 05	El sistema debe contar con una sección de usuarios en donde se podrá gestionar la búsqueda de los mismos por carácter o roles.	2
RF - 06	El sistema debe contar con una sección de usuarios en donde se podrá crear un nuevo usuario	1
RF - 07	El sistema debe contar con una sección de usuarios en donde se podrá editar un usuario existente	1
RF - 08	El sistema debe contar con una sección de usuarios en donde se podrá eliminar un usuario existente	1

RF - 09	El sistema debe contar con una sección de usuarios en donde se podrá crear una sucursal	1
RF - 10	El sistema debe contar con una sección de productos en donde se podrá gestionar la búsqueda de los mismos por carácter y categoría	2
RF - 11	El sistema debe contar con una sección de productos en donde se podrá crear un nuevo producto	1
RF - 12	El sistema debe contar con una sección de productos en donde se podrá editar un producto existente	1
RF - 13	El sistema debe contar con una sección de productos en donde se podrá eliminar un producto existente	1
RF - 14	El sistema debe contar con una sección de productos en donde se podrá crear una nueva categoría	1
RF - 15	El sistema debe contar con una sección de productos en donde se podrá buscar una categoría existente	1
RF - 16	El sistema debe contar con una sección de ventas en donde se podrá realizar una nueva venta	2
RF - 17	El sistema debe contar con una sección de ventas en donde al realizar una venta genera un comprobante	1

RF - 18	El sistema debe contar con una sección de cancelar venta en donde se gestionará las cancelaciones de ventas	1
RF - 19	El sistema debe contar con una sección de abastecimiento en donde se podrá gestionar un nuevo abastecimiento	1
RF - 20	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas por producto	1
RF - 21	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas por vendedor	1
RF - 22	El sistema debe permitir generar un documento EXCEL con el detalle del stock de producto	1
RF - 23	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas realizadas	1
RF - 24	El sistema debe permitir generar un documento EXCEL con el detalle de los abastecimientos realizados	1
RF - 25	El sistema debe permitir generar un documento EXCEL con el detalle de la rotación de inventarios.	1
RF - 26	El sistema debe permitir generar un documento EXCEL con el detalle del porcentaje del inventario sin rotación.	1
RF - 27	El sistema debe permitir generar un documento EXCEL con el detalle de la duración del inventario en ese mes.	1

RF - 28	El sistema debe permitir generar un documento EXCEL con el detalle del valor económico del inventario.	1
---------	--	---

Fuente: Elaboración propia.

PRODUCT BACKLOG PRIORIZADO

Actúa como una hoja de ruta flexible que guía el desarrollo del proyecto, asegurando que el equipo se concentre en las tareas más relevantes y urgentes. (Huatay, 2020).

Técnica MoSCoW:

La técnica MoSCoW reconoce que no todos los requisitos de un proyecto tienen la misma relevancia. Al clasificarlos en categorías como 'imprescindibles', 'recomendables', 'opcionales' y 'excluidos', esta metodología facilita la toma de decisiones y garantiza que el desarrollo se centre en las funcionalidades que más valoran los usuarios. La principal ventaja de MoSCoW es que ofrece una escala de prioridades con un significado claro, lo que permite a los equipos de desarrollo trabajar de manera más eficiente y efectiva (Sagrado, 2018).

Tabla 2. Nivel de prioridad según MoSCoW

ÍTEM	NIVELES	CONCEPTO
M	Must Have	Funcionalidades que sí o sí debe tener la solución.
S	Should Have	Funcionalidades que son importantes, pero no obligatorias.
C	Could Have	Funcionalidades que podrían incluirse en la solución.
W	Won't have	Funcionalidades que por el momento pueden estar inhabilitadas o que se harán a futuro.

Fuente: Elaboración propia

Tabla 3. Backlog priorizado

HU	N°	Requerimiento	Prioridad
HU1	RF - 01	El sistema debe tener una ventana que permita el inicio de sesión por medio de un usuario y contraseña	M
HU2	RF - 02	El sistema debe tener un módulo dashboard en donde se visualice un gráfico estadístico de productos no vendidos.	M
	RF - 03	El sistema debe tener un módulo dashboard en donde se visualice un gráfico estadístico de productos vendidos según categorías.	M
	RF - 04	El sistema debe tener un módulo dashboard en donde se visualice una tabla de productos con bajo stock.	M
HU3	RF - 05	El sistema debe tener un módulo de usuarios en donde se podrá gestionar la búsqueda de estos por carácter o roles.	S
	RF - 06	El sistema debe tener un módulo de usuarios en donde se podrá crear un nuevo usuario	M
	RF - 07	El sistema debe tener un módulo de usuarios en donde se podrá editar un usuario existente	S
	RF - 08	El sistema debe tener un módulo de usuarios en donde se podrá eliminar un usuario existente	S
HU4	RF - 09	El sistema debe tener un módulo de usuarios en donde se podrá crear una sucursal	C

HU5	RF - 10	El sistema debe tener un módulo de productos en donde se podrá gestionar la búsqueda de estos por carácter y categoría	M
	RF - 11	El sistema debe tener un módulo de productos en donde se podrá crear un nuevo producto	M
	RF - 12	El sistema debe tener un módulo de productos en donde se podrá editar un producto existente	M
	RF - 13	El sistema debe tener un módulo de productos en donde se podrá eliminar un producto existente	M
HU6	RF - 14	El sistema debe tener un módulo de productos en donde se podrá crear una nueva categoría	M
	RF - 15	El sistema debe tener un módulo de productos en donde se podrá buscar una categoría existente	M
HU7	RF - 16	El sistema debe tener un módulo de ventas en donde se podrá realizar una nueva venta	M
	RF - 17	El sistema debe tener un módulo de ventas en donde al realizar una venta genera un comprobante	M
	RF - 18	El sistema debe tener un módulo de cancelar venta en donde se gestionará las cancelaciones de ventas	M
HU8	RF - 19	El sistema debe tener un módulo de abastecimiento en donde se podrá gestionar un nuevo abastecimiento	M
HU9	RF - 20	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas por producto	C

	RF - 21	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas por vendedor	C
	RF - 22	El sistema debe permitir generar un documento EXCEL con el detalle del stock de producto	C
	RF - 23	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas realizadas	S
	RF - 24	El sistema debe permitir generar un documento EXCEL con el detalle de los abastecimientos realizados	S
HU10	RF - 25	El sistema debe permitir generar un documento EXCEL con el detalle de la rotación de inventarios.	M
	RF - 26	El sistema debe permitir generar un documento EXCEL con el detalle del porcentaje del inventario sin rotación.	M
	RF - 27	El sistema debe permitir generar un documento EXCEL con el detalle de la duración del inventario en ese mes.	M
	RF - 28	El sistema debe permitir generar un documento EXCEL con el detalle del valor económico del inventario.	M

Fuente: Elaboración propia

PLANIFICACIÓN DE LANZAMIENTO

En la figura 1 se detalla el cronograma de la planificación y posterior lanzamiento del proyecto.

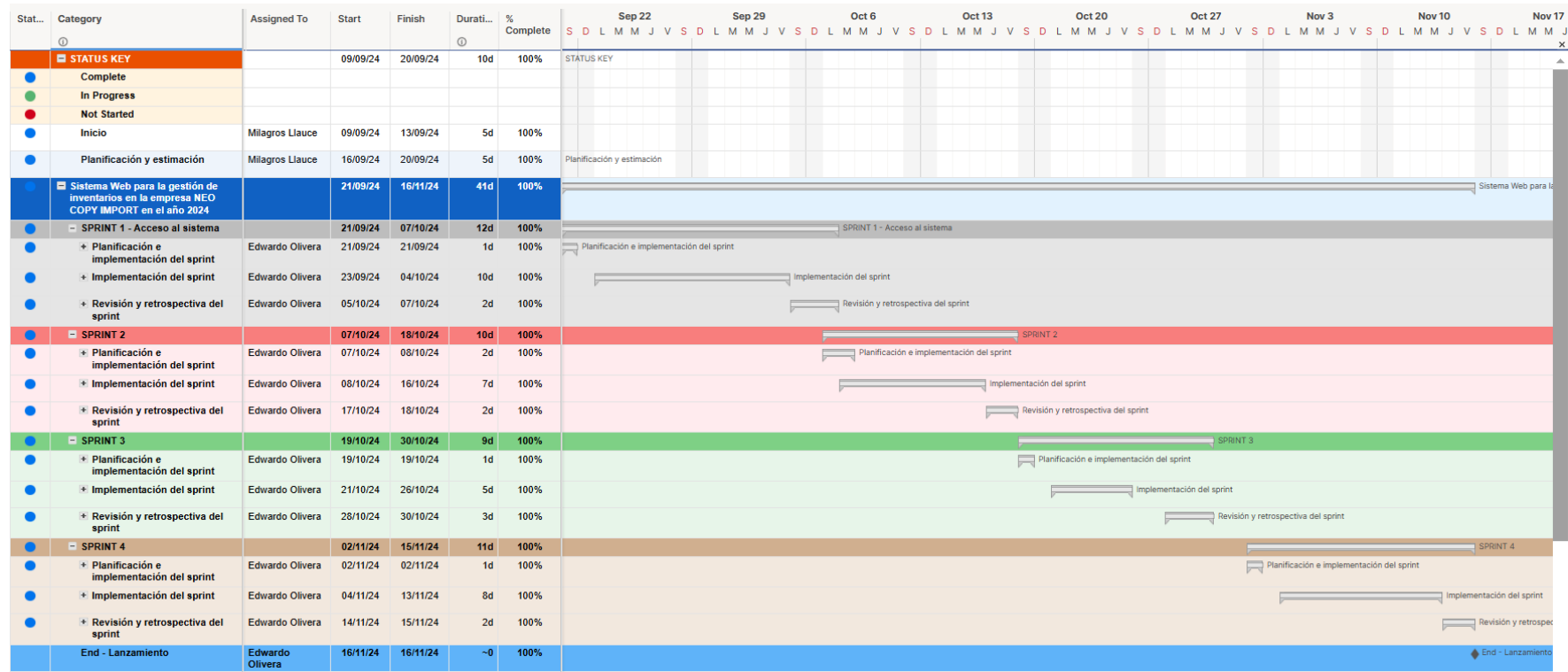


Figura 1. Cronograma de la planificación de lanzamiento.

Fuente: Elaboración propia

HISTORIAS DE USUARIO

En la tabla 4 se muestra un resumen de las historias de usuario con las fechas de inicio, encargado del desarrollo y estimación en días.

Tabla 4. Resumen historias de usuario

Código	Nombre	Fecha	Rol	Estimación
HU1	Acceso al sistema	23/09/2024	Developer	3
HU2	Administrar dashboard	24/09/2024	Developer	4
HU3	Administrar usuarios	27/09/2024	Developer	4
HU4	Administrar sucursales	2/11/2024	Developer	4
HU5	Administrar productos	28/09/2024	Developer	4
HU6	Administrar categoría	3/10/2024	Developer	4
HU7	Administrar ventas	8/10/2024	Developer	5
HU8	Administrar abastecimiento	11/10/2024	Developer	4
HU9	Administrar reportes opcionales	6/11/2024	Developer	4
HU10	Administrar reportes obligatorios	12/10/2024	Developer	4

Fuente: Elaboración propia

La figura 2 muestra la Historia N° 1, incluyendo su denominación, los usuarios involucrados, la estimación del tiempo en puntos, una descripción y los escenarios de validación.

Número: HU-1	Como: Usuario del sistema
Nombre historia: Acceso al sistema	
Prioridad en negocio: Alta	Riesgo en desarrollo: Baja
Puntos estimados: 5	Roles: Super Administrador / Administrador / Vendedor
Programador responsable: Eduardo Olivera	
Descripción: Como usuario del sistema, quiero ingresar al sistema para usar las funcionalidades del sistema.	
Criterios de Aceptación: • El sistema valida las credenciales y permite el acceso si son correctas. • El sistema muestra un mensaje de error si las credenciales son incorrectas. • El usuario al ingresar puede ver las funciones o la información a la que tiene permitido acceder según el rol que posee. • El sistema almacena las contraseñas de forma segura y cifradas.	
Validación: Escenario 1: Ingreso Exitoso Dado que, me encuentro en el formulario de login, Cuando ingreso usuario y contraseña de manera correcta Entonces el sistema permite el ingreso. Escenario 2: Ingreso Fallido Dado que, me encuentro en el formulario de login, Cuando ingreso usuario y contraseña de manera incorrecta, Entonces aparece el mensaje "Error! Las credenciales o el usuario no son correctos..."	

Figura 2. Historia de usuario N°1

Fuente: Elaboración propia

La figura 3 muestra la Historia N° 2, incluyendo su denominación, los usuarios involucrados, la estimación del tiempo en puntos, una descripción y los escenarios de validación.

Número: HU-2	Como: Usuario del sistema
Nombre historia: Administrar dashboard	
Prioridad en negocio: Alta	Riesgo en desarrollo: Baja
Puntos estimados: 5	Roles: Super Administrador / Administrador / Vendedor
Programador responsable: Eduardo Olivera	
Descripción: Como usuario del sistema, quiero visualizar el dashboard para consultar las estadísticas de productos no vendidos, productos vendidos según categorías y se debe visualizar una tabla de productos con bajo stock	
Criterios de Aceptación: • El usuario debe poder acceder al dashboard utilizando sus credenciales de inicio de sesión. • El sistema debe verificar la identidad del usuario y permitir el acceso solo a los usuarios registrados. • El dashboard debe mostrar datos de acuerdo al rol del usuario. • El dashboard debe cargar y mostrar los datos de manera rápida y eficiente.	
Validación: Escenario 1: Visualizar datos específicos Dado que , me encuentro en la pestaña del Dashboard, Cuando filtro por fecha de inicio, fecha de fin y sucursal, Entonces me brinda datos específicos. Escenario 2: Visualizar datos predeterminados Dado que , me encuentro en la pestaña del Dashboard Cuando no ingreso ningún valor en los filtros, Entonces aparece datos de manera predeterminada.	

Figura 3. Historia de usuario N°2

Fuente: Elaboración propia

La figura 4 muestra la Historia N° 3, incluyendo su denominación, los usuarios involucrados, la estimación del tiempo en puntos, una descripción y los escenarios de validación.

Número: HU-3	Como: Usuario del sistema
Nombre historia: Administrar Usuarios	
Prioridad en negocio: Alta	Riesgo en desarrollo: Baja
Puntos estimados: 5	Roles: Super Administrador / Administrador / Vendedor
Programador responsable: Eduardo Olivera	
Descripción: Como usuario del sistema, quiero tener una sección de usuarios para crear, editar, eliminar y listar los usuarios.	
Criterios de Aceptación: • El administrador puede buscar y filtrar usuarios por nombre, rol y sucursal. • El sistema solicita una confirmación antes de crear y eliminar un usuario. • El sistema muestra mensajes de error y confirmación concisos y claros. • El administrador puede crear, modificar, eliminar nuevos usuarios al sistema. • El sistema valida que los campos obligatorios estén completos y en formatos válidos. • El usuario se elimina de la base de datos y ya no puede acceder al sistema.	
Validación: Escenario 1: Búsqueda de usuario Dado que, me encuentro en el módulo de usuarios, Cuando filtro por rol, sucursal o ingreso datos, Entonces el sistema lista un resultado. Escenario 2: No se especifican criterios de filtrado Dado que, me encuentro en el módulo de usuarios, Cuando no ingreso ningún dato o filtro de rol y sucursal, Entonces el sistema lista todos los usuarios existentes. Escenario 3: Agregar usuarios con éxito Dado que, me encuentro en el módulo de usuarios, Cuando doy clic en "Crear Usuario" e ingreso data en todos los campos, Entonces en el sistema figura un pop-up indicando "Usuario Creado" Escenario 4: Error al agregar un usuario Dado que, me encuentro en el módulo de productos, Cuando doy clic en "Crear Usuario" y no ingreso ningún dato, Entonces el sistema indica "Debe ingresar datos en los campos para su creación" Escenario 5: Eliminar usuarios con éxito Dado que, me encuentro en el módulo de usuarios, Cuando doy click en el botón "Borrar" y posteriormente hago click en "Eliminar" Entonces el sistema indica "Producto Eliminado con éxito" Escenario 6: Modificar un usuario Dado que, me encuentro en el módulo de usuarios, Cuando doy click en el botón "Modificar" y edito los campos, Entonces el sistema indica "Usuario Actualizado".	

Figura 4. Historia de usuario N°3

Fuente: Elaboración propia

La figura 5 muestra la Historia N° 4, incluyendo su denominación, los usuarios involucrados, la estimación del tiempo en puntos, una descripción y los escenarios de validación.

Número: HU-4	Como: Usuario del sistema
Nombre historia: Administrar Sucursales	
Prioridad en negocio: Alta	Riesgo en desarrollo: Baja
Puntos estimados: 6	Roles: Super Administrador / Administrador / Vendedor
Programador responsable: Eduardo Olivera	
Descripción: Como usuario del sistema, quiero tener una sección de sucursales para crear, editar, eliminar y listar las sucursales	
Criterio de Aceptación: • El administrador puede buscar sucursales por nombre, ruc, dirección, o cualquier otro campo relevante. • El administrador puede crear, modificar, eliminar nuevas sucursales al sistema. • El sistema valida que los campos obligatorios estén completos y en formatos válidos. • El sistema valida los cambios realizados antes de guardar.	
Validación: Escenario 1: Listar sucursales con éxito Dado que, me encuentro en el módulo de usuarios, Cuando doy clic en "Sucursal", Entonces el sistema lista las sucursales de la empresa. Escenario 2: Agregar una sucursal con éxito Dado que, me encuentro en el módulo de usuarios, Cuando doy clic en "Sucursal" y en "Crear" e ingreso data en los campos, Entonces en el sistema figura un pop-up: "Sucursal agregada con éxito" Escenario 3: Editar una sucursal con éxito Dado que, me encuentro en el módulo de usuarios, Cuando doy clic en "Sucursal", posteriormente en "Editar" y edito los campos necesarios, Entonces el sistema figura un pop-up: "Sucursal editada con éxito" Escenario 4: Eliminar una sucursal con éxito Dado que, me encuentro en el módulo de usuarios, Cuando doy clic en "Sucursal", posteriormente en "Eliminar" Entonces el sistema figura un pop-up: "Sucursal eliminada con éxito"	

Figura 5. Historia de Usuario N°4

Fuente: Elaboración propia

La figura 6 muestra la Historia N° 5, incluyendo su denominación, los usuarios involucrados, la estimación del tiempo en puntos, una descripción y los escenarios de validación.

Número: HU-5	Como: Usuario del sistema
Nombre historia: Administrar Productos	
Prioridad en negocio: Alta	Riesgo en desarrollo: Baja
Puntos estimados: 5	Roles: Super Administrador / Administrador
Programador responsable: Eduardo Olivera	
Descripción: Como usuario del sistema, quiero tener una sección de productos para agregar, editar, eliminar y listar los usuarios.	
Criterio de Aceptación: • El administrador puede buscar productos por nombre, código, categoría u otros campos relevantes. • El sistema valida que los campos obligatorios estén completos y en formatos válidos. • Al guardar un producto, la interfaz debe mostrar una notificación de éxito. • El administrador puede modificar cualquier campo editable del producto. • El sistema solicita una confirmación antes de crear y eliminar un producto. • El sistema muestra mensajes de error y conformación de forma clara y concisa.	
Validación: Escenario 1: Agregar productos con éxito Dado que, me encuentro en el módulo de productos, Cuando doy clic en "Crear Producto" e ingreso data en todos los campos, Entonces en el sistema figura un pop-up indicando "Producto Creado" Escenario 2: Error al agregar un producto Dado que, me encuentro en el módulo de productos, Cuando doy clic en "Crear Producto" y no ingreso ningún dato, Entonces el sistema indica "Se debe ingresar datos para su creación" Escenario 3: Modificar un producto Dado que, me encuentro en el módulo de productos, Cuando doy click en el botón "Modificar" y edito los campos, Entonces el sistema indica "Producto Actualizado" Escenario 4: Eliminar productos con éxito Dado que, me encuentro en el módulo de productos, Cuando doy click en el botón "Borrar" y posteriormente hago click en "Eliminar" Entonces el sistema indica "Producto Eliminado"	

Figura 6. Historia de Usuario N°5

Fuente: Elaboración propia

La figura 7 muestra la Historia N° 6, incluyendo su denominación, los usuarios involucrados, la estimación del tiempo en puntos, una descripción y los escenarios de validación.

Número: HU-6	Como: Usuario del sistema
Nombre historia: Administrar Categorías	
Prioridad en negocio: Alta	Riesgo en desarrollo: Baja
Puntos estimados: 5	Roles: Super Administrador / Administrador
Programador responsable: Edwardo Olivera	
Descripción: Como usuario del sistema, quiero tener una sección de categorías para crear una nueva categoría.	
Criterio de Aceptación: • La lista de categorías muestra información esencial como los nombres de cada categoría. • El sistema valida que los campos obligatorios estén completos y en formatos válidos. • Cuando se crea una categoría, debe mostrar una notificación de éxito. • El administrador puede acceder a la edición de una categoría desde la lista. • El sistema valida los cambios antes de guardar. • El sistema solicita una confirmación antes de eliminar una categoría.	
Validación: Escenario 1: Crear nueva categoría Dado que, me encuentro en el módulo de productos, Cuando doy click en "Crear" e ingreso el nombre de la categoría, Entonces el sistema lista la nueva categoría. Escenario 2: No se especifican criterios de filtrado Dado que, me encuentro en el módulo de productos, Cuando doy click en "Crear" y no ingreso ningún dato, Entonces aparece el error "Se debe ingresar un nombre*." Escenario 3: Buscar una categoría Dado que, me encuentro en el módulo de productos, Cuando doy click en "Categorías", Entonces el sistema muestra todas las categorías existentes.	

Figura 7. Historia de Usuario N°6

Fuente: Elaboración propia

La figura 8 muestra la Historia N° 7, incluyendo su denominación, los usuarios involucrados, la estimación del tiempo en puntos, una descripción y los escenarios de validación.

Número: HU-7	Como: Usuario del sistema
Nombre historia: Administrar Ventas	
Prioridad en negocio: Alta	Riesgo en desarrollo: Baja
Puntos estimados: 5	Roles: Super Administrador / Administrador / Vendedor
Programador responsable: Eduardo Olivera	
Descripción: Como usuario del sistema, quiero tener una sección de categorías para generar y eliminar una venta.	
Criterio de Aceptación: • Los usuarios pueden generar una venta ingresando todos los campos obligatorios. • Al generar una venta, el sistema debe actualizar el inventario, reduciendo la cantidad de productos vendidos. • La venta debe ser registrada en la base de datos con todos los detalles relevantes (cliente, productos, fecha, total, etc.). • El sistema debe solicitar una confirmación antes de eliminar una venta. • El sistema debe permitir generar un PDF con el resumen de la venta.	
Validación: Escenario 1: Generar una venta Dado que, me encuentro en el módulo de ventas, Cuando edito todos los campos, busco un producto y selecciono la cantidad que deseo vender y le doy click en "Crear" Entonces el sistema indica "Venta concretada" Escenario 2: Eliminar una venta Dado que, me encuentro en el módulo de Ventas realizadas, Cuando selecciono un producto y le doy click en "Borrar" Entonces el sistema indica "Venta anulada".	

Figura 8. Historia de Usuario N°7

Fuente: Elaboración propia

La figura 9 muestra la Historia N° 8, incluyendo su denominación, los usuarios involucrados, la estimación del tiempo en puntos, una descripción y los escenarios de validación.

Número: HU-8	Como: Usuario del sistema
Nombre historia: Administrar abastecimiento	
Prioridad en negocio: Alta	Riesgo en desarrollo: Baja
Puntos estimados: 5	Roles: Super Administrador / Administrador
Programador responsable: Eduardo Olivera	
Descripción: Como usuario del sistema, quiero tener una sección de abastecimiento para abastecer los productos existentes.	
Criterio de Aceptación: • El sistema debe permitir generar un abastecimiento seleccionando los campos como proveedor y el producto. • El sistema debe calcular automáticamente el total del abastecimiento. • El sistema debe permitir el ingreso de la cantidad solicitada para cada producto. • El sistema debe validar los cambios antes de guardar.	
Validación: Escenario 1: Abastecer stock de productos Dado que, me encuentro en el módulo de Abastecimientos Cuando selecciono un proveedor, categoría y busco un producto, por último, doy clic en "Crear" Entonces el sistema indica "Stock agregado con éxito"	

Figura 9. Historia de Usuario N°8

Fuente: Elaboración propia

La figura 10 muestra la Historia N° 9, incluyendo su denominación, los usuarios involucrados, la estimación del tiempo en puntos, una descripción y los escenarios de validación.

Número: HU-9	Como: Usuario del sistema
Nombre historia: Administrar reportes opcionales	
Prioridad en negocio: Alta	Riesgo en desarrollo: Baja
Puntos estimados: 5	Roles: Super Administrador / Administrador
Programador responsable: Eduardo Olivera	
Descripción: Como usuario del sistema, quiero tener una sección de reportes para administrar todos los reportes opcionales del sistema.	
Criterio de Aceptación: - El sistema debe generar un archivo Excel que contenga la información del reporte. - Los usuarios deben poder acceder a los detalles completos de los productos, ventas realizadas, productos vendidos, abastecimientos de cada producto, entre otros. - La interfaz permite a los usuarios encontrar cualquier información mediante los distintos reportes de ventas, productos, stock, entre otros. - La interfaz debe ser intuitiva y fácil de usar.	
Validación: Escenario 1: Generar un documento de productos vendidos Dado que, me encuentro en el módulo de Reportes de Productos Vendidos, Cuando selecciono la categoría, sucursal y las fechas, doy click en "Generar Excel" Entonces el sistema generará un reporte de los productos vendidos. Escenario 2: Generar un reporte de ventas realizadas por vendedor Dado que, me encuentro en el módulo de Reportes de Ventas realizadas, Cuando selecciono el vendedor, sucursal y las fechas, doy click en "Generar Excel" Entonces el sistema generará un reporte de las ventas realizadas por vendedor filtrado. Escenario 3: Generar un reporte de stock de productos. Dado que, me encuentro en el módulo de Stock de productos Cuando selecciono la sucursal y las fechas, categoría, y busco el producto y doy click en "Generar Excel" Entonces el sistema generará un reporte de stock de productos filtrado. Escenario 4: Generar un reporte de abastecimiento de un producto. Dado que, me encuentro en el módulo de Reporte de Abastecimiento, Cuando selecciono el proveedor, sucursal, fechas y doy clic en "Generar Excel", Entonces el sistema generará un reporte de Abastecimiento de uno o varios productos. Escenario 5: Generar un reporte de ventas realizadas. Dado que, me encuentro en el módulo de Ventas realizadas, Cuando selecciono el vendedor, la sucursal y las fechas, categoría y doy click en "Generar Excel" Entonces el sistema generará un reporte de ventas realizadas. Escenario 6: Generar un reporte de ventas realizadas por producto Dado que, me encuentro en el módulo de Reportes de Productos Vendidos, Cuando selecciono la categoría, sucursal y las fechas, doy click en "Generar Excel" Entonces el sistema generará un reporte de las ventas realizadas por producto filtrado.	

Figura 10. Historia de Usuario N°9

Elaboración propia

La figura 11 muestra la Historia N° 10, incluyendo su denominación, los usuarios involucrados, la estimación del tiempo en puntos, una descripción y los escenarios de validación.

Número: HU-10	Como: Usuario del sistema
Nombre historia: Administrar reportes obligatorios	
Prioridad en negocio: Alta	Riesgo en desarrollo: Baja
Puntos estimados: 5	Roles: Super Administrador / Administrador
Programador responsable: Eduardo Olivera	
Descripción: Como usuario del sistema, quiero tener una sección de reportes para administrar todos los reportes obligatorios del sistema.	
Criterio de Aceptación: • El sistema debe generar un archivo Excel que contenga la información del reporte. • El sistema debe realizar cálculos para cada reporte dentro de los criterios seleccionados. • Los usuarios deben poder acceder a los detalles completos de cada reporte. • La interfaz permite a los usuarios encontrar cualquier información mediante los distintos reportes como rotación de inventarios, porcentaje de rotación de inventario, duración de inventario y el valor económico del inventario. • Los campos de año y mes deben permitir seleccionar rangos de fechas válidos.	
Validación: Escenario 1: Generar un Reporte de rotación de inventario mensual. Dado que , me encuentro en el módulo de rotación de inventario mensual, Cuando selecciono categoría, sucursal, año y mes, doy click en "Generar Reporte", Entonces el sistema generará un reporte de rotación de inventario mensual. Escenario 2: Generar un reporte de rotación de inventario Mensual en porcentaje Dado que , me encuentro en el módulo rotación de inventario Mensual en porcentaje, Cuando selecciono la categoría, sucursal, año y mes doy clic en "Generar Excel" Entonces el sistema generará un reporte de rotación de inventario Mensual en porcentaje. Escenario 3: Generar un reporte de duración del inventario Mensual Dado que , me encuentro en el módulo de Reporte de duración del inventario Mensual Cuando selecciono la categoría, sucursal, año y mes, doy click en "Generar Excel" Entonces el sistema generará un reporte de duración del inventario Mensual. Escenario 4: Generar un reporte de Porcentaje de valor económico de inventario. Dado que , me encuentro en el módulo de Reporte de valor económico del inventario Mensual, Cuando selecciono la categoría, sucursal, año y mes, doy click en "Generar Excel" Entonces el sistema generará un reporte de Porcentaje de valor económico de inventario.	

Figura 11. Historia de Usuario N°10

Fuente: Elaboración propia

TAREAS POR HISTORIA DE USUARIO

En la tabla 5 se muestran las tareas de cada historia de usuario y su estimación.

Tabla 5. Tareas por historia de usuario

#	Historia de usuario	Tareas	Tiempo estimado
HU1	Acceso al sistema	Crear Login	3
HU2	Administrar Dashboard	Hacer gráficos estadísticos	3
		Hacer tabla informativa de productos con bajo stock	1
HU3	Administrar usuarios	Listar usuarios	1
		Registrar usuario	1
		Editar usuario	1
		Eliminar usuario	1
HU4	Administrar sucursales	Registrar sucursal	4
HU5	Administrar productos	Listar productos	1
		Registrar productos	1
		Editar productos	1
		Eliminar productos	1
HU6	Administrar categoría	Registrar categoría	2
		Listar categoría	2
HU7	Administrar ventas	Realizar ventas	2
		Generar comprobante después de una venta	1
		Generar cancelaciones de ventas	2

HU8	Administrar abastecimiento	Realizar abastecimiento	4
HU9	Administrar reportes opcionales	Reporte ventas por producto	0,5
		Reporte de las ventas por vendedor	0,5
		Reporte con el detalle del stock de producto	1
		Reporte con el detalle de las ventas realizadas	1
		Reporte con el detalle de los abastecimientos realizados	1
HU10	Administrar reportes obligatorios	Reporte con el detalle de la rotación de inventarios.	1
		Reporte con el detalle del porcentaje del inventario sin rotación.	1
		Reporte con el detalle de la duración del inventario en ese mes.	1
		Reporte con el detalle del valor económico del inventario.	1

Fuente: Elaboración propia

CRONOGRAMA DEL SPRINT

En la figura 12 se detalla la estructura del cronograma del Sprint 1

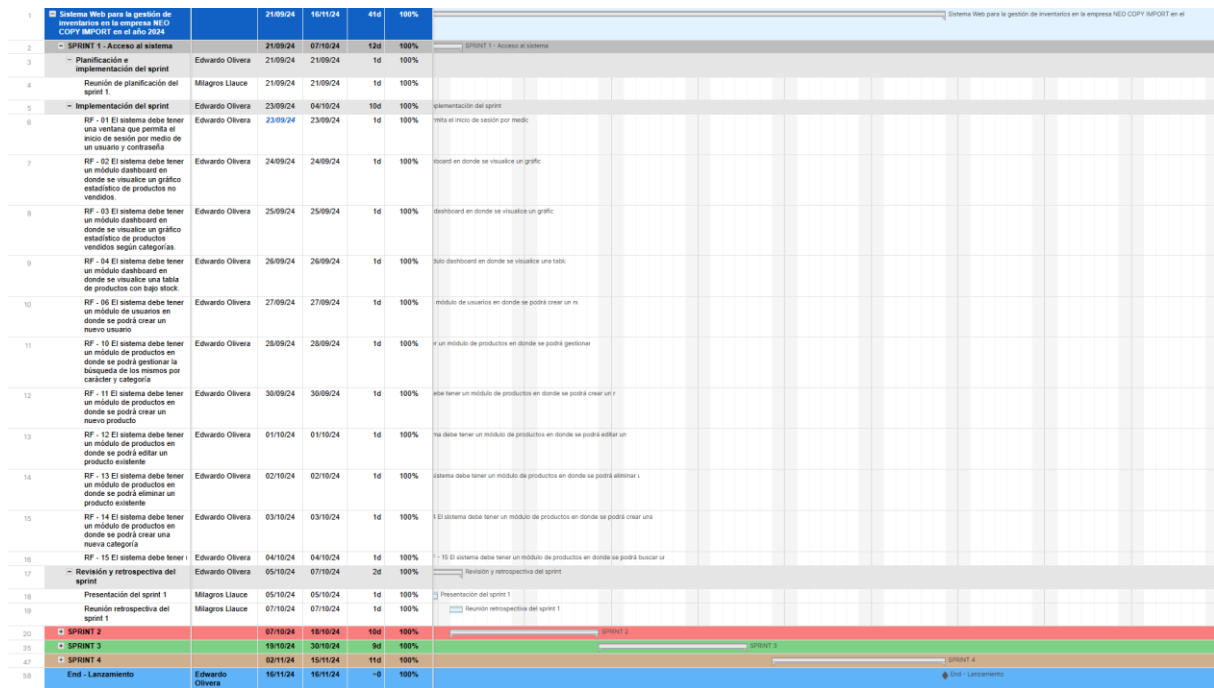


Figura 12. Cronograma del sprint 1

Fuente: Elaboración propia

En la figura 13 se detalla la estructura del cronograma del Sprint 2

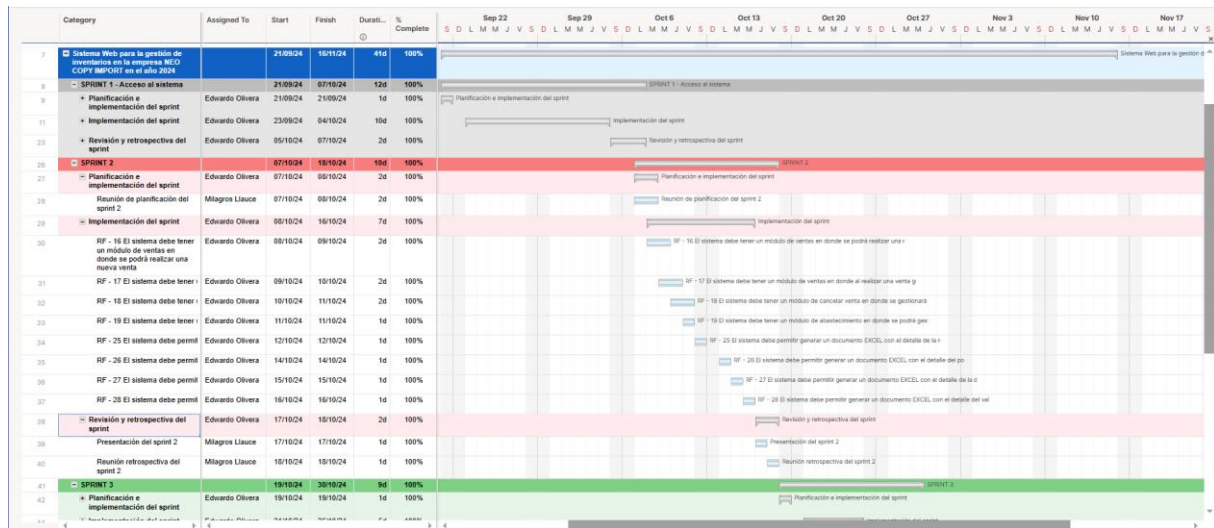


Figura 13. Cronograma del sprint 2

Fuente: Elaboración propia

En la figura 14 se detalla la estructura del cronograma del Sprint 3.

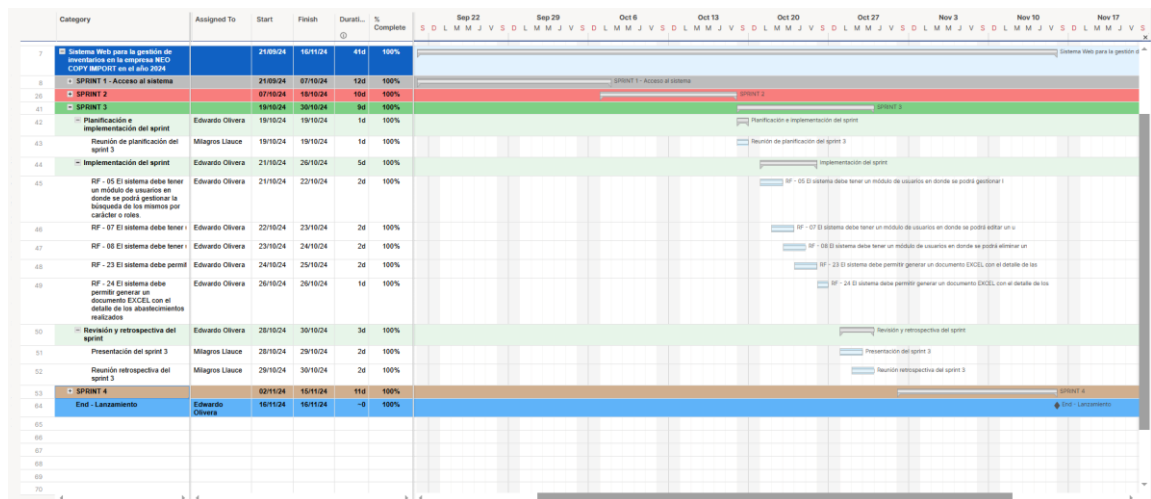


Figura 14. Cronograma del sprint 3

Fuente: Elaboración propia

En la figura 15 se detalla la estructura del cronograma del Sprint 4

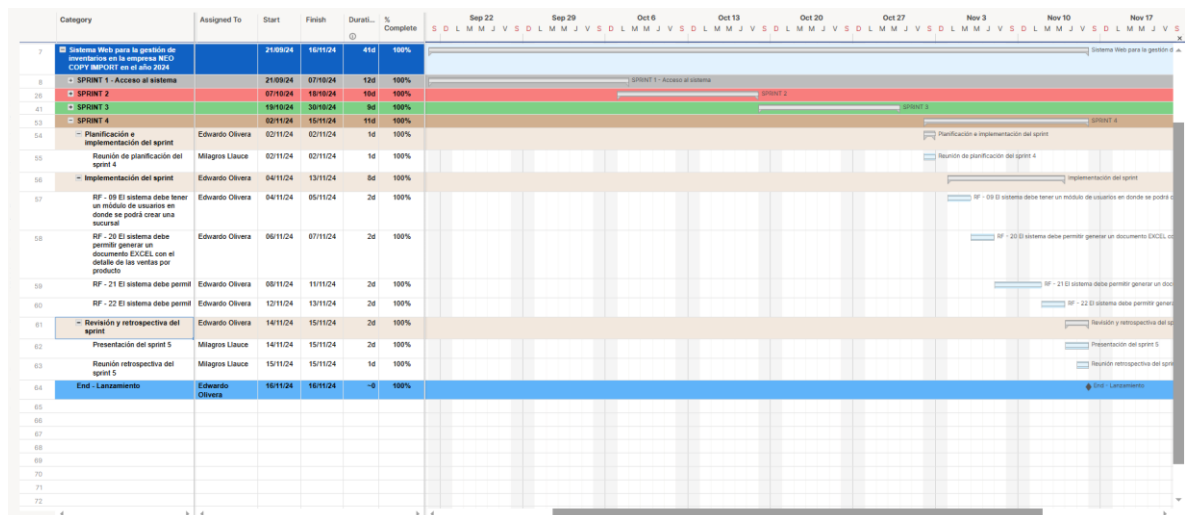


Figura 15. Cronograma del sprint 4

Fuente: Elaboración propia

SPRINT 1

El Sprint 1 se completó siguiendo un proceso documentado que abarcó las cuatro fases de desarrollo: análisis, diseño, implementación y pruebas.

ANÁLISIS DE REQUERIMIENTOS SPRINT 1

En la **tabla 6** se muestran los 11 requerimientos que conforman el sprint 1.

Tabla 6. Requerimientos del Sprint 1

HU	Nº	Requerimiento
HU1	RF - 01	El sistema debe tener una ventana que permita el inicio de sesión por medio de un usuario y contraseña
HU2	RF - 02	El sistema debe tener un módulo dashboard en donde se visualice un gráfico estadístico de productos no vendidos.
	RF - 03	El sistema debe tener un módulo dashboard en donde se visualice un gráfico estadístico de productos vendidos según categorías.
	RF - 04	El sistema debe tener un módulo dashboard en donde se visualice una tabla de productos con bajo stock.
HU3	RF - 06	El sistema debe tener un módulo de usuarios en donde se podrá crear un nuevo usuario
HU5	RF - 10	El sistema debe tener un módulo de productos en donde se podrá gestionar la búsqueda de estos por carácter y categoría
	RF - 11	El sistema debe tener un módulo de productos en donde se podrá crear un nuevo producto
	RF - 12	El sistema debe tener un módulo de productos en donde se podrá editar un producto existente
	RF - 13	El sistema debe tener un módulo de productos en donde se podrá eliminar un producto existente
HU6	RF - 14	El sistema debe tener un módulo de productos en donde se podrá crear una nueva categoría

	RF - 15	El sistema debe tener un módulo de productos en donde se podrá buscar una categoría existente
--	---------	---

Fuente: Elaboración propia

FUNCIONALIDADES DEL PRIMER SPRINT

La **Tabla 7** ofrece una descripción completa de las funcionalidades que implementará el sistema. Además, ilustra cómo interactúan los usuarios con la arquitectura de software para satisfacer cada uno de los requerimientos definidos en el Sprint 1.

Tabla 7. Funcionalidades del primer sprint

R.F.	Funcionalidad
RF-01	El usuario introduce su usuario y su contraseña en los campos respectivos para posteriormente dar clic en el botón Login. A continuación, al hacer clic en el botón "Iniciar sesión", se envía una solicitud al controlador. Este mismo actúa como intermediario, toma la información proporcionada por el usuario y la envía al modelo. El modelo es el responsable de interactuar con la base de datos para verificar si las credenciales son válidas. Completada la validación, se envía el resultado de vuelta al controlador. Con esta información, el controlador procede a actualizar la vista para que el usuario pueda acceder al dashboard con los componentes asignados según el rol.
RF-02	Al acceder al dashboard, el usuario visualizará un gráfico circular que mostrará la proporción de productos más vendidos. Esta representación visual le permitirá identificar rápidamente los productos con mayor y menor demanda, lo cual facilitará la toma de decisiones informadas sobre la gestión del inventario, como, por ejemplo, ajustar los niveles de stock, priorizar la reposición de productos clave o identificar aquellos con baja rotación que podrían requerir estrategias

	de promoción. El modelo obtiene y procesa los datos de ventas, la vista presenta el gráfico de forma interactiva con la ayuda de una librería especializada, y el controlador coordina la comunicación entre el modelo y la vista para que el usuario pueda visualizar la información y tomar decisiones sobre la gestión de inventario.
RF-03	Al acceder al dashboard, el usuario visualizará un gráfico circular que mostrará la proporción de productos más vendidos en cada categoría. Esta representación visual le permitirá identificar rápidamente cuáles son las categorías con mayor y menor demanda, lo cual puede ser útil para la toma de decisiones estratégicas en la empresa, como, por ejemplo, optimizar la gestión de inventario, identificar nuevas oportunidades de mercado o ajustar las estrategias de marketing. El modelo obtiene y procesa los datos de ventas, la vista presenta el gráfico de forma interactiva con la ayuda de una librería especializada, y el controlador coordina la comunicación entre el modelo y la vista para que el usuario pueda visualizar la información y tomar decisiones sobre la gestión de inventario.
RF-04	Al acceder al dashboard, el usuario visualizará una lista de productos con bajo stock, lo que le permitirá tomar medidas para ajustar el inventario y evitar la falta de existencias. Esta funcionalidad es crucial para asegurar la continuidad de las operaciones. Con el modelo se obtendrá la información de los productos y sus niveles de stock desde la base de datos. La vista se encargará de presentar la lista de los productos con bajo stock, y el controlador coordina la comunicación entre el modelo y la vista para que el usuario pueda visualizar la información y tomar decisiones sobre el bajo stock.

RF-06	El usuario al acceder al módulo de Usuarios visualizará un listado de los usuarios activos que maneja el aplicativo web. Al dar clic en el botón “Crear Usuario” se abrirá una nueva vista con los campos necesarios para registrar un nuevo usuario en el sistema. Los datos que se completaron son capturados para ser enviados al controlador para realizar un insert dentro de la base de datos y manda los datos al modelo para que se genere un nuevo registro dentro de la tabla usuarios.
RF-10	El usuario al acceder al módulo de productos visualizará un listado de los productos activos que maneja el sistema. Al seleccionar alguna categoría del combo box se filtrará por categoría. Con esto, el modelo se encargará de la interacción con el repositorio de datos donde se persiste la información de los productos, implementando la lógica para la construcción de consultas que permitan el filtrado de las entidades. La vista, haciendo uso de un mecanismo de enlace de datos, se encargará de la renderización de la colección de productos filtrada. Finalmente, el controlador, actuando como intermediario, gestionará la invocación de métodos del modelo y la actualización del estado de la vista.
RF-11	El usuario al acceder al módulo de productos visualizará un listado de los productos activos que maneja el sistema. Al hacer click en el botón “Crear Producto” abrirá una nueva vista con los campos necesarios para generar un nuevo registro. Cuando se hace click en el botón “Crear”, se realiza una captura de la información contenida en los campos, la cual es previamente validada. Estos datos son enviados al controlador, que se encarga de mapear la información a la estructura

	del modelo de datos y la envía al modelo para que persista la nueva entidad en el repositorio de datos (tabla productos). Posteriormente el modelo enviará una respuesta al controlador, indicando el éxito o fracaso de la operación. El controlador, a su vez, enviará una respuesta a la vista, la cual redirigirá al usuario a la vista de listado de productos.
RF-12	El usuario al acceder al módulo de productos visualizará un listado de los productos activos que maneja el sistema. Para editar un producto, el usuario hace clic en el botón "Editar", enviando el ID del producto al controlador, que solicita al modelo los datos del producto con ese ID. El modelo accede al repositorio de datos y recupera la información del producto. Una vez que el usuario realiza las modificaciones, se llama al método del controlador para guardar los cambios, enviando los datos actualizados al modelo, que actualiza la información en el repositorio de datos.
RF-13	El usuario al acceder al módulo de productos visualizará un listado de los productos activos que maneja el sistema. Para eliminar un producto, el usuario hace click en el botón "Eliminar" dentro de la tabla, lo que captura el ID del producto y lo envía al controlador. El controlador llama al método de eliminar del modelo, que cambia el estado del producto en la base de datos, ocultándolo de la vista del usuario.
RF-14	El usuario al acceder al módulo de productos visualizará un listado de los productos activos que maneja el sistema. Al hacer clic en "Categorías" abrirá una nueva vista listando todas las categorías existentes hasta ese momento. Luego, al hacer clic en "Crear", se abre otra vista con los campos para crear un nuevo registro. Una vez

	completados los campos, el modelo interactúa con la base de datos para guardar la nueva categoría.
RF-15	El usuario al acceder al módulo de productos visualizará un listado de los productos activos que maneja el sistema. Al hacer clic en “Categorías” abrirá una nueva vista listando todas las categorías existentes hasta ese momento.

Fuente: Elaboración propia

DISEÑO DEL PRIMER SPRINT

Diseño de Prototipos:

En la Figura 16 se muestra el prototipo del requerimiento funcional 01 la cual fue aprobado por el Product Owner para el login del sistema.

El prototipo muestra una ventana de navegador titulada "Sistema de Inventario". La barra de direcciones contiene la URL "https://neocopyimport.com/login". El contenido principal es un formulario de login con el título "Login". El formulario incluye dos campos de texto: "Usuario" con el placeholder "<username>" y "Contraseña" con el placeholder "<password>". Debajo de estos campos hay un botón rectangular etiquetado "Ingresar".

Figura 16. Prototipo para el requerimiento RF-01.

Fuente: Elaboración propia

The image shows a login form titled "Login" in bold black text. Below the title, there are two input fields. The first is labeled "Email" and contains the placeholder text "User Name". The second is labeled "Contraseña" and contains the placeholder text "*****". Below these fields is a dark blue button with the text "Login" in white.

Figura 17. Login

Fuente: Elaboración propia

En la Figura 18 se muestra el prototipo del requerimiento funcional 02 la cual fue aprobado por el Product Owner para la vista de un dashboard en donde se visualice un gráfico estadístico de productos más vendidos.

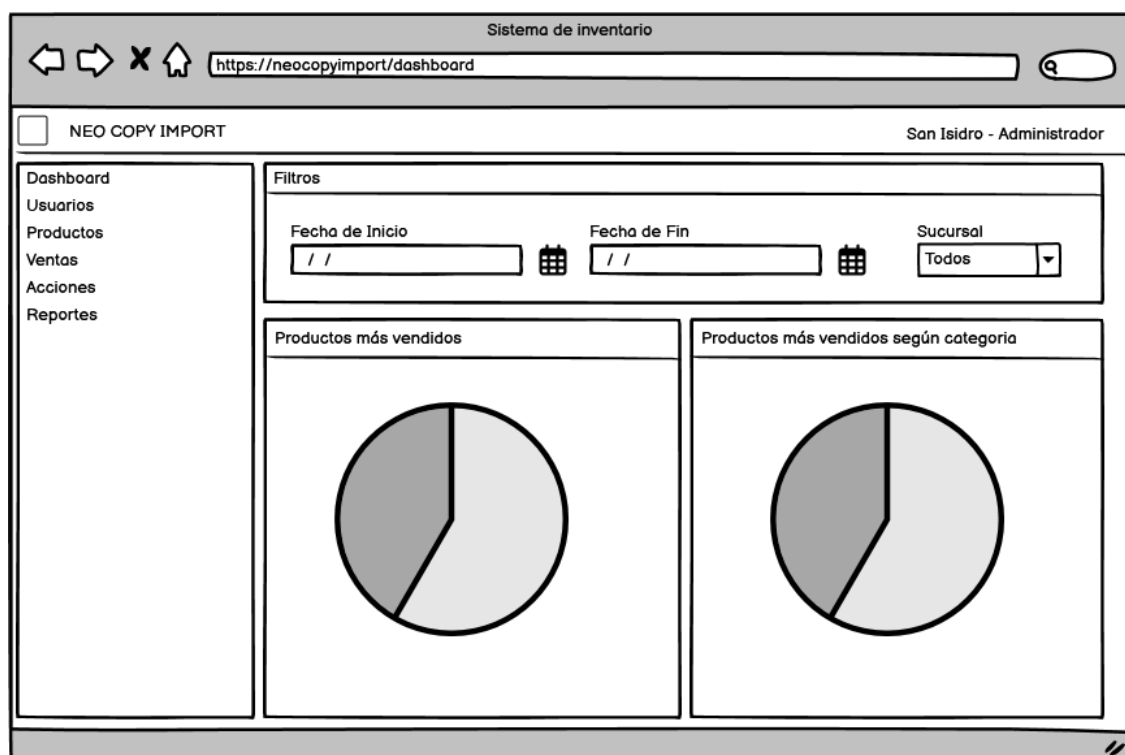


Figura 18. Prototipo para el requerimiento RF-02.

Fuente: Elaboración propia

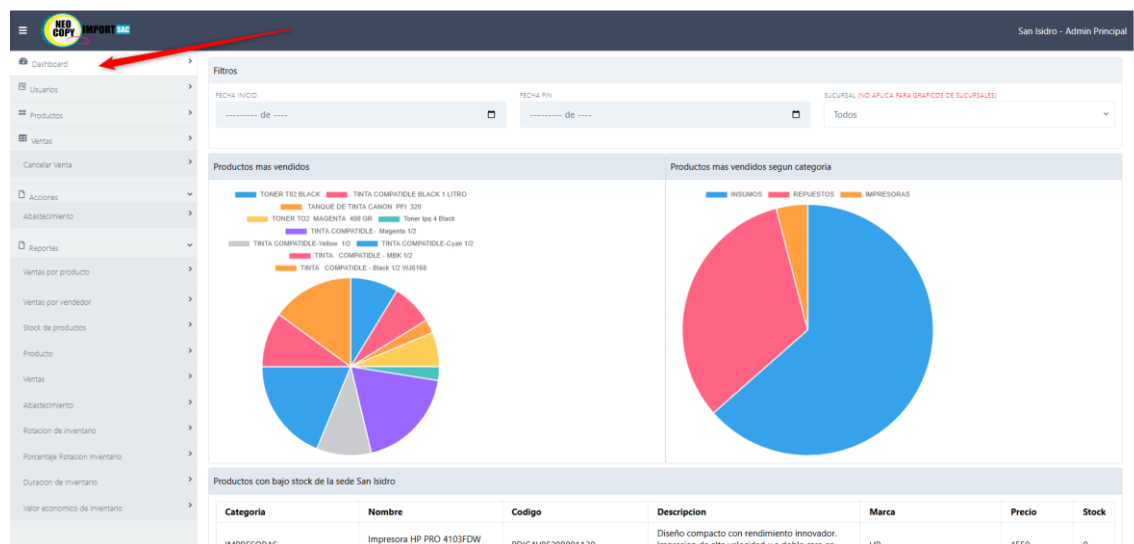


Figura 19. Dashboard – productos más vendidos

Fuente: Elaboración propia

En la Figura 20 se muestra el prototipo del requerimiento funcional 03 la cual fue aprobado por el Product Owner para la vista de un dashboard en donde se visualice un gráfico estadístico de productos más vendidos según categoría

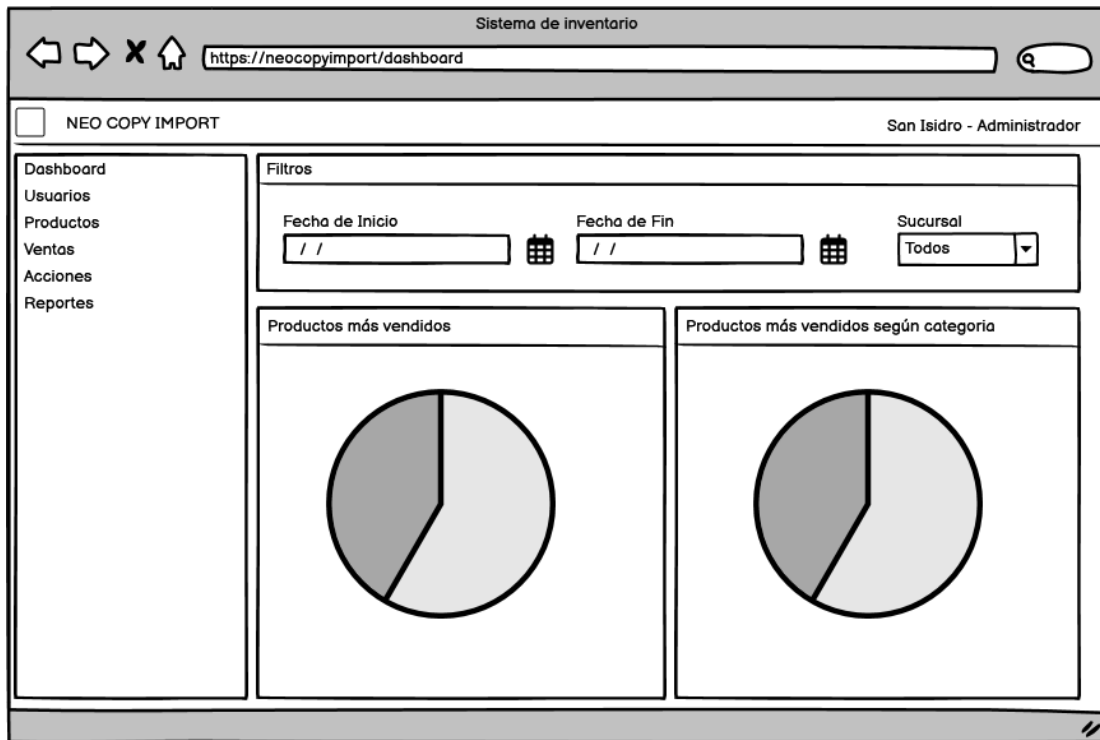


Figura 20. Prototipo para el requerimiento RF-03.

Fuente: Elaboración propia

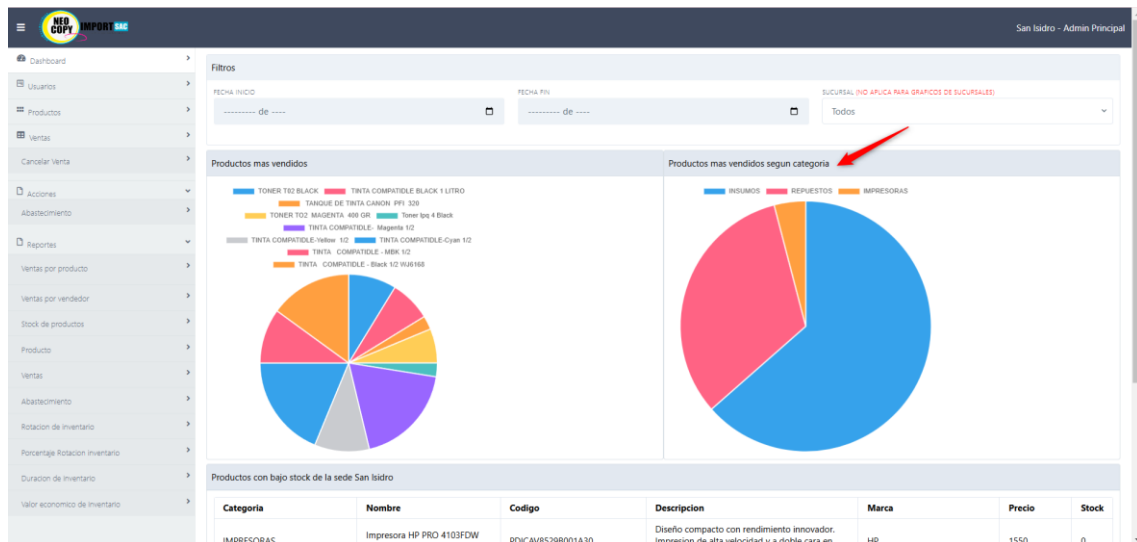


Figura 21. Productos más vendidos por categoría

Elaboración propia

En la Figura 22 se muestra el prototipo del requerimiento funcional 04 la cual fue aprobado por el Product Owner para la vista de un dashboard en donde se visualice una tabla de productos con bajo stock.

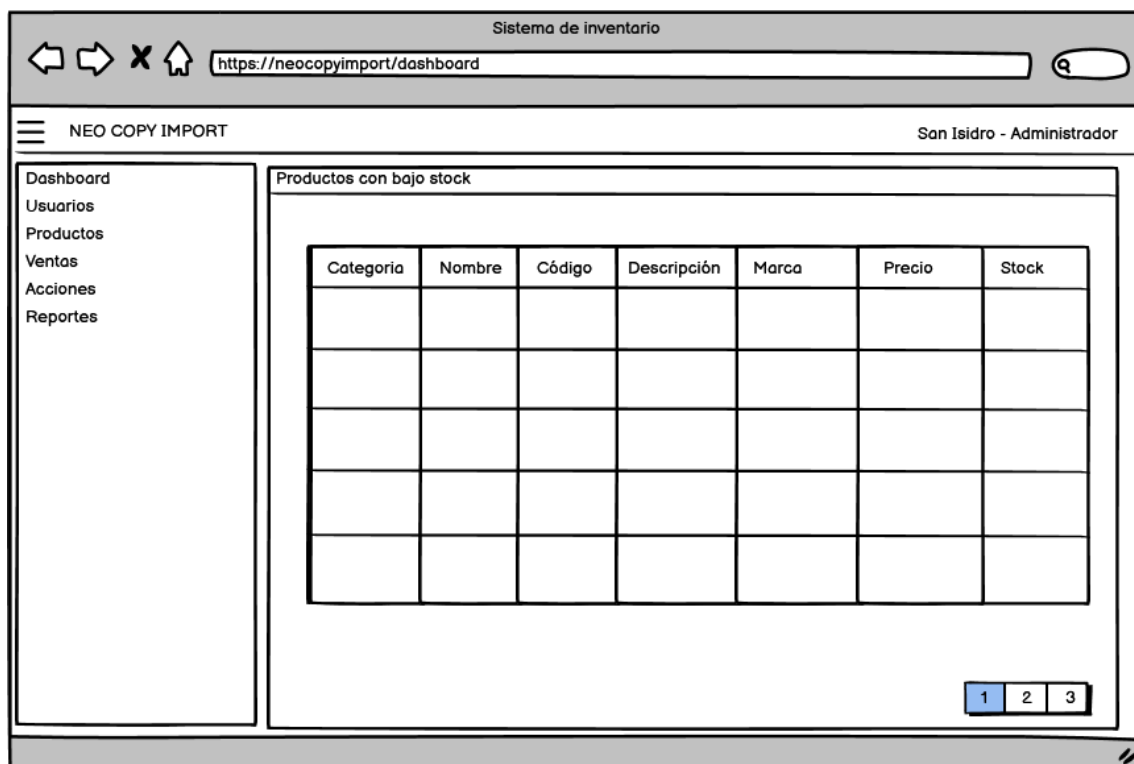


Figura 22. Prototipo para el requerimiento RF-05.

Fuente: Elaboración propia

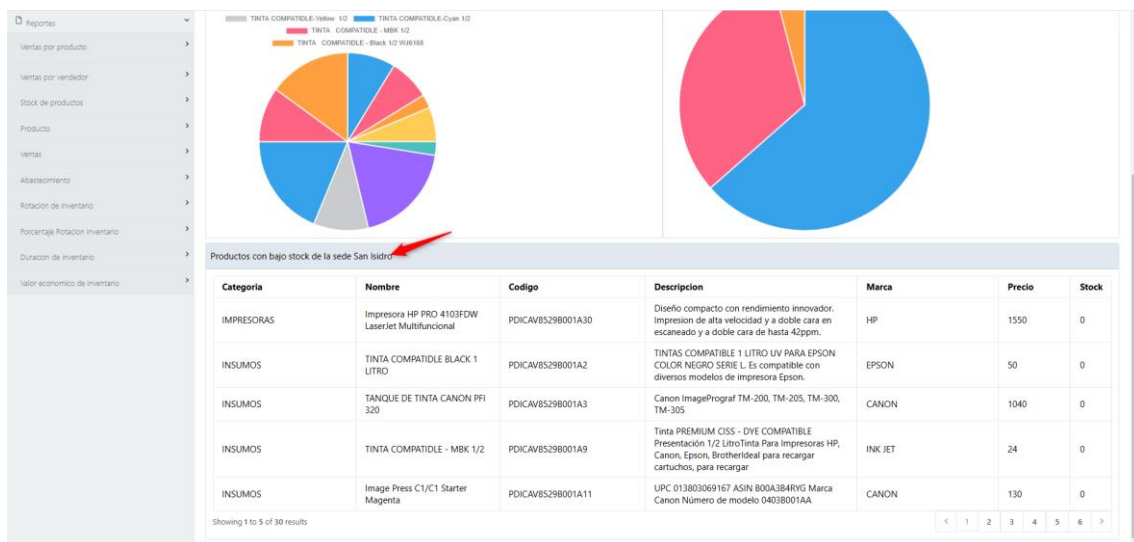


Figura 23. Tabla informativa de stock

Elaboración propia

En la Figura 24 se muestra el prototipo del requerimiento funcional 05 la cual fue aprobado por el Product Owner para la vista del módulo de usuarios en donde se podrá crear un nuevo usuario.

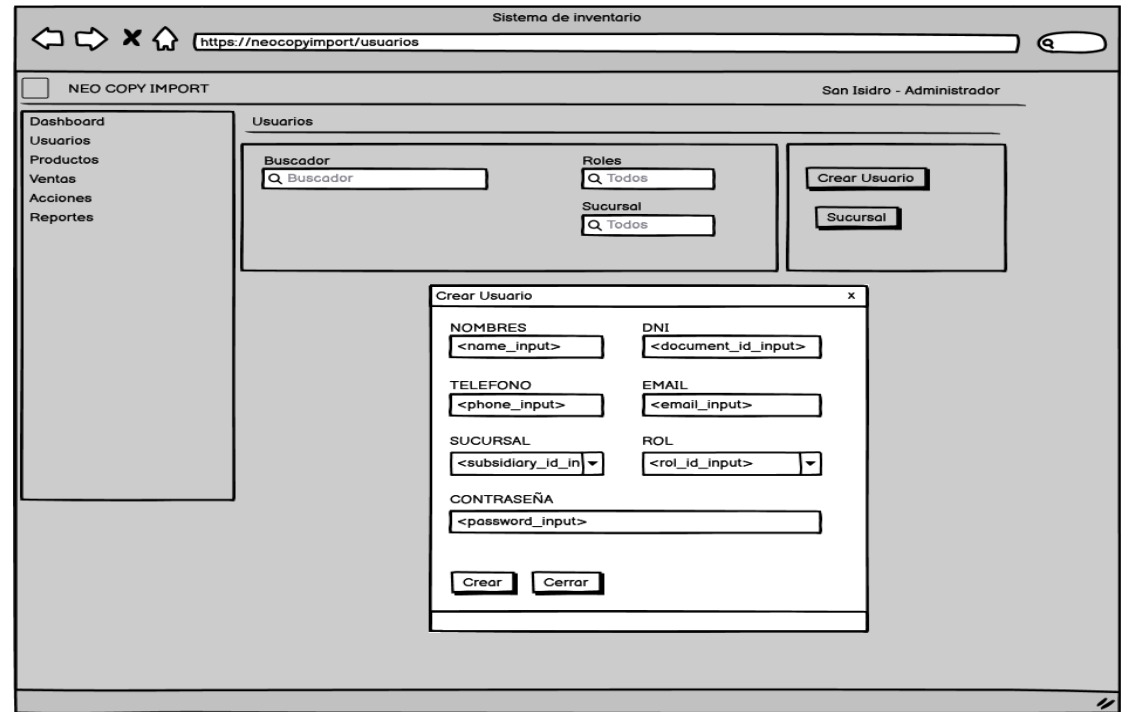


Figura 24. Prototipo para el requerimiento RF-06.

Fuente: Elaboración propia

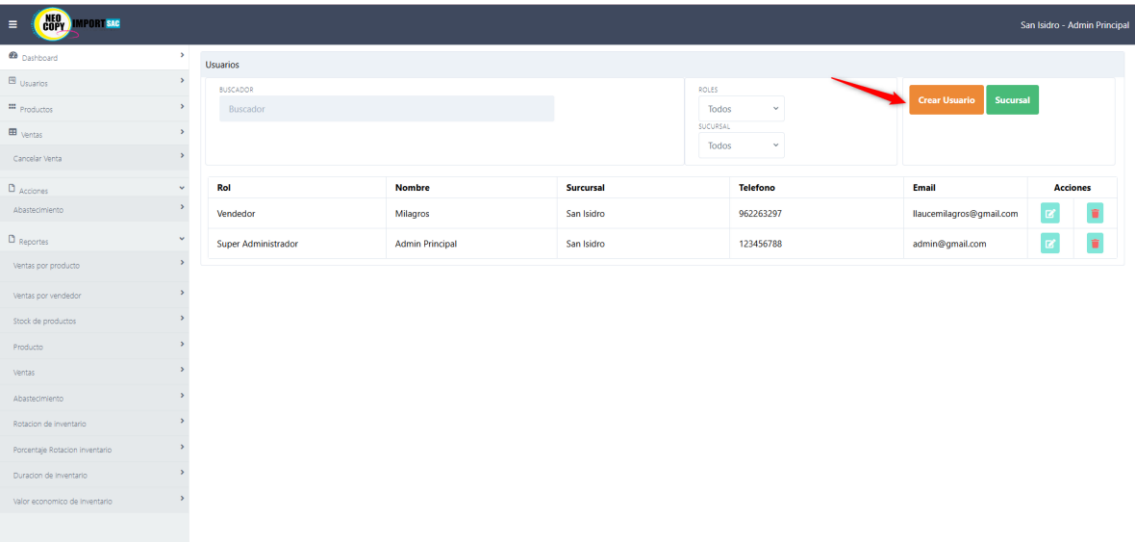


Figura 25. Crear nuevo usuario

Fuente: Elaboración propia

En la Figura 26 se muestra el prototipo del requerimiento funcional 10 la cual fue aprobado por el Product Owner para la vista del módulo de productos en donde se podrá gestionar la búsqueda de estos.

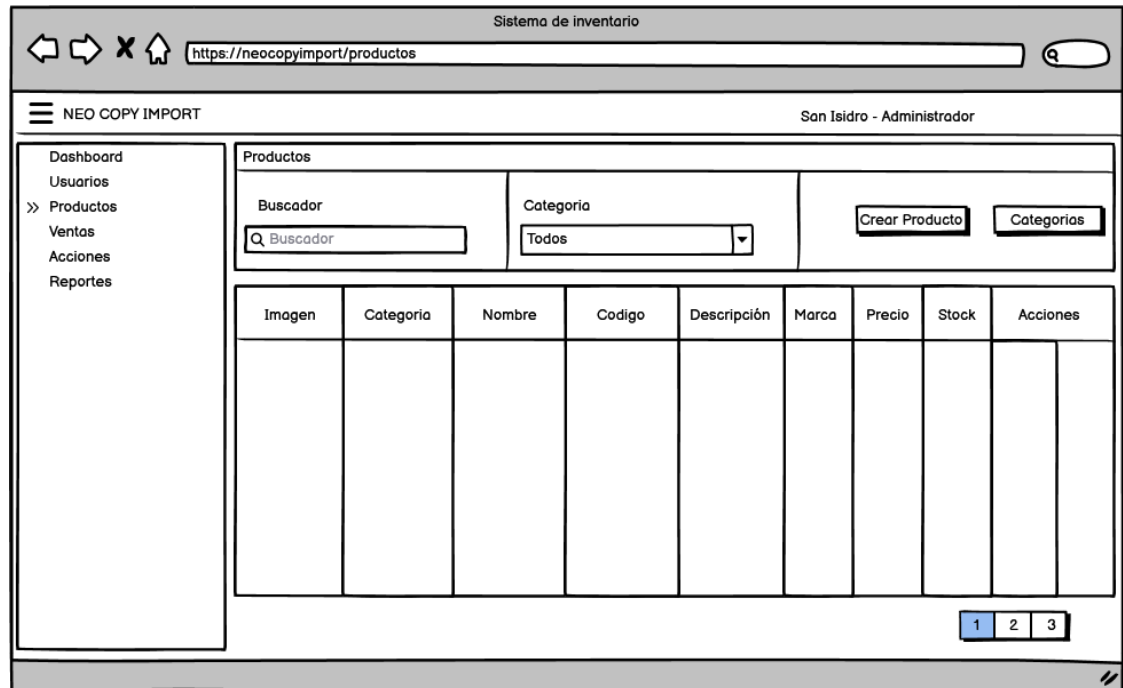


Figura 26. Prototipo para el requerimiento RF-10

Fuente: Elaboración propia

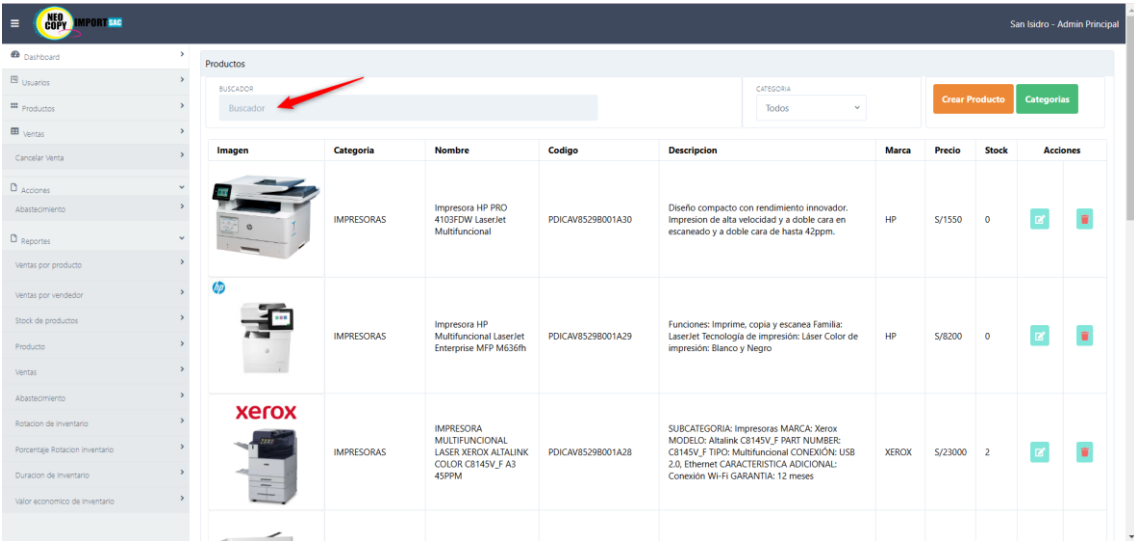


Figura 27. Buscar productos

Fuente: Elaboración propia

En la Figura 28 se muestra el prototipo del requerimiento funcional 11 la cual fue aprobado por el Product Owner para la vista del módulo de productos en donde se podrá crear un nuevo producto.

El prototipo muestra una interfaz web para el 'Sistema de inventario'. En la parte superior, hay una barra de navegación con el logo 'NEO COPY IMPORT' y el nombre de usuario 'San Isidro - Administrador'. A la izquierda, un menú lateral contiene opciones como 'Dashboard', 'Usuarios', 'Productos', 'Ventas', 'Acciones' y 'Reportes'. El área principal está dividida en tres secciones: 'Productos' con un buscador, 'Crear Producto' (que es un modal de formulario) y 'Categorías'. El modal 'Crear Producto' contiene campos para: IMAGEN (con un botón 'Seleccionar Archivo'), NOMBRES, MARCA, CODIGO, DESCRIPCIÓN, MONEDA, PRECIO y CATEGORIA. En la parte inferior del modal hay botones 'Crear' y 'Salir'. A la derecha del modal, hay una tabla con columnas 'Precio', 'Stock' y 'Acciones'. En la parte inferior derecha, hay una barra de paginación con los números 1, 2 y 3.

Figura 28. Prototipo para el requerimiento RF-11.

Fuente: Elaboración propia

NEO COPYIMPORT

San Isidro - Admin Principal

Dashboard

Usuarios

Productos

Ventas

Cancelar Venta

Acciones

Abastecimiento

Reportes

Ventas por producto

Ventas por vendedor

Stock de productos

Producto

Ventas

Abastecimiento

Rotación de inventario

Porcentaje Rotación inventario

Duración de inventario

Valor económico de inventario

Productos

BUSCADOR

Buscador

CATEGORIA

Todos

Crear Producto

Categorías

Imagen	Categoría	Nombre	Codigo	Descripción	Marca	Precio	Stock	Acciones
	IMPRESORAS	Impresora HP PRO 4103FDW LaserJet Multifuncional	PDICAV8529B001A30	Diseño compacto con rendimiento innovador. Impresión de alta velocidad y a doble cara en escaneado y a doble cara de hasta 42ppm.	HP	S/1550	0	
	IMPRESORAS	Impresora HP Multifuncional LaserJet Enterprise MFP M636fh	PDICAV8529B001A29	Funciones: Imprime, copia y escanea Familia: LaserJet Tecnología de impresión: Láser Color de impresión: Blanco y Negro	HP	S/8200	0	
	IMPRESORAS	IMPRESORA MULTIFUNCIONAL LASER XEROX ALTALINK COLOR C8145V_F A3 45PPM	PDICAV8529B001A28	SUBCATEGORIA: Impresoras MARCA: Xerox MODELO: Altalink C8145V_F PART NUMBER: C8145V_F TIPO: Multifuncional CONEXIÓN: USB 2.0, Ethernet CARACTERÍSTICA ADICIONAL: Conexión Wi-Fi GARANTÍA: 12 meses	XEROX	S/23000	2	

Figura 29. Crear productos

Fuente: Elaboración propia

En la Figura 30 se muestra el prototipo del requerimiento funcional 12 la cual fue aprobado por el Product Owner para la vista del módulo de productos en donde se podrá editar un producto existente.

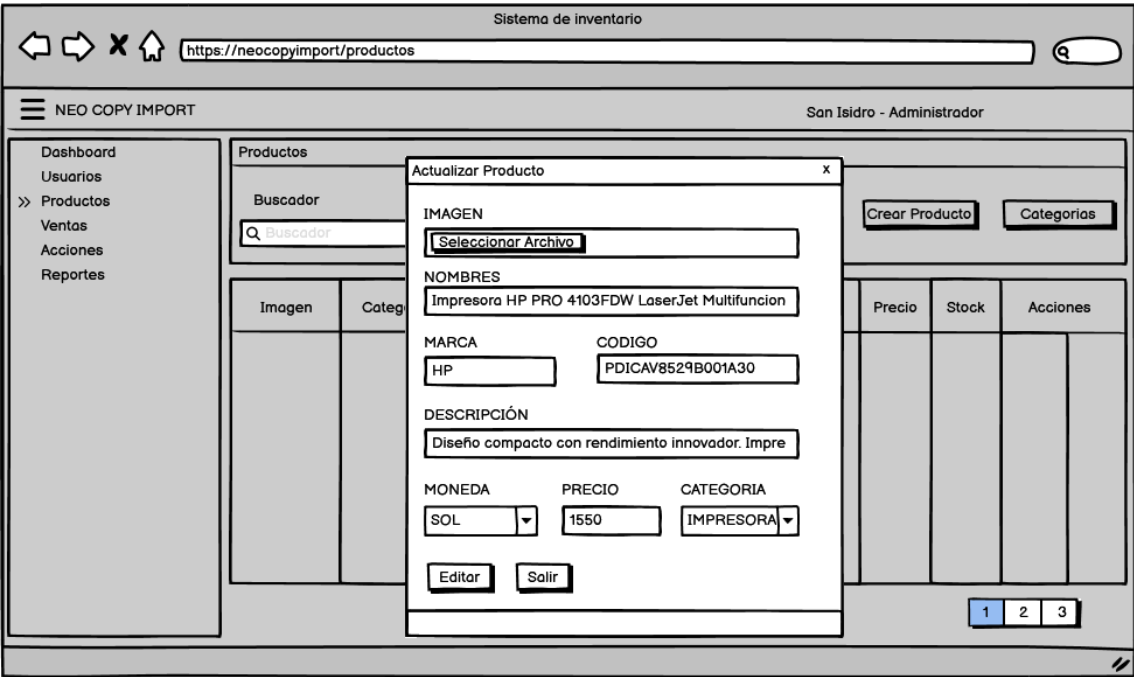


Figura 30. Prototipo para el requerimiento RF-12.

Fuente: Elaboración propia

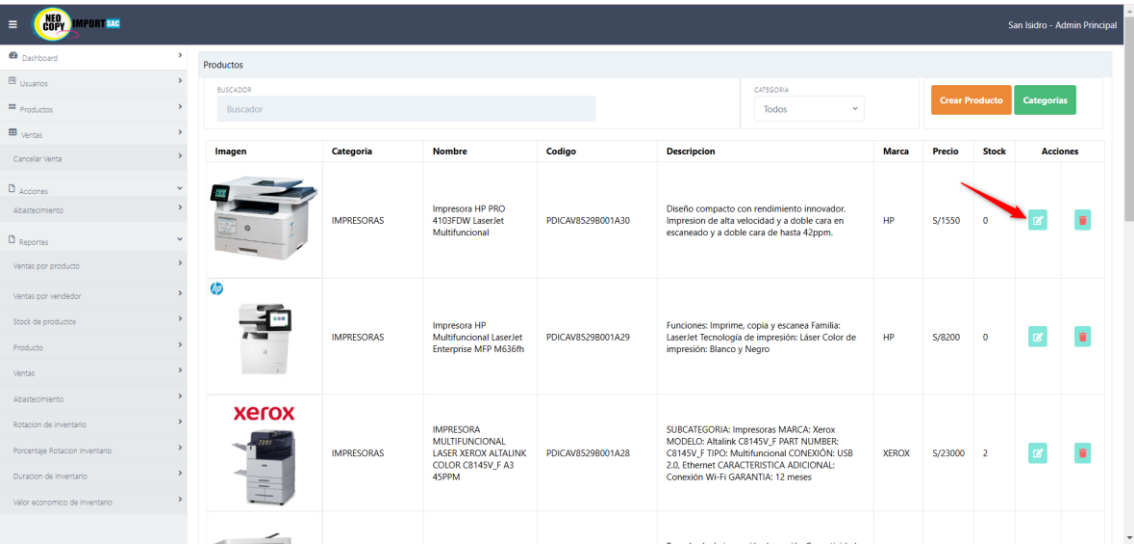


Figura 31. Editar productos

Fuente: Elaboración propia

En la Figura 32 se muestra el prototipo del requerimiento funcional 13 la cual fue aprobado por el Product Owner para la vista del módulo de productos en donde se podrá eliminar un producto existente.

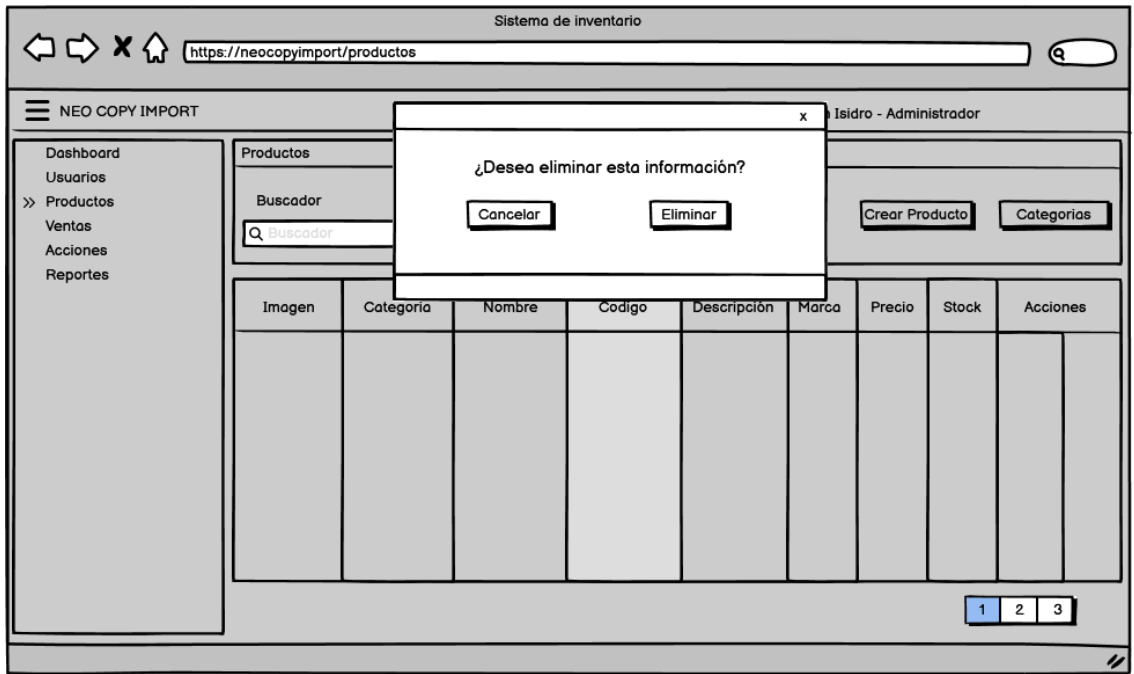


Figura 32. Prototipo para el requerimiento RF-13.

Fuente: Elaboración propia

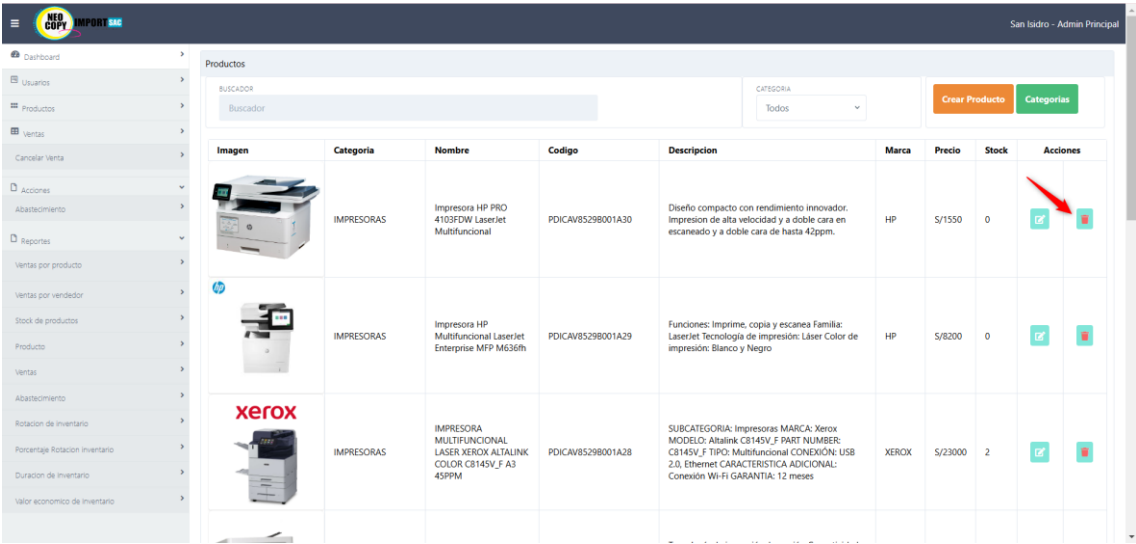


Figura 33. Eliminar producto

Elaboración propia

En la Figura 34 se muestra el prototipo del requerimiento funcional 14 la cual fue aprobado por el Product Owner para la vista del módulo de productos en donde se podrá crear una nueva categoría.

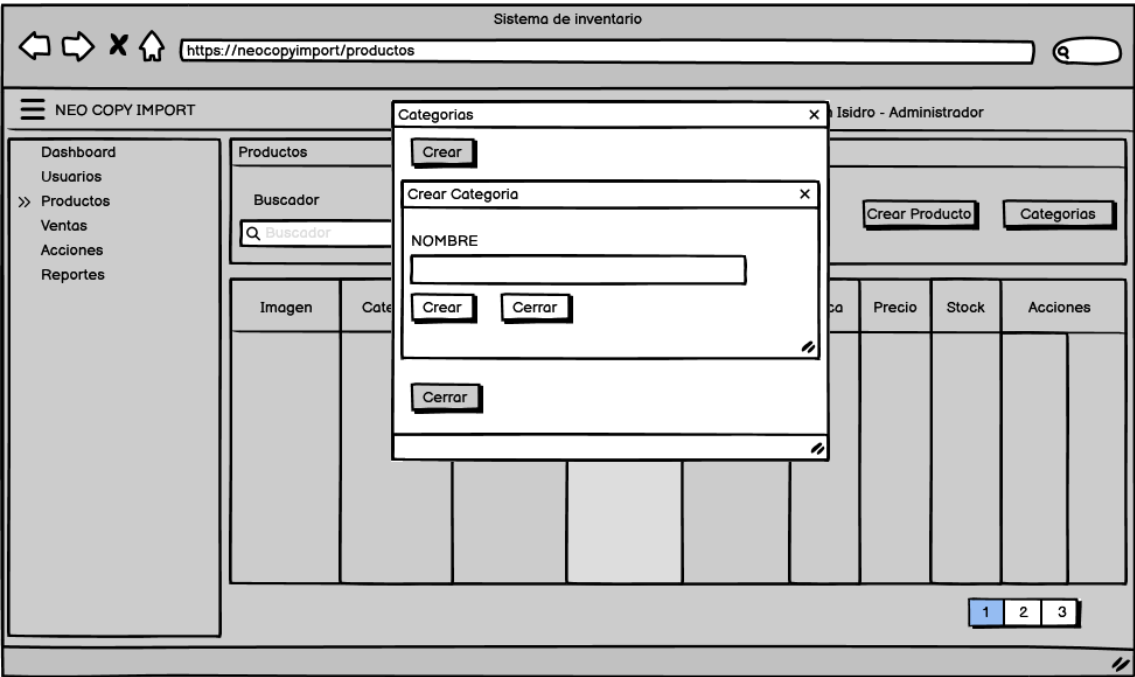


Figura 34. Prototipo para el requerimiento RF-14.

Fuente: Elaboración propia

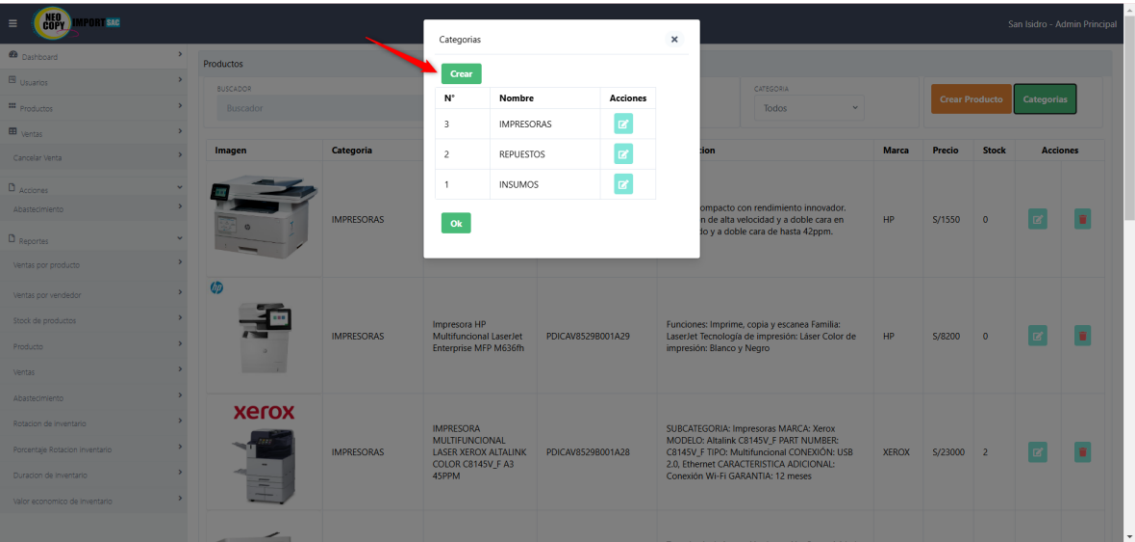


Figura 35. Crear categoría

Elaboración propia

En la Figura 36 se muestra el prototipo del requerimiento funcional 15 la cual fue aprobado por el Product Owner para la vista del módulo de productos en donde se podrá buscar una categoría existente.

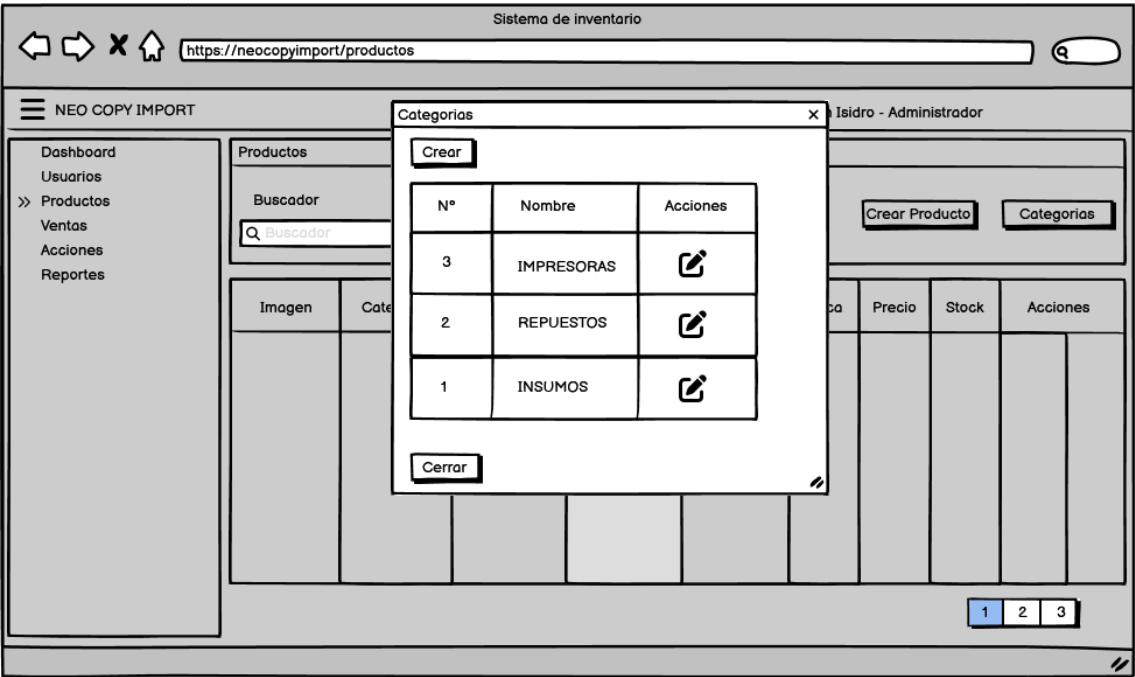


Figura 36. Prototipo para el requerimiento RF-15.

Fuente: Elaboración propia

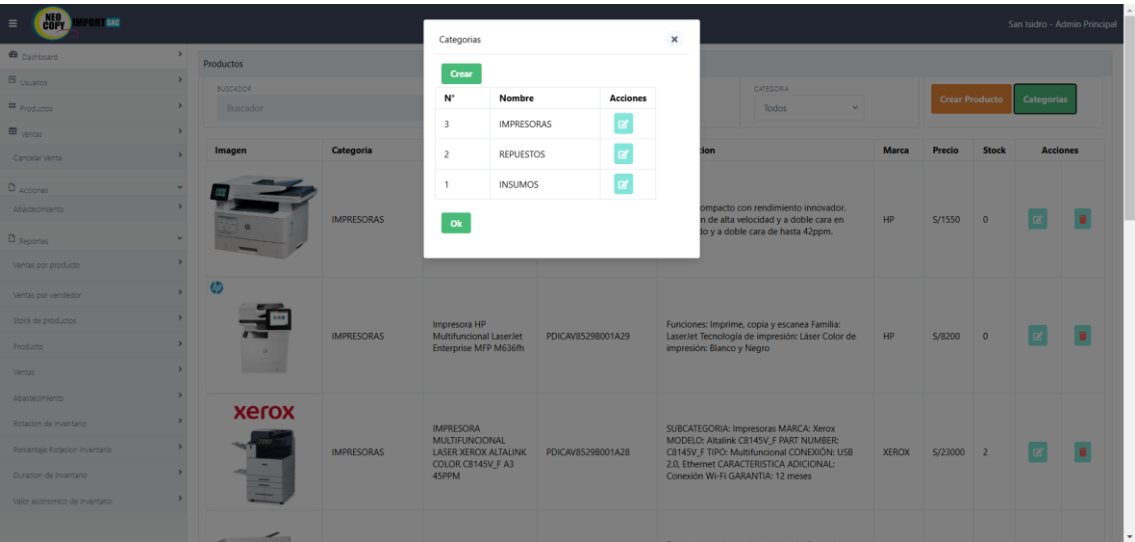


Figura 37. Editar categoría existente

Elaboración propia

ENTIDADES DEL PRIMER SPRINT

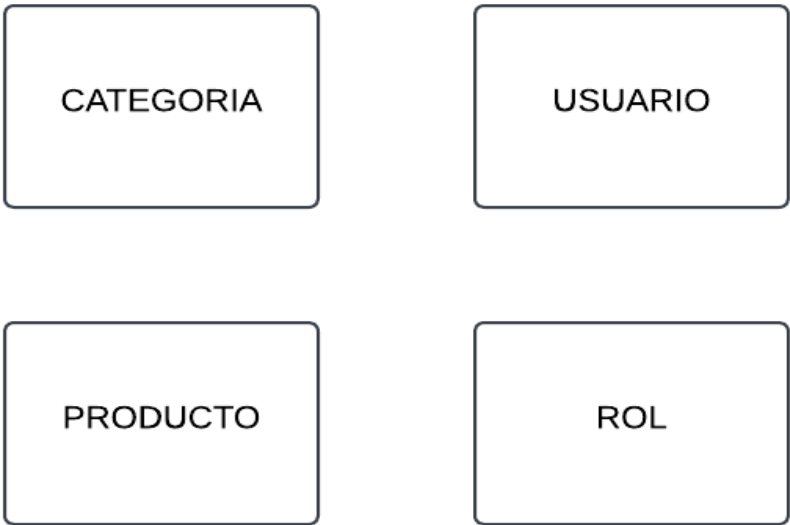


Figura 38. Entidades del sprint 1

Fuente: Elaboración propia

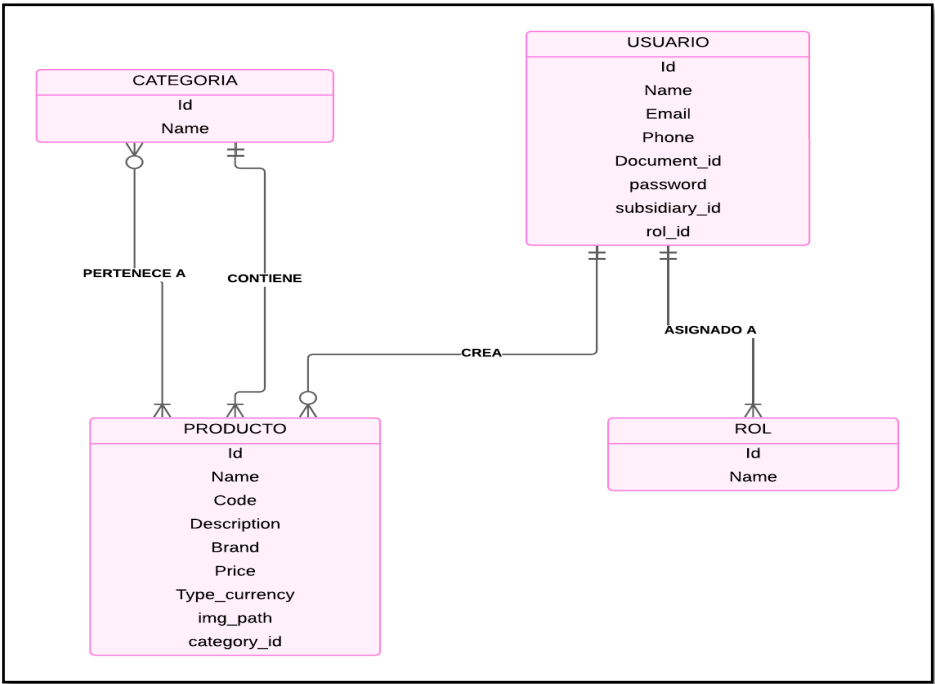


Figura 39. Diagrama entidad relación del sprint 1

Fuente: Elaboración propia

DIAGRAMA LÓGICO DEL PRIMER SPRINT

En la figura 40 se muestra el diagrama físico que representa la descripción de los datos con el mayor detalle, su estructura y las relaciones de los datos fundamentales del negocio.

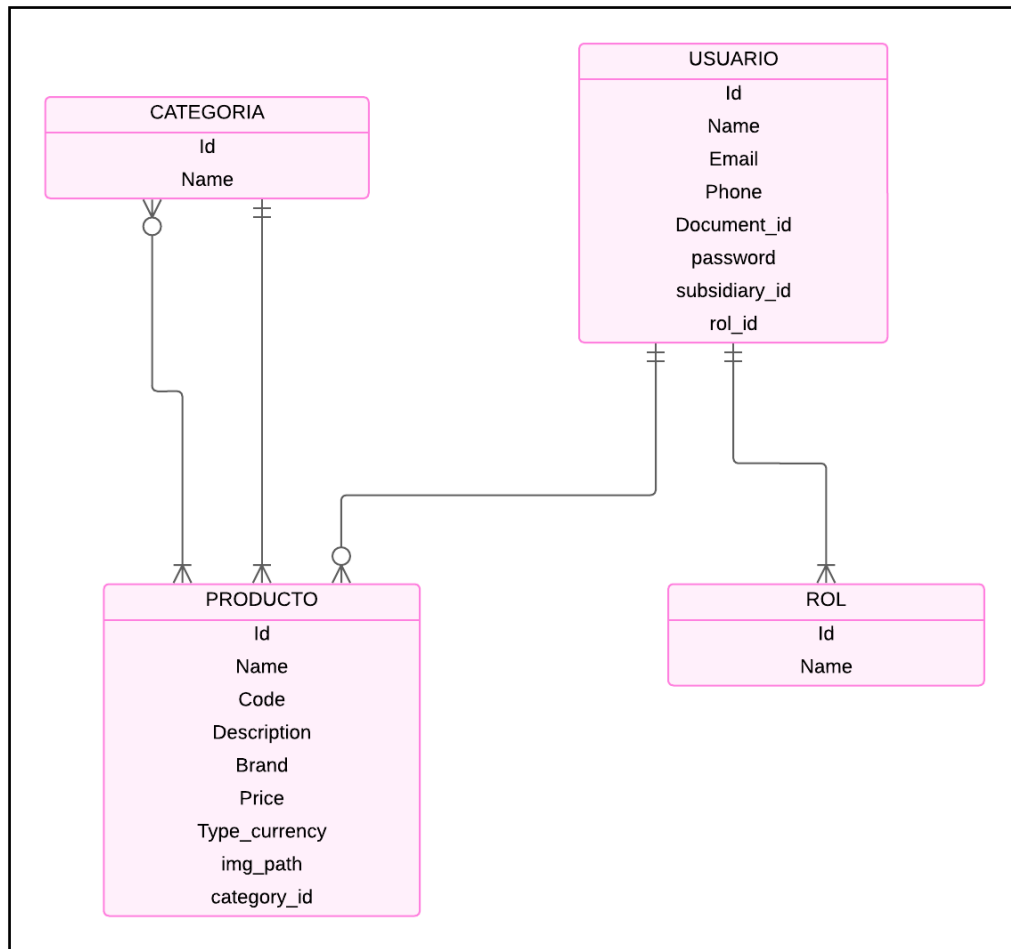


Figura 40. Diagrama Lógico del Sprint 1

Fuente: Elaboración propia

DIAGRAMA FÍSICO DEL PRIMER SPRINT

En la figura 41 se muestra el diagrama físico que representa como se construirá el modelo en la base de datos.

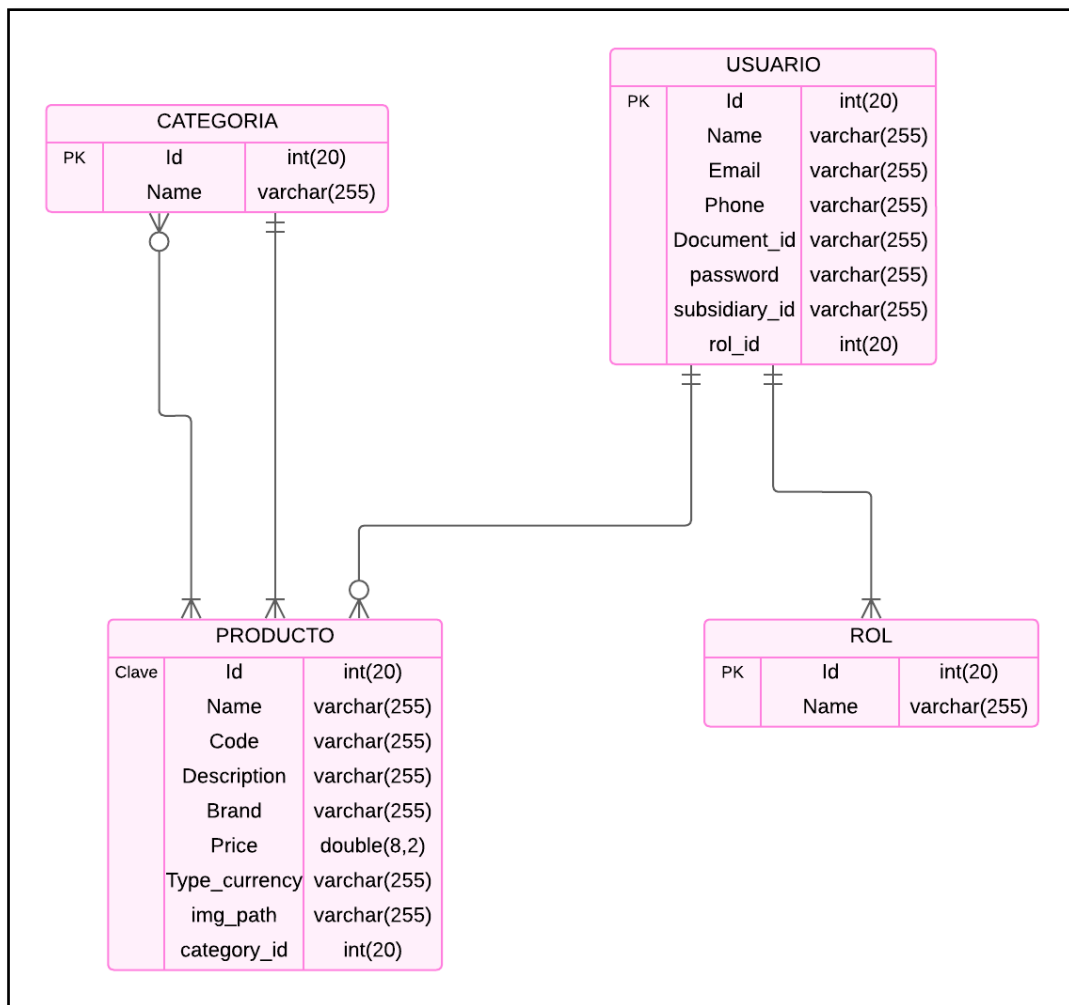


Figura 41. Diagrama físico del sprint 1

Fuente: Elaboración propia

IMPLEMENTACIÓN DEL PRIMER SPRINT

Diagrama de la Base de Datos:

En la Figura 42 tenemos el diseño de la base de datos en MySQL que vamos a usar para el Sprint 1.

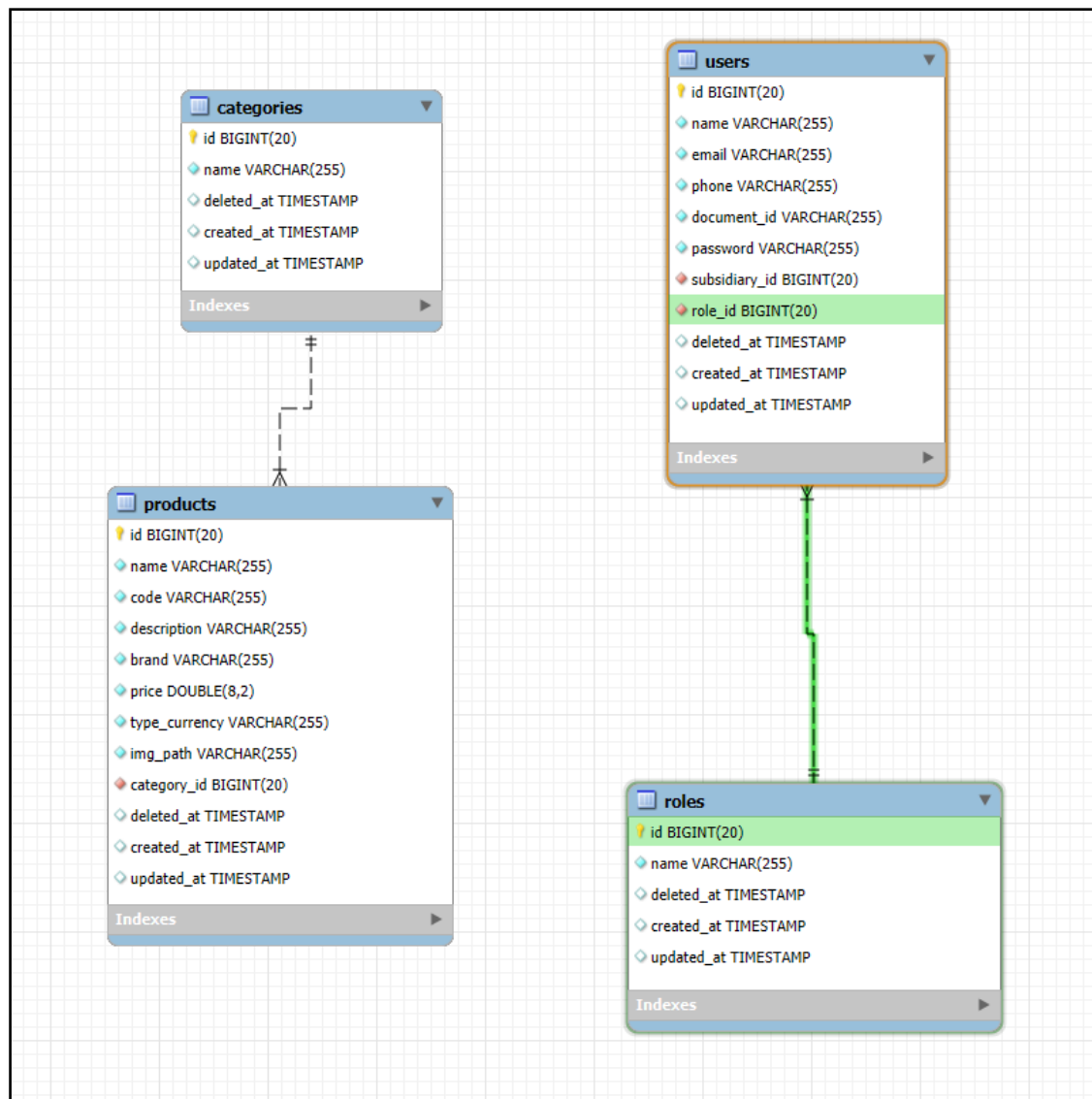


Figura 42. Diagrama de la base de datos del Sprint 1.

Fuente: Elaboración propia

En la Figura 43 se muestra el modelo del RF-01, con el nombre User.php

RF-01: El sistema debe tener una ventana que permita el inicio de sesión por medio de un usuario y contraseña

```

class User extends Authenticatable
{
    use Notifiable;
    use HasFactory;
    use HasApiTokens;

    /**
     * The attributes that are mass assignable.
     *
     * @var array<int, string>
     */
    protected $fillable = [
        'name',
        'phone',
        'email',
        'password',
        'document_id',
        'subsidiary_id',
        'role_id'
    ];

    /**
     * The attributes that should be hidden for serialization.
     *
     * @var array<int, string>
     */
    protected $hidden = [
        'password'
    ];

    public function subsidiary()
    {
        return $this->belongsTo(Subsidiary::class);
    }

    public function role()
    {
        return $this->belongsTo(Role::class);
    }
}

```

Figura 43. Modelo del Requerimiento Funcional 01.

Fuente: Elaboración propia

En la Figura 44 ilustra la vista correspondiente al RF-01, denominada login-component.blade.php. Esta vista colabora con el controlador LoginComponent.php para

aplicar estilos, manejar los datos, verificar la validez de los campos de entrada y construir la interfaz destinada a la interacción del usuario.

```

<div class="container mx-auto h-full flex flex-1 justify-center items-center">
  <div class="w-full max-w-lg">
    <div class="leading-loose">
      <div class="max-w-xl m-4 p-10 bg-white rounded shadow-xl">
        <p class="text-gray-800 font-medium text-center text-lg font-bold">Login</p>
        <div class="">
          <label class="block text-sm text-gray-600" for="username">Email</label>
          <input class="w-full px-5 py-1 text-gray-700 bg-gray-200 rounded" wire:model.lazy="email" id="username" name="username" type="text" required="" placeholder="User Name" aria-label="username">
          @error('email') <span class="text-danger error">{{ $message }}</span>@enderror
        </div>
        <div class="mt-2">
          <label class="block text-sm text-gray-600" for="password">Contraseña</label>
          <input class="w-full px-5 py-1 text-gray-700 bg-gray-200 rounded" wire:model.lazy="password" id="password" name="password" type="password" required="" placeholder="*****" aria-label="password">
          @error('password') <span class="text-danger error">{{ $message }}</span>@enderror
        </div>
        <div class="mt-4 items-center justify-between">
          <button class="px-4 py-1 text-white font-light tracking-wider bg-gray-900 rounded" type="submit" wire:click.prevent="loginUser">login</button>
        </div>

        @if (session()->has('error'))
          <div class="bg-red-300 border border-red-300 text-red-dark px-4 py-3 rounded relative" role="alert">
            <strong class="font-bold">Error</strong>
            <span class="block sm:inline">{{ session('error') }}</span>
          </div>
        @endif

      </div>
    </div>
  </div>
</div>

```

Figura 44. Vista del Requerimiento Funcional 01

Fuente: Elaboración propia

En la figura 45 se muestra el controlador del RF-01, con nombre LoginComponent.php

```

class LoginComponent extends Component
{
    public $email, $password;

    public function render()
    {
        return view('livewire.login-component');
    }

    public function loginUser()
    {
        $this->validate([
            'email' => 'required|email|max:255',
            'password' => 'required|max:255',
        ]);

        if(Auth::attempt(['email' => $this->email, 'password' => $this->password]))
        {
            return redirect()->to('/dashboard');
        }
        else
        {
            session()->flash('error', 'Las credenciales o el usuario no son correctos.');
```

Figura 45. Controlador del Requerimiento Funcional 01.

Fuente: Elaboración propia

RF – 01 El sistema debe tener un módulo dashboard en donde se visualice un gráfico estadístico de productos no vendidos.

En la figura 46 se muestra la vista del RF-02, con el nombre “Productos más vendidos según categoría”

```
<div class="mb-2 border-solid border-grey-light rounded border shadow-sm w-full md:w-1/2 lg:w-1/2">
  <div class="bg-gray-300 px-2 py-3 border-solid border-gray-400 border-b">
    Productos mas vendidos segun categoria
  </div>
  <div class="p-3">
    <form class="w-full" style="position: relative; height:50vh; width:50vw">
      <canvas id="chart2"></canvas>
    </form>
  </div>
</div>
```

Figura 46. Vista del requerimiento funcional 02

Fuente: Elaboración propia

En la figura 47 se muestra el controlador del RF-02, con el nombre getMostSolProducts.

```
public function getMostSoldProducts()
{
    $data = Sale::select(
        DB::raw('SUM(sale_details.quantity) as data'),
        DB::raw('products.name as label')
    )
    ->join('sale_details', 'sale_details.sale_id', '=', 'sales.id')
    ->join('products', 'products.id', '=', 'sale_details.product_id');

    if(!empty($this->subsidiary_filter))
    {
        $data = $data->where('sales.subsidiary_id', '=', $this->subsidiary_filter);
    }

    if(!empty($this->init_date))
    {
        $data = $data->where('created_date', '>=', $this->init_date . "-1");
    }

    if(!empty($this->end_date))
    {
        $data = $data->where('created_date', '<=', $this->end_date . "-31");
    }

    $data = $data->groupBy("products.id")->get();

    $ar = [];
    $ar[] = $data->pluck("label")->take(10);
    $ar[] = $data->pluck("data")->take(10);

    return $ar;
}
```

Figura 47. Vista del modelo del requerimiento funcional 02

Fuente: Elaboración propia

RF-03: El sistema debe tener un módulo dashboard en donde se visualice un gráfico estadístico de productos vendidos según categorías.

En la figura 48 se muestra el modelo del RF-03, con el nombre Product.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Product extends Model
{
    use HasFactory;
    use SoftDeletes;

    protected $fillable = ['name',
        'code',
        'description',
        'brand',
        'price',
        'type_currency',
        'img_path',
        'category_id'];

    public function category()
    {
        return $this->belongsTo(Category::class);
    }

    public function getPriceAttribute($value)
    {
        return round($value, 2);
    }
}
```

Figura 48. Modelo del requerimiento 03

Fuente: Elaboración propia

En la figura 49 se muestra la vista del RF-03, con el nombre Dashboard-component.blade.php

```

<div class="mb-2 border-solid border-grey-light rounded border shadow-sm w-full md:w-1/2 lg:w-1/2">
  <div class="bg-gray-300 px-2 py-3 border-solid border-gray-400 border-b">
    Productos mas vendidos segun categoria
  </div>
  <div class="p-3">
    <form class="w-full" style="position: relative; height:50vh; width:50vw">
      <canvas id="chart2"></canvas>
    </form>
  </div>
</div>

```

Figura 49. Vista del requerimiento 03

Fuente: Elaboración propia

En la figura 50 se muestra el controlador del RF-03, con el nombre DashboardComponent.php

```

public function getMostSoldProductsByCategory()
{
    $data = Sale::select(
        DB::raw('SUM(sale_details.quantity) as data'),
        DB::raw('categories.name as label')
    )
    ->join('sale_details', 'sale_details.sale_id', '=', 'sales.id')
    ->join('products', 'products.id', '=', 'sale_details.product_id')
    ->join('categories', 'categories.id', '=', 'products.category_id');

    if(!empty($this->subsidiary_filter))
    {
        $data = $data->where('sales.subsidiary_id', '=', $this->subsidiary_filter);
    }

    if(!empty($this->init_date))
    {
        $data = $data->where('created_date', '>=', $this->init_date . "-1");
    }

    if(!empty($this->end_date))
    {
        $data = $data->where('created_date', '<=', $this->end_date . "-31");
    }

    $data = $data->groupBy("categories.id")->get();

    $ar = [];
    $ar[] = $data->pluck("label")->take(10);
    $ar[] = $data->pluck("data")->take(10);

    return $ar;
}

```

Figura 50. Controlador del requerimiento 03

Fuente: Elaboración propia

En la figura 51 se muestra el modelo del RF-04, con el nombre ProductStock.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class ProductStock extends Model
{
    use HasFactory;
    use SoftDeletes;

    protected $fillable = ['product_id', 'stock', 'subsidiary_id'];

    public function product()
    {
        return $this->belongsTo(Product::class);
    }

    public function subsidiary()
    {
        return $this->belongsTo(Subsidiary::class);
    }
}
```

Figura 51. Modelo del requerimiento 04

Fuente: Elaboración propia

En la figura 52 se muestra la vista del RF-04, con el nombre Dashboard-component.blade.php

```

<div class="flex flex-4 flex-col md:flex-row lg:flex-row mx-2">
  <!--Underline form-->
  <div class="mb-2 border-solid border-grey-light rounded border shadow-sm w-full md:w lg:w">
    <div class="bg-gray-300 px-2 py-3 border-solid border-gray-400 border-b">
      Productos con bajo stock de la sede {{ $subsidiary_name }}
    </div>
    <div class="p-3">
      <div class="w-full">
        <table class="table-responsive w-full rounded">
          <thead>
            <tr>
              <th class="border w-1/6 px-4 py-2">Categoria</th>
              <th class="border w-1/6 px-4 py-2">Nombre</th>
              <th class="border w-1/6 px-4 py-2">Codigo</th>
              <th class="border w-1/4 px-4 py-2">Descripcion</th>
              <th class="border w-1/6 px-4 py-2">Marca</th>
              <th class="border w-1/6 px-4 py-2">Precio</th>
              <th class="border w-1/6 px-4 py-2">Stock</th>
            </tr>
          </thead>
          <tbody>
            @foreach ($products as $product)
              <tr>
                <td class="border px-4 py-2">{{ $product->product->category->name }}</td>
                <td class="border px-4 py-2">{{ $product->product->name }}</td>
                <td class="border px-4 py-2">{{ $product->product->code }}</td>
                <td class="border px-4 py-2">{{ $product->product->description }}</td>
                <td class="border px-4 py-2">{{ $product->product->brand }}</td>
                <td class="border px-4 py-2">{{ $product->product->price }}</td>
                <td class="border px-4 py-2">{{ $product->stock }}</td>
              </tr>
            @endforeach
          </tbody>
        </table>
        <div class="row">
          <div class="col-md-12">
            {{ $products->withQueryString()->links() }}
          </div>
        </div>
      </div>
    </div>
  </div>
  <!--/Underline form-->
</div>

```

Figura 52. Vista del requerimiento 04

Fuente: Elaboración propia

En la figura 53 se muestra el modelo del RF-06, con el nombre User.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Foundation\Auth\User as Authenticatable;
use Illuminate\Notifications\Notifiable;
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use Notifiable;
    use HasFactory;
    use HasApiTokens;

    /**
     * The attributes that are mass assignable.
     *
     * @var array<int, string>
     */
    protected $fillable = [
        'name',
        'phone',
        'email',
        'password',
        'document_id',
        'subsidiary_id',
        'role_id'
    ];

    /**
     * The attributes that should be hidden for serialization.
     *
     * @var array<int, string>
     */
    protected $hidden = [
        'password'
    ];

    public function subsidiary()
    {
        return $this->belongsTo(Subsidiary::class);
    }

    public function role()
    {
        return $this->belongsTo(Role::class);
    }
}

```

Figura 53. Modelo del requerimiento 06

Fuente: Elaboración propia

En la figura 54 se muestra la vista del RF-06, con el nombre Create-user.blade.php

```
<form class="w-full">
  <div class="flex flex-wrap -mx-3 mb-6">
    <div class="w-full md:w-1/2 px-3 mb-6 md:mb-0">
      <label class="block uppercase tracking-wide text-gray-700 text-xs font-light mb-1" for="name_input">
        Nombres
      </label>
      <input class="appearance-none block w-full bg-gray-200 text-gray-darker border border-gray-200 rounded py-3 px-4 leading-tight focus:outline-none focus:bg-white focus:border-gray-600"
        id="name_input" type="text" placeholder="" wire:model.lazy="name_c" value="{{ old('name_c') }}" />
      @error('name_c') <p class="text-red-500 text-xs italic">{{ $message }}</p>@enderror
    </div>
    <div class="w-full md:w-1/2 px-3 mb-6 md:mb-0">
      <label class="block uppercase tracking-wide text-gray-darker text-xs font-light mb-1" for="document_input">
        DNI
      </label>
      <input
        class="appearance-none block w-full bg-gray-200 text-gray-darker border border-gray-200 rounded py-3 px-4 leading-tight focus:outline-none focus:bg-white focus:border-gray"
        id="document_id_input" type="text" placeholder="" wire:model.lazy="document_id_c" value="{{ old('document_id_c') }}" />
      @error('document_id_c') <span class="text-red-500 text-xs italic">{{ $message }}</span>@enderror
    </div>
  </div>
  <div class="flex flex-wrap -mx-3 mb-6">
    <div class="w-full md:w-1/2 px-3 mb-6 md:mb-0">
      <label class="block uppercase tracking-wide text-gray-700 text-xs font-light mb-1" for="phone_input">
        Telefono
      </label>
      <input class="appearance-none block w-full bg-gray-200 text-gray-darker border border-gray-200 rounded py-3 px-4 leading-tight focus:outline-none focus:bg-white focus:border-gray-600"
        id="phone_input" type="text" placeholder="" wire:model.lazy="phone_c" value="{{ old('phone_c') }}" />
      @error('phone_c') <p class="text-red-500 text-xs italic">{{ $message }}</p>@enderror
    </div>
    <div class="w-full md:w-1/2 px-3 mb-6 md:mb-0">
      <label class="block uppercase tracking-wide text-gray-darker text-xs font-light mb-1" for="email_input">
        Email
      </label>
      <input
        class="appearance-none block w-full bg-gray-200 text-gray-darker border border-gray-200 rounded py-3 px-4 leading-tight focus:outline-none focus:bg-white focus:border-gray"
        id="email_input" type="text" placeholder="" wire:model.lazy="email_c" value="{{ old('email_c') }}" />
      @error('email_c') <span class="text-red-500 text-xs italic">{{ $message }}</span>@enderror
    </div>
  </div>
  <div class="flex flex-wrap -mx-3 mb-2">
    <div class="w-full md:w-1/2 px-3 mb-6 md:mb-0">
      <label class="block uppercase tracking-wide text-gray-darker text-xs font-light mb-1" for="subsidiary_id_input">
        Sucursal
      </label>
      <div class="relative">
        <select class="block appearance-none w-full bg-gray-200 border border-gray-200 text-gray-darker py-3 px-4 pr-8 rounded leading-tight focus:outline-none focus:bg-white focus:border-gray"
          id="subsidiary_id_input" wire:model.lazy="subsidiary_id_c" value="{{ old('subsidiary_id_c') }}">
          <option value="" hidden selected>Selecione</option>
          @foreach ($subsidiary_list as $subsidiary)
            <option value="{{ ($subsidiary->id) }}">{{ ($subsidiary->name)}}</option>
          @endforeach
        </select>
        <div class="pointer-events-none absolute inset-y-0 right-0 flex items-center px-2 text-gray-darker">
          <svg class="fill-current h-4 w-4" xmlns="http://www.w3.org/2000/svg"
            viewBox="0 0 20 20">
            <path d="M0 20 20 20 12.5 12.5 15 10 8.5 7.5 15 5 12.5 2.5 0 0 20 20" />
          </svg>
        </div>
        @error('subsidiary_id_c') <span class="text-red-500 text-xs italic">{{ $message }}</span>@enderror
      </div>
    </div>
    <div class="w-full md:w-1/2 px-3 mb-6 md:mb-0">
      <label class="block uppercase tracking-wide text-gray-darker text-xs font-light mb-1" for="role_id_input">
        Rol
      </label>
      <div class="relative">
        <select class="block appearance-none w-full bg-gray-200 border border-gray-200 text-gray-darker py-3 px-4 pr-8 rounded leading-tight focus:outline-none focus:bg-white focus:border-gray"
          id="role_id_input" wire:model.lazy="role_id_c" value="{{ old('role_id_c') }}">
          <option value="" hidden selected>Selecione</option>
          @foreach ($role_list as $role)
            <option value="{{ ($role->id) }}">{{ ($role->name)}}</option>
          @endforeach
        </select>
        <div class="pointer-events-none absolute inset-y-0 right-0 flex items-center px-2 text-gray-darker">
          <svg class="fill-current h-4 w-4" xmlns="http://www.w3.org/2000/svg"
            viewBox="0 0 20 20">
            <path d="M0 20 20 20 12.5 12.5 15 10 8.5 7.5 15 5 12.5 2.5 0 0 20 20" />
          </svg>
        </div>
        @error('role_id_c') <span class="text-red-500 text-xs italic">{{ $message }}</span>@enderror
      </div>
    </div>
  </div>
  <div class="flex flex-wrap -mx-3 mb-5">
    <div class="w-full px-3 mb-6 md:mb-0">
      <label class="block uppercase tracking-wide text-gray-700 text-xs font-light mb-1" for="password_input">
        Contraseña
      </label>
      <input class="appearance-none block w-full bg-gray-200 text-gray-darker border border-gray-200 rounded py-3 px-4 leading-tight focus:outline-none focus:bg-white focus:border-gray-600"
        id="password_input" type="password" placeholder="" wire:model.lazy="password_c" value="{{ old('password_c') }}" />
      @error('password_c') <p class="text-red-500 text-xs italic">{{ $message }}</p>@enderror
    </div>
  </div>
  <div class="mt-5">
    <button class="bg-green-500 hover:bg-green-800 text-white font-bold py-2 px-4 rounded" wire:click.prevent="store()" type="button">
      Crear
    </button>
    <button type="button" class="close-modal cursor-pointer bg-red-200 hover:bg-red-500 text-red-900 font-bold py-2 px-4 rounded" onclick="closeCreate('create_user_modal')">
      Cerrar
    </button>
  </div>
</form>
```

Figura 54. Vista del requerimiento 06

Fuente: Elaboración propia

En la figura 55 se muestra el controlador del RF-06, con el nombre UserComponent.php

```

public function store()

$this->validate([
    'name_c' => 'required|max:255',
    'phone_c' => 'required|unique:users,phone|regex:/[0-9]{9}/',
    'email_c' => 'required|unique:users,email|max:255',
    'password_c' => 'required|max:255',
    'document_id_c' => 'required|unique:users,document_id|max:255',
    'subsidiary_id_c' => 'required',
    'role_id_c' => 'required',
]);

User::create([
    'name' => $this->name_c,
    'phone' => $this->phone_c,
    'email' => $this->email_c,
    'password' => Hash::make($this->password_c),
    'document_id' => $this->document_id_c,
    'subsidiary_id' => $this->subsidiary_id_c,
    'role_id' => $this->role_id_c,
]);

$this->resetInputFields();

session()->flash('message', 'Usuario creado con exito');

$this->dispatchBrowserEvent('closeModal', ['modal' => 'create_user_modal', 'show' => 'confirmation_modal']);

```

Figura 55. Controlador del requerimiento 6

Fuente: Elaboración propia

RF-10: El sistema debe tener un módulo de productos en donde se podrá gestionar la búsqueda de estos por carácter y categoría.

En la figura 56 se muestra el modelo del RF-10, con el nombre Product.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Product extends Model
{
    use HasFactory;
    use SoftDeletes;

    protected $fillable = ['name',
        'code',
        'description',
        'brand',
        'price',
        'type_currency',
        'img_path',
        'category_id'];

    public function category()
    {
        return $this->belongsTo(Category::class);
    }

    public function getPriceAttribute($value)
    {
        return round($value, 2);
    }
}

```

Figura 56. Modelo del requerimiento 10

Fuente: Elaboración propia

En la figura 57 se muestra la vista del RF-10, con el nombre Product-component.blade.php

[illegible]

```

@can('manage_general_admins')
    <td class="border px-4 py-2">
        <button wire:click="edit({{ $product->product_id }})" class="bg-teal-300 cursor-pointer rounded p-2 mx-2 text-white">
            <i class="fas fa-edit"></i>
        </button>
    </td>
    <td class="border px-4 py-2">
        <button wire:click="drop({{ $product->product_id }})" class="bg-teal-300 cursor-pointer rounded p-2 mx-2 text-red-500">
            <i class="fas fa-trash"></i>
        </button>
    </td>
@endcan
</tr>
@endforeach
</tbody>
</table>
<div class="row">
    <div class="col-md-12">
        {{ $products->withQueryString()->links() }}
    </div>
</div>
</div>
</div>

```

Figura 57. Vista del requerimiento 10

Fuente: Elaboración propia

RF-12: El sistema debe tener un módulo de productos en donde se podrá editar un producto existente.

En la figura 58 se muestra el modelo del RF-12, con el nombre Product.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Product extends Model
{
    use HasFactory;
    use SoftDeletes;

    protected $fillable = ['name',
        'code',
        'description',
        'brand',
        'price',
        'type_currency',
        'img_path',
        'category_id'];

    public function category()
    {
        return $this->belongsTo(Category::class);
    }

    public function getPriceAttribute($value)
    {
        return round($value, 2);
    }
}
```

Figura 58. Modelo del requerimiento 12

Fuente: Elaboración propia

En la figura 59 se muestra la vista del RF-12, con el nombre update-product.blade.php

```
<form class="w-full">
<div class="flex flex-wrap -m-3 mb-6">
<div class="w-full px-3 mb-6 md:mb-8">
<div class="rounded rounded-t-lg overflow-hidden shadow max-w-xs my-3">

</div>
<div class="rounded rounded-t-lg overflow-hidden shadow max-w-xs my-3">

</div>
</div>
</div>
<div class="flex flex-wrap -m-3 mb-6">
<div class="w-full px-3 mb-6 md:mb-8">
<label class="block uppercase tracking-wide text-gray-700 text-xs font-light mb-1" for="image_input">
Imagen
</label>
<input type="file" class="bg-gray-200 hover:bg-gray-300 text-gray-800 font-bold py-2 px-4 rounded inline-flex items-center"
id="image_input" wire:model.lazy="img_file">
<error("img_file") > class="text-red-500 text-xs italic">{!! $message !!}</error>
</div>
</div>
<div class="flex flex-wrap -m-3 mb-6">
<div class="w-full md:w-px-3 mb-6 md:mb-8">
<label class="block uppercase tracking-wide text-gray-700 text-xs font-light mb-1" for="name_input">
Nombre
</label>
<input class="appearance-none block w-full bg-gray-200 text-gray-900 border border-gray-300 rounded py-3 px-4 leading-tight focus:outline-none focus:bg-white focus:border-gray-600"
id="name_input" type="text" placeholder="" wire:model.lazy="name_e" value="{!! old('name_e') !!}" />
<error("name_e") > class="text-red-500 text-xs italic">{!! $message !!}</error>
</div>
</div>
<div class="flex flex-wrap -m-3 mb-6">
<div class="w-full md:w-px-3 mb-6 md:mb-8">
<label class="block uppercase tracking-wide text-gray-700 text-xs font-light mb-1" for="brand_input">
Marca
</label>
<input class="appearance-none block w-full bg-gray-200 text-gray-900 border border-gray-300 rounded py-3 px-4 leading-tight focus:outline-none focus:bg-white focus:border-gray-600"
id="brand_input" type="text" placeholder="" wire:model.lazy="brand_e" value="{!! old('brand_e') !!}" />
<error("brand_e") > class="text-red-500 text-xs italic">{!! $message !!}</error>
</div>
<div class="block uppercase tracking-wide text-gray-700 text-xs font-light mb-1" for="code_input">
Codigo
</div>
<input class="appearance-none block w-full bg-gray-200 text-gray-900 border border-gray-300 rounded py-3 px-4 leading-tight focus:outline-none focus:bg-white focus:border-gray-600"
id="code_input" type="text" placeholder="" wire:model.lazy="code_e" value="{!! old('code_e') !!}" onkeyup="this.value = this.value.toUpperCase();" />
<error("code_e") > class="text-red-500 text-xs italic">{!! $message !!}</error>
</div>
</div>
<div class="flex flex-wrap -m-3 mb-6">
<div class="w-full px-3 mb-6 md:mb-8">
<label class="block uppercase tracking-wide text-gray-700 text-xs font-light mb-1" for="description_input">
Description
</label>
<textarea class="appearance-none block w-full bg-gray-200 text-gray-900 border border-gray-300 rounded py-3 px-4 leading-tight focus:outline-none focus:bg-white focus:border-gray-600"
id="description_input" type="text" placeholder="" wire:model.lazy="description_e" value="{!! old('description_e') !!}" />
<error("description_e") > class="text-red-500 text-xs italic">{!! $message !!}</error>
</div>
</div>
<div class="flex flex-wrap -m-3 mb-2">
<div class="w-full md:w-1/3 px-3 mb-6 md:mb-8">
<label class="block uppercase tracking-wide text-gray-900 text-xs font-light mb-1" for="type_currency_input">
Moneda
</label>
<div class="relative">
<select class="block appearance-none w-full bg-gray-200 border border-gray-300 text-gray-900 py-3 px-4 pr-8 rounded leading-tight focus:outline-none focus:bg-white focus:border-gray"
id="type_currency_input" wire:model.lazy="type_currency_e" value="{!! old('type_currency_e') !!}">
<option value="SOL">SOL</option>
<option value="DOLAR">DOLAR</option>
</select>
<div class="pointer-events-none absolute inset-y-0 right-0 flex items-center px-2 text-gray-900">
<svg class="fill-current h-4 w-4" xmlns="http://www.w3.org/2000/svg"
viewBox="0 0 20 20">
<path d="M0 0 20 20" />
</svg>
</div>
<error("type_currency_id_e") > class="text-red-500 text-xs italic">{!! $message !!}</error>
</div>
</div>
<div class="w-full md:w-1/3 px-3 mb-6 md:mb-8">
<label class="block uppercase tracking-wide text-gray-900 text-xs font-light mb-1" for="price_input">
Precio
</label>
<input
class="appearance-none block w-full bg-gray-200 text-gray-900 border border-gray-300 rounded py-3 px-4 leading-tight focus:outline-none focus:bg-white focus:border-gray"
id="price_input" type="text" placeholder="" wire:model.lazy="price_e" value="{!! old('price_e') !!}" pattern="[0-9]" />
<error("price_e") > class="text-red-500 text-xs italic">{!! $message !!}</error>
</div>
<div class="w-full md:w-1/3 px-3 mb-6 md:mb-8">
<label class="block uppercase tracking-wide text-gray-900 text-xs font-light mb-1" for="category_input">
Categoría
</label>
<div class="relative">
<select class="block appearance-none w-full bg-gray-200 border border-gray-300 text-gray-900 py-3 px-4 pr-8 rounded leading-tight focus:outline-none focus:bg-white focus:border-gray"
id="category_input" wire:model.lazy="category_id_e" value="{!! old('category_id_e') !!}">
<option value="">Hidden selected</option>
<@foreach ($categories_list as $category)>
<option value="{!! $category->id !!}">{!! $category->name !!</option>
</foreach>
</select>
<div class="pointer-events-none absolute inset-y-0 right-0 flex items-center px-2 text-gray-900">
<svg class="fill-current h-4 w-4" xmlns="http://www.w3.org/2000/svg"
viewBox="0 0 20 20">
<path d="M0 0 20 20" />
</svg>
</div>
<error("category_id_e") > class="text-red-500 text-xs italic">{!! $message !!}</error>
</div>
</div>
</div>
<div class="mt-5">
<button class="bg-green-500 hover:bg-green-800 text-white font-bold py-2 px-4 rounded" wire:click.prevent="update()" type="button">
Actualizar
</button>
<button type="button" class="close-modal cursor-pointer bg-red-200 hover:bg-red-500 text-red-900 font-bold py-2 px-4 rounded" onclick="closeCreate('update_product_modal')">
Cerrar
</button>
</div>
</form>
```

Figura 59. Vista del requerimiento 12

Fuente: Elaboración propia

En la figura 60 se muestra el controlador del RF-12, con el nombre ProductComponent.php

```
public function edit($id)
{
    $product = Product::findOrFail($id);
    $this->product_id_e = $id;
    $this->img_path_e = $product->img_path;
    $this->name_e = $product->name;
    $this->code_e = $product->code;
    $this->description_e = $product->description;
    $this->brand_e = $product->brand;
    $this->type_currency_e = $product->type_currency;
    $this->price_e = $product->price;
    $this->category_id_e = $product->category_id;
    $this->subsidiary_id_e = $product->subsidiary_id;

    $this->dispatchBrowserEvent('openModal', ['modal' => 'update_product_modal', 'show' => 'confirmation_modal']);
}
```

Figura 60. Controlador del requerimiento 12

Fuente: Elaboración propia

RF-13: El sistema debe tener un módulo de productos en donde se podrá eliminar un producto existente.

En la figura 61 se muestra el modelo del RF-13, con el nombre Product.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Product extends Model
{
    use HasFactory;
    use SoftDeletes;

    protected $fillable = ['name',
        'code',
        'description',
        'brand',
        'price',
        'type_currency',
        'img_path',
        'category_id'];

    public function category()
    {
        return $this->belongsTo(Category::class);
    }

    public function getPriceAttribute($value)
    {
        return round($value, 2);
    }
}
```

Figura 61. Modelo del requerimiento 13

Fuente: Elaboración propia

En la figura 62 se muestra la vista del RF-13, con el nombre delete.blade.php

```
<div id='delete_modal' class='modal-wrapper' wire:ignore.self>
  <div class='overlay close-modal'></div>
  <div class='modal'>
    <div class='modal-content shadow-lg p-5'>
      <div class='border-b p-2 pb-3 pt-0 mb-4'>
        <div class='flex justify-between items-center'>
          Desea eliminar esta información?
          <span class='close-modal cursor-pointer px-3 py-1 rounded-full bg-gray-100 hover:bg-gray-200'>
            <button type='button' class='fas fa-times text-gray-700' onclick='closeCreate('delete_modal')'></button>
          </span>
        </div>
      </div>
      <!-- Modal content -->
      <p>{{ session('message') }}</p>
      <div class='flex flex-wrap -mx-3 mb-2'>
        <div class='w-full md:w-1/4 px-3 mb-6 md:mb-0'>
          <button class='bg-orange-500 hover:bg-orange-800 text-white font-bold py-2 px-4 rounded' onclick='closeCreate('delete_modal')'> Cancelar </button>
        </div>
        <div class='w-full md:w-2/4 px-3 mb-6 md:mb-0'>
          </div>
        <div class='w-full md:w-1/4 px-3 mb-6 md:mb-0'>
          <button class='bg-red-500 hover:bg-red-800 text-white font-bold py-2 px-4 rounded' wire:click.prevent='delete()'> Eliminar </button>
        </div>
      </div>
    </div>
  </div>
</div>
```

Figura 62. Vista del requerimiento 13

Fuente: Elaboración propia

En la figura 63 se muestra el controlador del RF-13, con el nombre ProductComponent.php

```
public function delete()
{
    Product::find($this->delete_id)->delete();
    ProductStock::where('product_id', $this->delete_id)->delete();
    session()->flash('message', 'Producto eliminado con éxito');

    $this->dispatchBrowserEvent('closeModal', ['modal' => 'delete_modal', 'show' => 'confirmation_modal']);
}
```

Figura 63. Controlador del requerimiento 13

Fuente: Elaboración propia

RF-14: El sistema debe tener un módulo de productos en donde se podrá crear una nueva categoría.

En la figura 64 se muestra el modelo del RF-14, con el nombre Product.php y category.php

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Product extends Model
{
    use HasFactory;
    use SoftDeletes;

    protected $fillable = ['name',
        'code',
        'description',
        'brand',
        'price',
        'type_currency',
        'img_path',
        'category_id'];

    public function category()
    {
        return $this->belongsTo(Category::class);
    }

    public function getPriceAttribute($value)
    {
        return round($value, 2);
    }
}

```

```

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Category extends Model
{
    use HasFactory;
    use SoftDeletes;

    protected $fillable = ['name'];
}

```

Figura 64. Modelo del requerimiento 14

Fuente: Elaboración propia

En la figura 65 se muestra la vista del RF-14, con el nombre create-product.blade.php

Figura 65. Vista del requerimiento 14

En la figura 66 se muestra el controlador del RF-14, con el nombre ProductComponent.php

Figura 66. Controlador del requerimiento 14

RF-15: El sistema debe tener un módulo de productos en donde se podrá buscar una categoría existente.

En la figura 67 se muestra el modelo del RF-15, con el nombre Producto.php y Category.php

```
<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Product extends Model
{
    use HasFactory;
    use SoftDeletes;

    protected $fillable = ['name',
        'code',
        'description',
        'brand',
        'price',
        'type_currency',
        'img_path',
        'category_id'];

    public function category()
    {
        return $this->belongsTo(Category::class);
    }

    public function getPriceAttribute($value)
    {
        return round($value, 2);
    }
}

<?php

namespace App\Models;

use Illuminate\Database\Eloquent\Factories\HasFactory;
use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\SoftDeletes;

class Category extends Model
{
    use HasFactory;
    use SoftDeletes;

    protected $fillable = ['name'];
}
```

Figura 67. Modelo del requerimiento 15

Fuente: Elaboración propia

En la figura 68 se muestra la vista del RF-15, con el nombre create-category.blade.php

```

<div id="create_category_modal" class="modal-wrapper" wire:ignore.self>
  <div class="overlay close-modal"></div>
  <div class="modal modal-centered">
    <div class="modal-content shadow-lg p-5">
      <div class="border-b p-2 pb-3 pt-0 mb-4">
        <div class="flex justify-between items-center">
          <div>
            Crear Categoría
            <button type="button" class="close-modal cursor-pointer px-3 py-1 rounded-full bg-gray-100 hover:bg-gray-200" onclick="closeCreate('create_category_modal')">
              <i class="fas fa-times text-gray-700"></i>
            </button>
          </div>
        </div>
      </div>
      <div>
        <!-- Modal content -->
        <form class="w-full">
          <div class="flex flex-wrap -mx-3 mb-6">
            <div class="w-full px-3 mb-6 md:mb-8">
              <label class="block uppercase tracking-wide text-gray-700 text-xs font-light mb-1" for="name_input">
                Nombre
              </label>
              <input class="appearance-none block w-full bg-gray-200 text-gray-darker border border-gray-200 rounded py-3 px-4 leading-tight focus:outline-none focus:bg-white-500 focus:border-gray-600"
                id="name_input" type="text" placeholder="" wire:model.lazy="name_c" value="{{ old('name_c') }}" />
              <div>
                @error('name_c') <p class="text-red-500 text-xs italic">{{ $message }}</p>@enderror
              </div>
            </div>
          </div>
          <div class="mt-5">
            <button class="bg-green-500 hover:bg-green-600 text-white font-bold py-2 px-4 rounded" wire:click.prevent="store()" type="button">
              Crear
            </button>
            <button type="button" class="close-modal cursor-pointer bg-red-200 hover:bg-red-500 text-red-900 font-bold py-2 px-4 rounded" onclick="closeCreate('create_category_modal')">
              Cerrar
            </button>
          </div>
        </form>
      </div>
    </div>
  </div>
</div>

```

Figura 68. Vista del requerimiento 15

Fuente: Elaboración propia

En la figura 69 se muestra el controlador del RF-15, con el nombre

```

public $name_c;
public $name_e, $id_e;
public $delete_id;

public function render()
{
    return view('livewire.category-component', [
        'categories' => Category::orderBy('id', 'desc')->paginate(5),
    ]);
}

public function create()
{
    $this->resetInputFields();
}

public function store()
{
    $this->validate([
        'name_c' => 'required|max:255',
    ]);

    Category::create([
        'name' => $this->name_c,
    ]);

    $this->resetInputFields();

    session()->flash('message', 'Categoría creada con éxito');

    $this->dispatchBrowserEvent('closeModal', ['modal' => 'create_category_modal']);
}

```

Figura 69. Controlador del requerimiento 15

Fuente: Elaboración propia

PRUEBAS DE CAJA NEGRA DEL PRIMER SPINRT

PRUEBA DE CAJA NEGRA						
Código	PCN01			Fecha	23/12/2024	
Tarea	Acceso al sistema			Módulo	Login	
Descripción	El sistema debe tener una ventana que permita el inicio de sesión por medio de un usuario y contraseña					
Caso de prueba						
Precondiciones						
El usuario debe estar creado en la base de datos						
Pasos de la prueba						
Escenario 1: Ingreso Exitoso Dado que, me encuentro en el formulario de login, Cuando ingreso usuario y contraseña de manera correcta Entonces el sistema permite el ingreso. Escenario 2: Ingreso Fallido Dado que, me encuentro en el formulario de login, Cuando ingreso usuario y contraseña de manera incorrecta, Entonces aparece el mensaje “Error! Las credenciales o el usuario no son correctos...”						
Datos de entrada			Respuesta esperada	Coincide		Respuesta del sistema
Campo	Valor	Escenario		Sí	No	
		Prueba	Valores no permitidos	x		Se visualiza el mensaje de error
		Prueba	Valores correctos	x		Se logra acceder al sistema
		Prueba	Valores obligatorios	x		El sistema pide ingresar los datos obligatorios
Post condiciones						
No se aplica						
Defectos				Veredicto		
No existen				Pasó		
Observaciones				Probador		
No aplica				María Milagros Llauce Chapoñan		

Figura 70. Pruebas del HU01

Elaboración propia

PRUEBA DE CAJA NEGRA						
Código	PCN02			Fecha	23/12/2024	
Tarea	Administrar dashboard			Módulo	Dashboard	
Descripción	Se debe verificar la visualización y funcionalidad de la sección dashboard					
Caso de prueba						
Precondiciones						
Debe haber data generada en la base de datos						
Pasos de la prueba						
Escenario 1: Visualizar datos específicos Dado que, me encuentro en la pestaña del Dashboard, Cuando filtro por fecha de inicio, fecha de fin y sucursal, Entonces me brinda datos específicos. Escenario 2: Visualizar datos predeterminados Dado que, me encuentro en la pestaña del Dashboard Cuando no ingreso ningún valor en los filtros, Entonces aparece datos de manera predeterminada.						
Datos de entrada			Respuesta esperada	Coincide		Respuesta del sistema
Campo	Valor	Escenario		Sí	No	
		Prueba	Filtrar datos específicos	x		Facilita la data requerida
		Prueba	Se muestran todos los datos de forma predeterminada	x		Se muestran todos los datos sin filtro
Post condiciones						
No se aplica						
Defectos				Veredicto		
No existen				Pasó		
Observaciones				Probador		
No aplica				María Milagros Llauce Chapoñan		

Figura 71. Prueba del HU02

Elaboración propia

PRUEBA DE CAJA NEGRA						
Código		PCN03		Fecha		23/12/2024
Tarea		Administrar usuarios		Módulo		Usuarios
Descripción		Se debe validar la visualización y funcionalidad de la sección de usuarios				
Caso de prueba						
Precondiciones						
Debe haber data generada en la base de datos						
Pasos de la prueba						
Escenario 3: Agregar usuarios con éxito Dado que, me encuentro en el módulo de usuarios, Cuando doy clic en “Crear Usuario” e ingreso data en todos los campos, Entonces en el sistema figura un pop-up indicando “Usuario Creado” Escenario 4: Error al agregar un usuario Dado que, me encuentro en el módulo de productos, Cuando doy clic en “Crear Usuario” y no ingreso ningún dato, Entonces el sistema indica “Debe ingresar datos en los campos para su creación”						
Datos de entrada			Respuesta esperada	Coincide		Respuesta del sistema
Campo	Valor	Escenario		Sí	No	
		Prueba	Se agregó un usuario	x		Se agregó un usuario correctamente
		Prueba	Error al agregar un usuario	x		No registra el usuario
Post condiciones						
No se aplica						
Defectos				Veredicto		
No existen				Pasó		
Observaciones				Probador		
No aplica				María Milagros Llauce Chapoñan		

Figura 72. Prueba del HU03

Elaboración propia

PRUEBA DE CAJA NEGRA						
Código		PCN05		Fecha		23/12/2024
Tarea		Administrar productos		Módulo		Productos
Descripción		Se debe validar la visualización y funcionalidad de la sección de productos				
Caso de prueba						
Precondiciones						
Debe haber productos creados en la base de datos						
Pasos de la prueba						
Escenario 1: Agregar productos con éxito Dado que, me encuentro en el módulo de productos, Cuando doy clic en “Crear Producto” e ingreso data en todos los campos, Entonces en el sistema figura un pop-up indicando “Producto Creado” Escenario 2: Error al agregar un producto Dado que, me encuentro en el módulo de productos, Cuando doy clic en “Crear Producto” y no ingreso ningún dato, Entonces el sistema indica “Se debe ingresar datos para su creación” Escenario 3: Modificar un producto Dado que, me encuentro en el módulo de productos, Cuando doy click en el botón “Modificar” y edito los campos, Entonces el sistema indica “Producto Actualizado” Escenario 4: Eliminar productos con éxito Dado que, me encuentro en el módulo de productos, Cuando doy click en el botón “Borrar” y posteriormente hago click en “Eliminar” Entonces el sistema indica “Producto Eliminado”						
Datos de entrada			Respuesta esperada	Coincide		Respuesta del sistema
Campo	Valor	Escenario		Sí	No	
			Agregar productos	x		Se visualiza el usuario agregado
			Error al agregar un producto	x		Se visualiza un error y no se agrega el producto
			Modificar producto	x		Se modificó el producto
			Eliminar producto	x		El producto se eliminó
Post condiciones						
No se aplica						
Defectos				Veredicto		
No existen				Pasó		
Observaciones				Probador		
No aplica				María Milagros Llauce Chapoñan		

Figura 73. Prueba del HU05

Elaboración propia

PRUEBA DE CAJA NEGRA						
Código	PCN06			Fecha	23/12/2024	
Tarea	Administrar categoría			Módulo	Categoría	
Descripción	Se debe probar la funcionalidad de la opción categorías					
Caso de prueba						
Precondiciones						
No aplica						
Pasos de la prueba						
Escenario 1: Crear nueva categoría Dado que, me encuentro en el módulo de productos, Cuando doy click en “Crear” e ingreso el nombre de la categoría, Entonces el sistema lista la nueva categoría. Escenario 2: No se especifican criterios de filtrado Dado que, me encuentro en el módulo de productos, Cuando doy <u>click</u> en “Crear” y no ingreso ningún dato, Entonces aparece el error “Se debe ingresar un nombre*.” Escenario 3: Buscar una categoría Dado que, me encuentro en el módulo de productos, Cuando doy <u>click</u> en “Categorías”, Entonces el sistema muestra todas las categorías existentes.						
Datos de entrada			Respuesta esperada	Coincide		Respuesta del sistema
Campo	Valor	Escenario		Sí	No	
		Prueba	Crear categoría	x		Se crea una nueva categoría
		Prueba	No se aplican filtros	x		Se muestran todas las categorías
		Prueba	Buscar categoría	x		Se muestra la categoría
		Prueba		x		
Post condiciones						
No se aplica						
Defectos				Veredicto		
No existen				Pasó		
Observaciones				Probador		
No aplica				María Milagros Llauce Chapoñan		

Figura 74. Prueba del HU06

Elaboración propia

Tabla 8. Retrospectiva del sprint 1

Problema	Causas	Acciones
Se presentaron algunos errores de desarrollo.	Mal manejo de las versiones de desarrollo.	Se empezó a crear versiones con cada cambio y/o mejora.

Fuente: Elaboración propia

ACTAS DE PLANIFICACIÓN Y REVISIÓN DEL SPRINT 1

Acta de reunión de planificación del Sprint

Fecha: 21/09/2024

Scrum Máster: María Milagros Llaue Chapoñan

Product Owner: Olimpia Gómez Alarcón

SPRINT 1

En el presente documento se da la conformidad sobre los objetivos que se realizarán en el primer sprint por parte del equipo de desarrollo y a su vez, informar las historias de usuario que contemplan el sprint.

En el sprint 1 se determinó lo siguiente:

HU	Nº requerimiento	Objetivos
HU1	RF - 01	El sistema debe tener una ventana que permita el inicio de sesión por medio de un usuario y contraseña
HU2	RF - 02	El sistema debe tener un módulo dashboard en donde se visualice un gráfico estadístico de productos no vendidos.
	RF - 03	El sistema debe tener un módulo dashboard en donde se visualice un gráfico estadístico de productos vendidos según categorías.
	RF - 04	El sistema debe tener un módulo dashboard en donde se visualice una tabla de productos con bajo stock.
HU3	RF - 06	El sistema debe tener un módulo de usuarios en donde se podrá crear un nuevo usuario
HU5	RF - 10	El sistema debe tener un módulo de productos en donde se podrá gestionar la búsqueda de estos por carácter y categoría
	RF - 11	El sistema debe tener un módulo de productos en donde se podrá crear un nuevo producto
	RF - 12	El sistema debe tener un módulo de productos en donde se podrá editar un producto existente
	RF - 13	El sistema debe tener un módulo de productos en donde se podrá eliminar un producto existente
HU6	RF - 14	El sistema debe tener un módulo de productos en donde se podrá crear una nueva categoría
	RF - 15	El sistema debe tener un módulo de productos en donde se podrá buscar una categoría existente

Firma de conformidad

Olimpia Gomez Alarcon
Sub Gerente General Neo Copy Import

Figura 75. Acta de planificación Sprint 1

Fuente: Elaboración propia

Acta de reunión de revisión del Sprint

Fecha: 05/10/2024

Scrum Máster: María Milagros Llauce Chapoñan

Product Owner: Olimpia Gómez Alarcón

SPRINT 1

En el presente documento se da la conformidad que la Sra. Olimpia Gomez Alarcon estuvo durante el desarrollo de cada objetivo pactado en la reunión del sprint 1.

En el sprint 1 se desarrolló lo siguiente:

HU	Nº requerimiento	Objetivos
HU1	RF - 01	El sistema debe tener una ventana que permita el inicio de sesión por medio de un usuario y contraseña
HU2	RF - 02	El sistema debe tener un módulo dashboard en donde se visualice un gráfico estadístico de productos no vendidos.
	RF - 03	El sistema debe tener un módulo dashboard en donde se visualice un gráfico estadístico de productos vendidos según categorías.
	RF - 04	El sistema debe tener un módulo dashboard en donde se visualice una tabla de productos con bajo stock.
HU3	RF - 06	El sistema debe tener un módulo de usuarios en donde se podrá crear un nuevo usuario
HU5	RF - 10	El sistema debe tener un módulo de productos en donde se podrá gestionar la búsqueda de los mismos por carácter y categoría
	RF - 11	El sistema debe tener un módulo de productos en donde se podrá crear un nuevo producto
	RF - 12	El sistema debe tener un módulo de productos en donde se podrá editar un producto existente
	RF - 13	El sistema debe tener un módulo de productos en donde se podrá eliminar un producto existente
HU6	RF - 14	El sistema debe tener un módulo de productos en donde se podrá crear una nueva categoría
	RF - 15	El sistema debe tener un módulo de productos en donde se podrá buscar una categoría existente

Firma de conformidad



Olimpia Gomez Alarcon
Sub Gerente General Neo Copy Import

Figura 76. Acta de revisión del sprint 1

Fuente: Elaboración propia

SPRINT 2
ANÁLISIS DEL SEGUNDO SPRINT

En la **tabla 9** se muestran los 8 requerimientos que conforman el sprint 2.

Tabla 9. Sprint 2

HU	Nº	Requerimiento
HU7	RF - 16	El sistema debe tener un módulo de ventas en donde se podrá realizar una nueva venta
	RF - 17	El sistema debe tener un módulo de ventas en donde al realizar una venta genera un comprobante
	RF - 18	El sistema debe tener un módulo de cancelar venta en donde se gestionará las cancelaciones de ventas
HU8	RF - 19	El sistema debe tener un módulo de abastecimiento en donde se podrá gestionar un nuevo abastecimiento
HU10	RF - 25	El sistema debe permitir generar un documento EXCEL con el detalle de la rotación de inventarios.
	RF - 26	El sistema debe permitir generar un documento EXCEL con el detalle del porcentaje del inventario sin rotación.
	RF - 27	El sistema debe permitir generar un documento EXCEL con el detalle de la duración del inventario en ese mes.
	RF - 28	El sistema debe permitir generar un documento EXCEL con el detalle del valor económico del inventario.

Fuente: Elaboración propia

FUNCIONALIDADES DEL SEGUNDO SPRINT

La **Tabla 10** ofrece una descripción completa de las funcionalidades que implementará el sistema. Además, ilustra cómo interactúan los usuarios con la arquitectura de software para satisfacer cada uno de los requerimientos definidos en el Sprint 2.

Tabla 10. Funcionalidades del sprint 2

R.F.	Funcionalidad
RF-16	El usuario visualiza un formulario donde completará todos los campos necesarios para concretar una venta. El controlador recibe estos valores y actualiza el modelo en la base de datos.
RF-17	El usuario visualiza un formulario donde completará todos los campos necesarios para concretar una venta. Al generar un comprobante en formato PDF, es donde actúa el controlador con los datos de la venta. La vista muestra un resumen de la venta con los datos completados en el formulario. Por último, el modelo nos proporciona los datos necesarios para generar el comprobante.
RF-18	El usuario visualiza una lista de las ventas concretadas en el sistema. Hacer click en "Anular", se envía una solicitud al controlador con el ID de la venta a anular. Posteriormente, un cuadro de diálogo que pregunta al usuario si está seguro de anular la venta, ofreciendo botones de "Confirmar" y "Cancelar". Por último, se muestra un mensaje de confirmación al usuario indicando que la venta ha sido anulada correctamente.
RF-19	El usuario visualiza un formulario donde se deberá seleccionar el proveedor del producto, ingresar el nombre del producto y seleccionar la categoría. El controlador recibe los datos, los valida y crea un objeto de producto para mostrar los resultados. Posteriormente, se redirige a

	la vista que muestra el modal para ingresar la cantidad a agregar. Por último, El controlador actualiza el objeto de producto con la cantidad y lo guarda en la base de datos a través del modelo.
RF-25	El usuario visualiza un formulario de rotación de inventario mensual donde se debe seleccionar la categoría, la sucursal, el mes y año. Luego de seleccionar los datos al hacer click en “Generar Excel”, el controlador valida los datos ingresados en el formulario y luego consulta al modelo para obtener los datos del inventario que coincidan con los filtros. Por último, la vista devuelve al usuario un archivo Excel con los datos solicitados.
RF-26	El usuario visualiza un formulario de porcentaje de rotación de inventario y selecciona los criterios de búsqueda y hace click en “Generar Excel”. El controlador recibe la solicitud, valida los datos y llama al modelo para obtener los datos de la base de datos. El controlador genera el archivo Excel utilizando una librería. Posteriormente, el modelo devuelve los resultados al controlador en un formato adecuado para generar el documento Excel.
RF-27	El usuario visualiza un formulario de la duración del inventario mensual y selecciona los criterios de búsqueda y hace click en “generar Excel”. El controlador recibe la solicitud, valida los datos y llama al modelo para obtener los datos de la base de datos. El controlador genera el archivo Excel utilizando una librería. Posteriormente, el modelo devuelve los resultados al controlador en un formato adecuado para generar el documento Excel.

RF-28	El usuario visualiza un formulario del valor económico mensual y selecciona los criterios de búsqueda y hace click en “generar Excel”. El controlador recibe la solicitud, valida los datos y llama al modelo para obtener los datos de la base de datos. El controlador genera el archivo Excel utilizando una librería. Posteriormente, el modelo devuelve los resultados al controlador en un formato adecuado para generar el documento Excel.
--------------	--

Fuente: Elaboración propia

DISEÑO DEL SEGUNDO SPRINT

Diseño de Prototipos:

En la Figura 77 se muestra el prototipo del requerimiento funcional 16 la cual fue aprobado por el Product Owner para realizar una venta.

El prototipo muestra una interfaz web para el "Sistema de inventario". La barra de direcciones indica la URL "https://neocopyimport/ventas". El encabezado de la página incluye el logo "NEO COPY IMPORT" y el nombre de usuario "San Isidro - Administrador".

El menú lateral izquierdo contiene las siguientes opciones: Dashboard, Usuarios, Productos, Ventas (seleccionada), Acciones y Reportes.

El formulario principal, titulado "Ventas", contiene los siguientes campos:

- NOMBRE: Campo de texto.
- DNI/RUC: Campo de texto.
- DIRECCIÓN: Campo de texto.
- TELÉFONO: Campo de texto.
- FECHA: Campo de texto con formato " / / " y un icono de calendario.
- BUSCADOR: Campo de texto.
- CATEGORÍA: Selector de lista desplegable.

Figura 77. Prototipo para el requerimiento RF-16.

Fuente: Elaboración propia

NEO COPY IMPORT

San Isidro - Admin Principal

Dashboard

Usuarios

Productos

Ventas

Cancelar Venta

Acciones

Abastecimiento

Reportes

Ventas por producto

Ventas por vendedor

Stock de productos

Producto

Ventas

Abastecimiento

Rotacion de inventario

Porcentaje Rotacion inventario

Duracion de inventario

Valor economico de inventario

Ventas

NOMBRE

DNI/RUC

DIRECCION

TELEFONO

FECHA

09/01/2025 17:37

BUSCADOR

Buscador

CATEGORIA

Todos

Figura 78. Realizar venta

Fuente: Elaboración propia

En la Figura 79 se muestra el prototipo del requerimiento funcional 17 la cual fue aprobado por el Product Owner para generar un comprobante de venta

Sistema de inventario

https://neocopyimport/ventas-realizadas

NEO COPY IMPORT

San Isidro - Administrador

Dashboard

Usuarios

Productos

✓ Ventas

Cancelar Venta

Acciones

Reportes

Ventas Realizadas

VENDEDOR

FECHA DE INICIO

FECHA DE FIN

Nombre	DNI/RUC	Direccion	Telefono	Generador Por	Fecha	Acciones

Figura 79. Prototipo para el requerimiento RF-17.

Fuente: Elaboración propia

NEO COPY IMPORT SAC
San Isidro, Lima
TELEFONO: 123414234

CLIENTE: _____
DIRECCIÓN: _____
DNI/RUC: _____
TELEFONO: _____
VENDEDOR: _____

CANTIDAD	CÓDIGO	NOMBRE	MARCA	P. UNITARIO	IMPORTE

RUC 123456789
VENTA
N° 1
FECHA
2025-01-19 11:09:00

Figura 81. Prototipo para el requerimiento RF-18.

Fuente: Elaboración propia

San Isidro - Admin Principal

Ventas realizadas

VENDEDOR: Todos FECHA INICIO: 09/01/2025 FECHA FIN: 10/01/2025

Nombre	DNI/RUC	Direccion	Telefono	Generado por	Fecha	Acciones	Acciones
Milagros	78887494	asds	962263297	Admin Principal	2025-01-09 17:37:00		

Figura 82. Cancelar venta

Fuente: Elaboración propia

En la Figura 83 se muestra el prototipo del requerimiento funcional 19 la cual fue aprobado por el Product Owner.

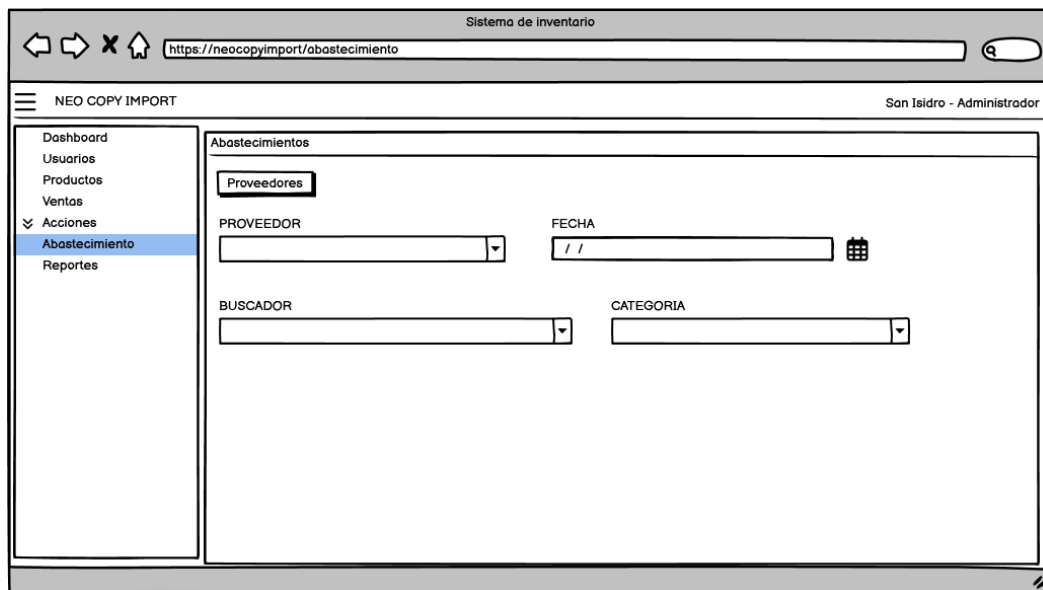


Figura 83. Prototipo para el requerimiento RF-19.

Fuente: Elaboración propia

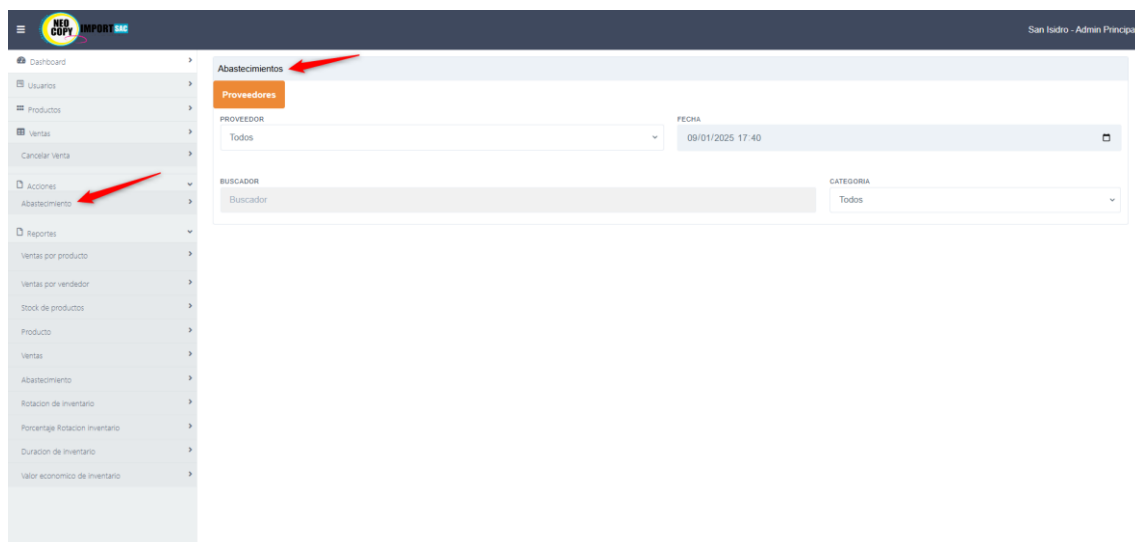
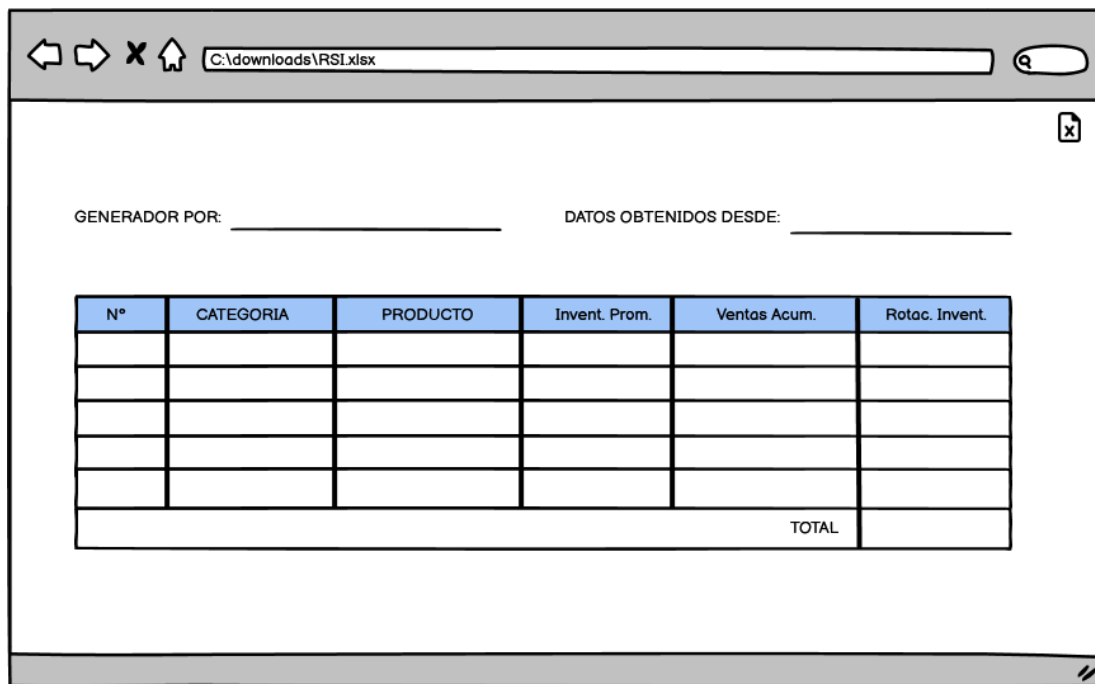


Figura 84. Realizar abastecimiento

Fuente: Elaboración propia

En la Figura 85 se muestra el prototipo del requerimiento funcional 25 la cual fue aprobado por el Product Owner para la vista del reporte de rotación de inventarios.

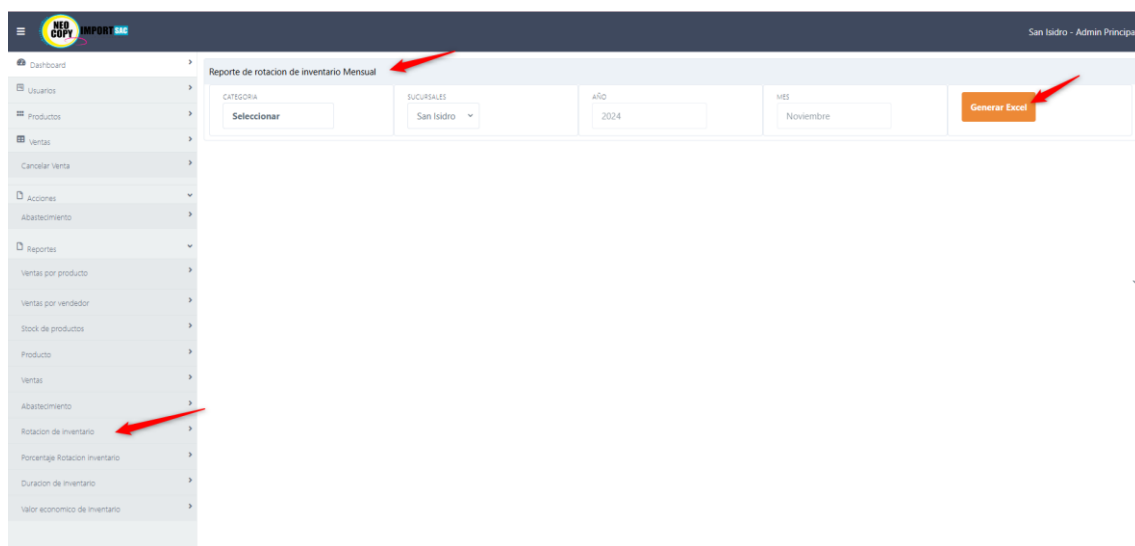


Generador POR: _____ DATOS OBTENIDOS DESDE: _____

Nº	CATEGORIA	PRODUCTO	Invent. Prom.	Ventas Acum.	Rotac. Invent.
TOTAL					

Figura 85. Prototipo para el requerimiento RF-25

Fuente: Elaboración propia



San Isidro - Admin Principal

Reporte de rotación de inventario Mensual

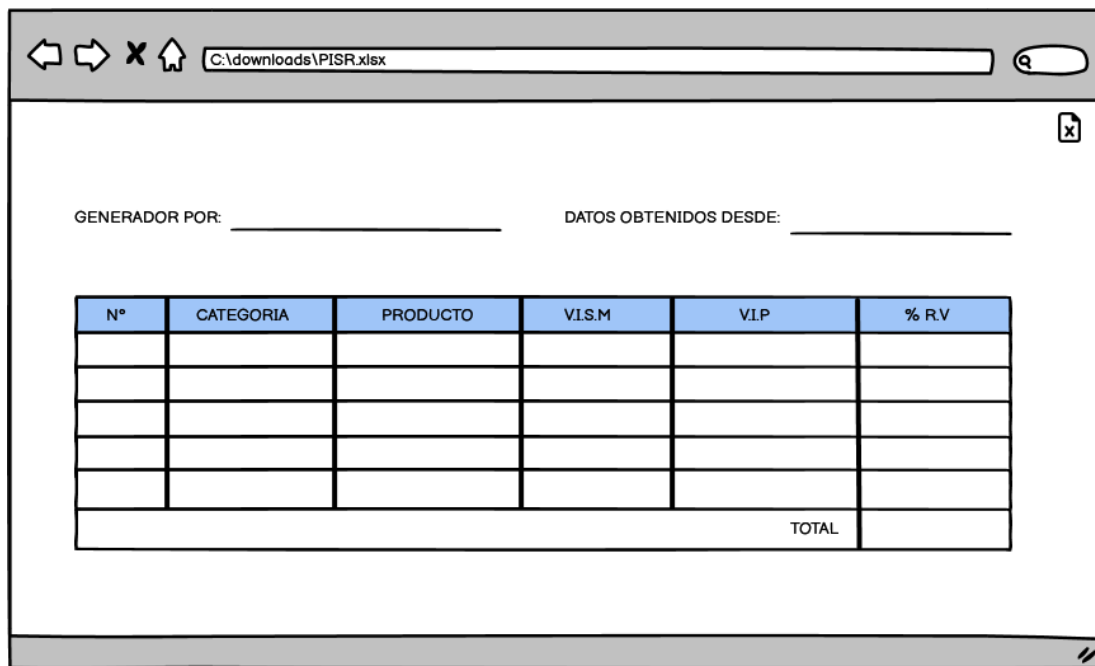
CATEGORIA: BUCURSALES: AÑO: MES:

Rotación de inventario

Figura 86. Reporte rotación de inventarios

Fuente: Elaboración propia

En la Figura 87 se muestra el prototipo del requerimiento funcional 26 la cual fue aprobado por el Product Owner para la vista del reporte del porcentaje de inventario sin rotación

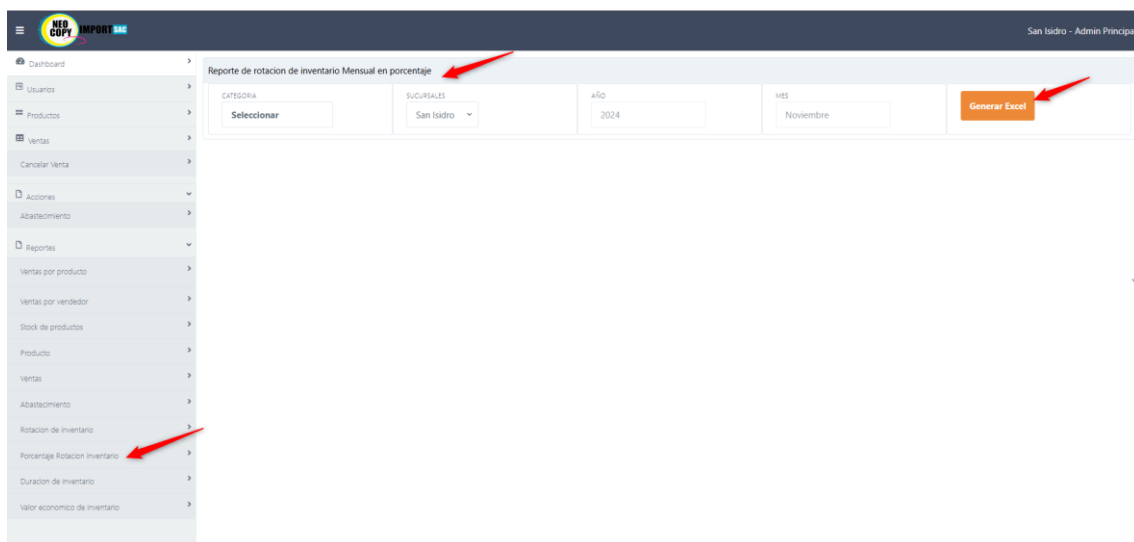


Generador POR: _____ DATOS OBTENIDOS DESDE: _____

N°	CATEGORIA	PRODUCTO	V.I.S.M	V.I.P	% R.V
TOTAL					

Figura 87. Prototipo para el requerimiento RF-26

Fuente: Elaboración propia



San Isidro - Admin Principal

Reporte de rotación de inventario Mensual en porcentaje

CATEGORIA: SUCCURSALES: AÑO: MES:

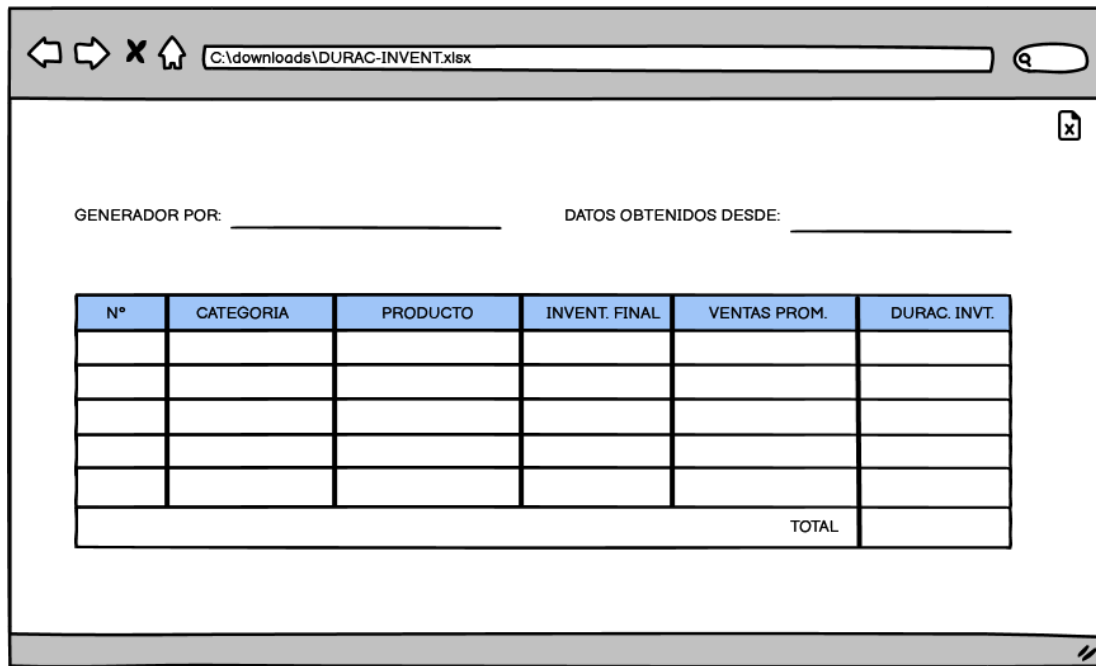
Rotación de inventario

Porcentaje Rotación inventario

Figura 88. Reporte porcentaje de inventario sin rotación

Fuente: Elaboración propia

En la Figura 89 se muestra el prototipo del requerimiento funcional 27 la cual fue aprobado por el Product Owner para la vista del reporte de duración del inventario.

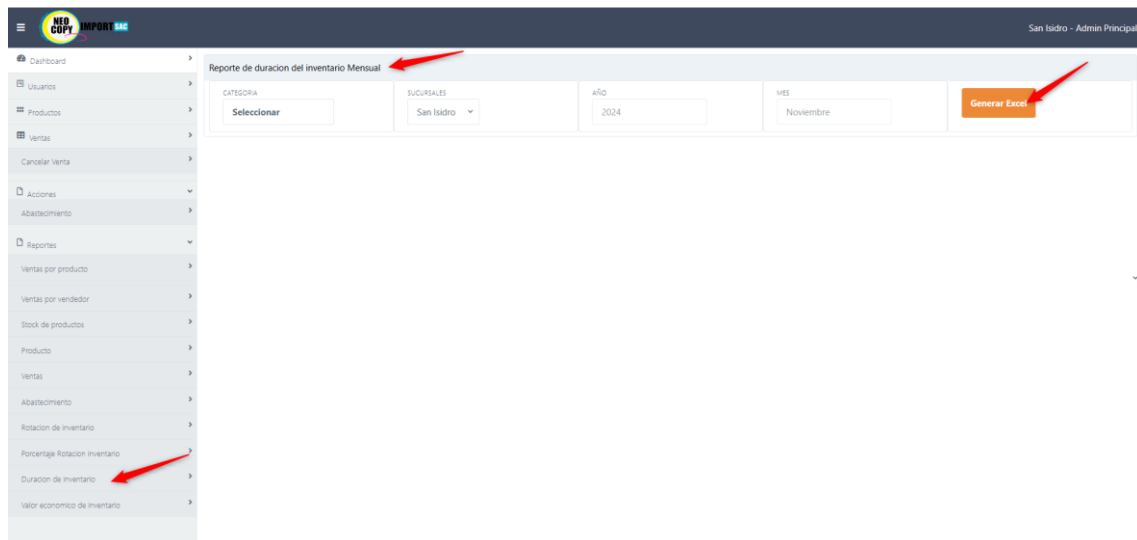


Generador POR: _____ DATOS OBTENIDOS DESDE: _____

N°	CATEGORIA	PRODUCTO	INVENT. FINAL	VENTAS PROM.	DURAC. INVT.
TOTAL					

Figura 89. Prototipo para el requerimiento RF-27

Fuente: Elaboración propia



San Isidro - Admin Principal

Reporte de duración del inventario Mensual

CATEGORIA: SUCURSALES: AÑO: MES:

Dashboard

- Usuarios
- Productos
- Ventas
- Cancelar venta
- Acciones
- Abastecimiento
- Reportes
- Ventas por producto
- Ventas por vendedor
- Stock de productos
- Producto
- Ventas
- Abastecimiento
- Rotación de inventario
- Porcentaje Rotación inventario
- Duración de inventario
- Valor económico de inventario

Figura 90. Reporte duración del inventario

Elaboración propia

En la Figura 91 se muestra el prototipo del requerimiento funcional 28 la cual fue aprobado por el Product Owner para la vista del reporte del valor económico del inventario

GENERADOR POR: _____ DATOS OBTENIDOS DESDE: _____

N°	CATEGORIA	PRODUCTO	COSTO VENTA	VALOR INVEN. FISICO	VALOR ECON.
TOTAL					

Figura 91. Prototipo para el requerimiento RF-28.

Fuente: Elaboración propia

Reporte de valor economico del inventario Mensual

CATEGORIA: SUCURSALES: Año: MES:

Figura 92. Reporte Valor económico del inventario

Fuente: Elaboración propia

ENTIDADES DEL SEGUNDO SPRINT:

En la Figura 93 se muestran las entidades del Sprint 2



Figura 93. Entidades del Sprint 2

Fuente: Elaboración Propia

DIAGRAMA ENTIDAD / RELACIÓN DEL SEGUNDO SPRINT

En la Figura 94 se muestra el diagrama conceptual de la base de datos del Sprint 2

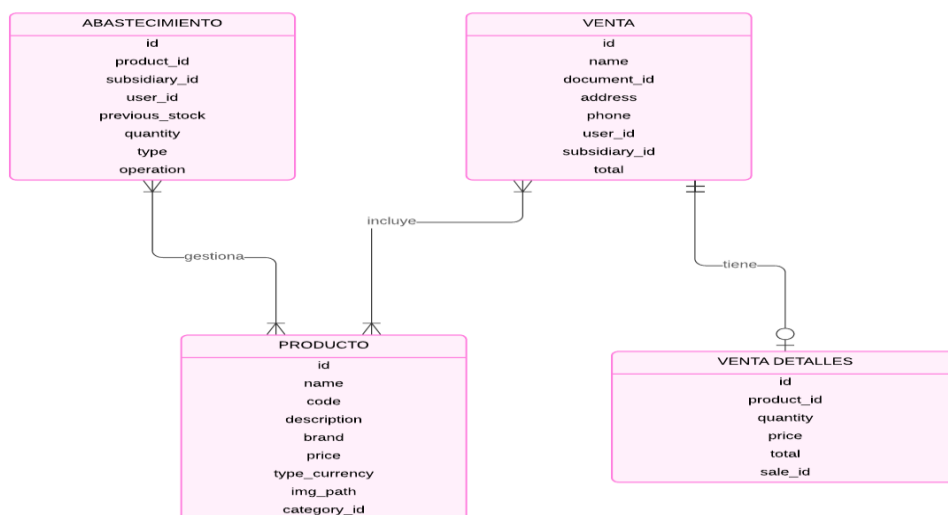


Figura 94. Diagrama Entidad / Relación del Sprint 2

Fuente: Elaboración Propia

DIAGRAMA LÓGICO DEL SEGUNDO SPRINT

En la figura 95 se muestra el diagrama físico que representa la descripción de los datos con el mayor detalle, su estructura y las relaciones de los datos fundamentales del negocio.

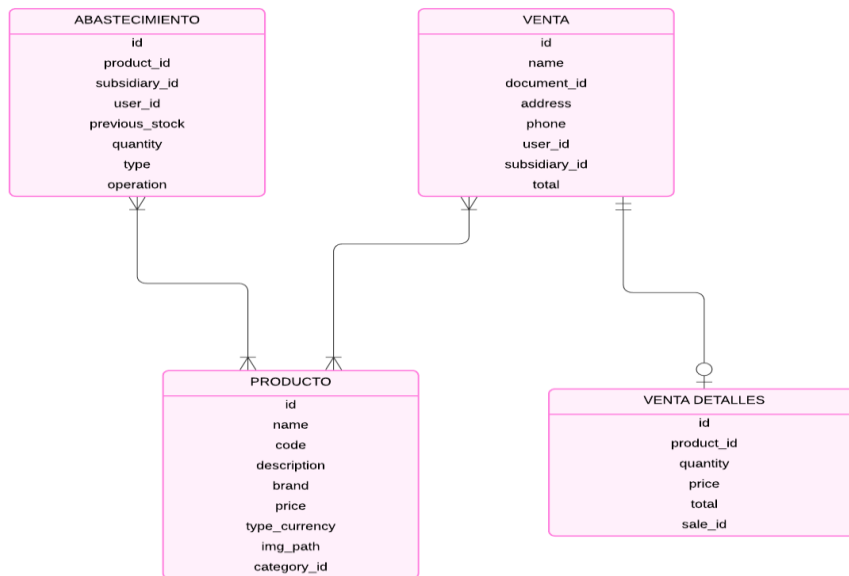


Figura 95. Diagrama Lógico del Sprint 2

Fuente: Elaboración Propia

DIAGRAMA FÍSICO DEL SEGUNDO SPRINT

En la figura 96 se muestra el diagrama físico que representa como se construirá el modelo en la base de datos.

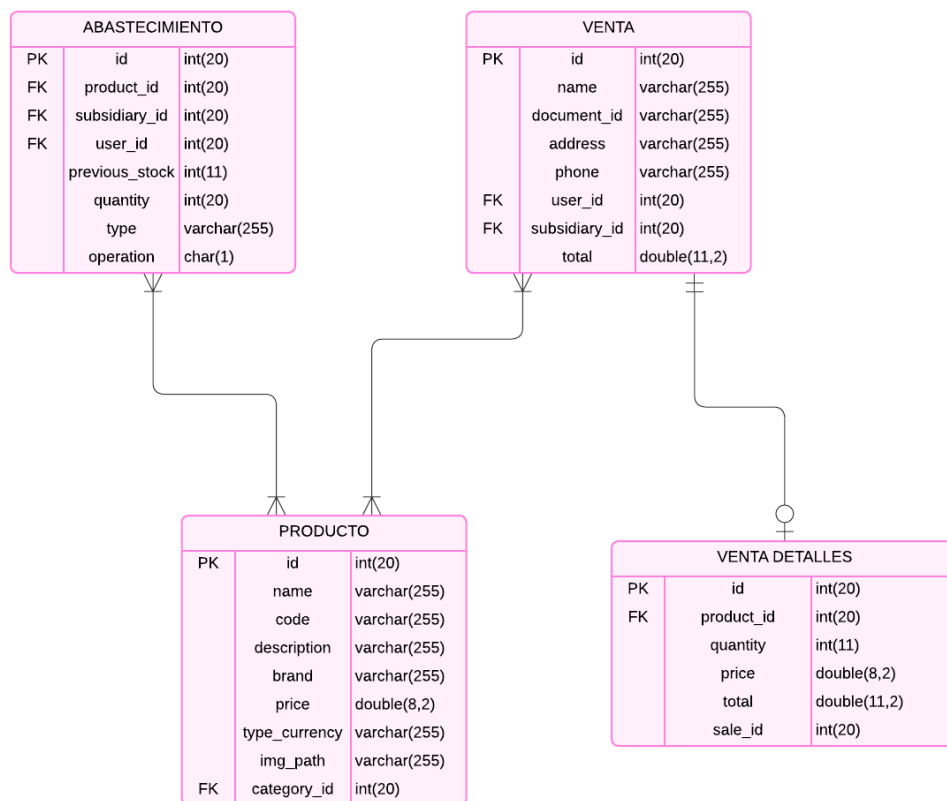


Figura 96. Diagrama Físico del Sprint 2

Fuente: Elaboración Propia

IMPLEMENTACIÓN DEL SEGUNDO SPRINT

Diagrama de la Base de Datos:

En la Figura 97 tenemos el diseño de la base de datos en MySQL que vamos a usar para el Sprint 2.

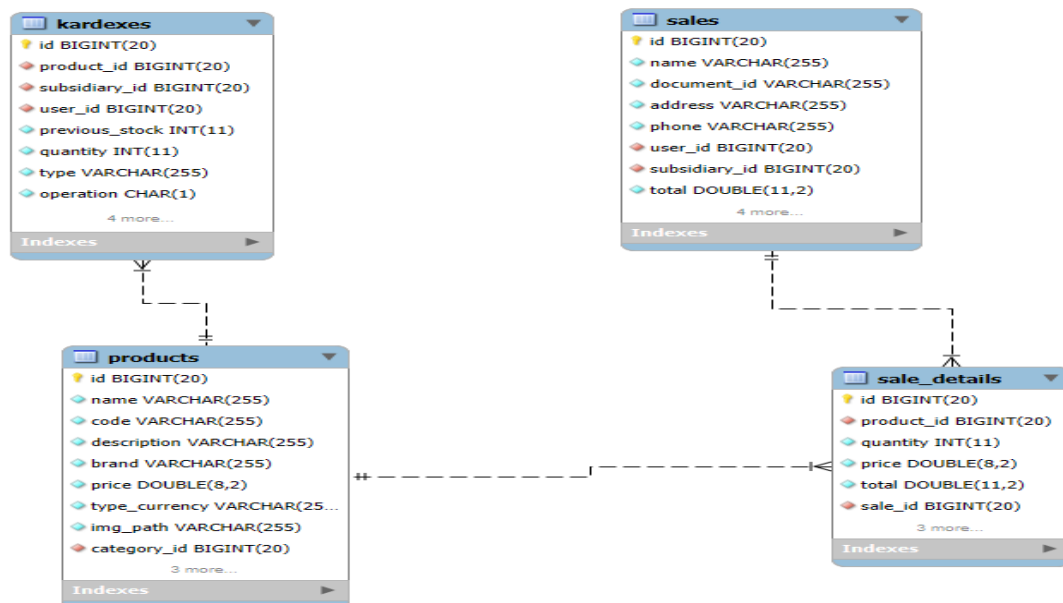


Figura 97. Diagrama de la base de datos del Sprint 2

Fuente: Elaboración Propia

RF-16: El sistema debe tener un módulo de ventas en donde se podrá realizar una nueva venta

En la figura 98 se muestra el modelo del RF-16:

```

1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7  use Illuminate\Database\Eloquent\SoftDeletes;
8
9  class Sale extends Model
10 {
11     use HasFactory;
12     use SoftDeletes;
13
14     protected $fillable = ['name', 'document_id', 'address', 'phone', 'user_id', 'subsidiary_id', 'total', 'created_date'];
15
16     public function user()
17     {
18         return $this->belongsTo(User::class);
19     }
20
21     public function subsidiary()
22     {
23         return $this->belongsTo(Subsidiary::class);
24     }
25
26     public function getTotalAttribute($value)
27     {
28         return round($value, 2);
29     }
30 }

```

Figura 98. Modelo del requerimiento 16

Fuente: Elaboración propia

En la figura 99 se muestra la vista del RF-16:

```
1 @extends('layouts.system-layout')
2 @section('style')
3 style:
4 {
5     font-size: 14px;
6 }
7
8 body {
9     font: 14px Arial;
10 }
11
12 /*The container must be positioned relative*/
13 .autocomplete {
14     position: relative;
15     display: inline-block;
16 }
17
18 input {
19     border: 1px solid transparent;
20     background-color: #f1f1f1;
21     padding: 10px;
22     font-size: 14px;
23 }
24
25 input[type=text] {
26     background-color: #f1f1f1;
27     width: 100%;
28 }
29
30 input[type=submit] {
31     background-color: #d9d9d9;
32     color: #000;
33     cursor: pointer;
34 }
35
36 .autocomplete-items {
37     position: absolute;
38     border: 1px solid #f0f0f0;
39     border-bottom: none;
40     border-top: none;
41     width: 100%;
42     /*position the autocomplete items to be the same width as the container*/
43     top: 100%;
44     left: 0;
45     right: 0;
46 }
47
48 .autocomplete-items div {
49     padding: 10px;
50     cursor: pointer;
51     background-color: #fff;
52     border-bottom: 1px solid #f0f0f0;
53 }
54
55 /*when hovering an item*/
56 .autocomplete-items div:hover {
57     background-color: #f0f0f0;
58 }
59
60 /*when navigating through the items using the arrow keys*/
61 .autocomplete-active {
62     background-color: #d9d9d9;
63     color: #000;
64 }
65
66 </style>
```

Figura 99. Vista del requerimiento 16

Fuente: Elaboración propia

En la figura 100 se muestra el controlador del RF-16:

```
app > Http > Livewire > SaleComponent.php
18 {
19     {
20         foreach ($this->array_sale_details as $sale_detail)
21         {
22             $total = $total + ($sale_detail["quantity"] * $sale_detail["price"]);
23         }
24
25         $id = Sale::create([
26             'name' => $this->name,
27             'document_id' => $this->document_id,
28             'address' => $this->address,
29             'phone' => $this->phone,
30             'total' => $total,
31             'user_id' => Auth::user()->id,
32             'subsidiary_id' => Auth::user()->subsidiary_id,
33             'created_date' => $this->created_date,
34         ]->id);
35
36         foreach ($this->array_sale_details as $sale_detail)
37         {
38             SaleDetail::create([
39                 'product_id' => $sale_detail["product_id"],
40                 'quantity' => $sale_detail["quantity"],
41                 'price' => $sale_detail["price"],
42                 'total' => $sale_detail["quantity"] * $sale_detail["price"],
43                 'sale_id' => $id,
44             ]);
45
46             $product_stock_sales = ProductStock::findOrFail($sale_detail["product_stock_id"]);
47
48             $stock::create([
49                 'product_id' => $sale_detail["product_id"],
50                 'subsidiary_id' => Auth::user()->subsidiary_id,
51                 'user_id' => Auth::user()->id,
52                 'previous_stock' => $product_stock_sales->stock,
53                 'quantity' => $sale_detail["quantity"],
54                 'type' => 'VENTA',
55                 'operation' => '-',
56                 'created_date' => now(),
57             ]);
58
59             $product_stock_sales->update([
60                 'stock' => $product_stock_sales->stock - $sale_detail["quantity"],
61             ]);
62         }
63
64         $this->resetInputFields();
65
66         session()->flash('message', 'Venta creada con éxito');
67
68         $this->dispatchBrowserEvent('closeModal', ['modal' => 'validation_modal', 'show' => 'confirmation_modal']);
69         $this->dispatchBrowserEvent('openPDF', ['url' => '/sales/pdf/' . $id]);
70     }
71 }
```

Figura 100. Controlador del requerimiento 16

Fuente: Elaboración propia

RF-17: El sistema debe tener un módulo de ventas en donde al realizar una venta genera un comprobante

En la figura 101 se muestra el modelo del RF-17:

```
app > Models > SaleDetail.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7  use Illuminate\Database\Eloquent\SoftDeletes;
8
9  class SaleDetail extends Model
10 {
11     use HasFactory;
12     use SoftDeletes;
13
14     protected $fillable = ['product_id','quantity','price','total','sale_id'];
15
16     public function product()
17     {
18         return $this->belongsTo(Product::class);
19     }
20
21     public function sale()
22     {
23         return $this->belongsTo(Sale::class);
24     }
25
26     public function getTotalAttribute($value)
27     {
28         return round($value, 2);
29     }
30
31     public function getPriceAttribute($value)
32     {
33         return round($value, 2);
34     }
35 }
36
```

Figura 101. Modelo del requerimiento 17

Fuente: Elaboración propia

En la figura 102 se muestra la vista del RF-17:

```

1 @extends('layouts.system-layout')
2 @section('styles')
3     <style>
4     {
5         box-sizing: border-box;
6     }
7
8     body {
9         font: 14px Arial;
10    }
11
12    /*The container must be positioned relative*/
13    .autocomplete {
14        position: relative;
15        display: inline-block;
16    }
17
18    input {
19        border: 1px solid transparent;
20        background-color: #ffffff;
21        padding: 5px;
22        font-size: 14px;
23    }
24
25    input[type="text"] {
26        background-color: #ffffff;
27        width: 100%;
28    }
29
30    input[type="submit"] {
31        background-color: #d9d9d9;
32        color: #fff;
33        cursor: pointer;
34    }
35
36    .autocomplete-items {
37        position: absolute;
38        border: 1px solid #d9d9d9;
39        border-bottom: none;
40        border-top: none;
41        z-index: 99;
42        /*position the autocomplete items to be the same width as the container*/
43        top: 100%;
44        left: 0;
45        right: 0;
46    }
47
48    .autocomplete-items div {
49        padding: 5px;
50        cursor: pointer;
51        background-color: #fff;
52        border-bottom: 1px solid #d9d9d9;
53    }
54
55    /*when hovering an item*/
56    .autocomplete-items div:hover {
57        background-color: #d9d9d9;
58    }
59
60    /*when navigating through the items using the arrow keys*/
61    .autocomplete-active {
62        background-color: #d9d9d9;
63        color: #ffffff;
64    }
65    </style>
66

```

Figura 102. Vista del requerimiento 17

Fuente: Elaboración propia

En la figura 103 se muestra el controlador del RF-17, con el nombre

```

1 SalesModelReportComponent.php X
2
3 app > http > livewire > SalesModelReportComponent.php
4
5 class SalesModelReportComponent extends Component
6 {
7     public $subsidiary_filter;
8     public $seller_list;
9
10    public $init_date;
11    public $end_date;
12
13    public $list_months = ["Enero", "Febrero", "Marzo", "Abril", "Mayo", "Junio", "Julio", "Agosto", "Septiembre", "Octubre", "Noviembre", "Diciembre"];
14
15    public $list_dates = [];
16    public $list_dates_number = [];
17
18    public $sales_data = [];
19
20    public $subsidiaries_list = [];
21
22    public function render()
23    {
24        $this->subsidiaries_list = Subsidiary::all()->toArray();
25
26        if($this->init_date == null)
27        {
28            $this->init_date = Carbon::now()->format('Y-m');
29        }
30
31        if($this->end_date == null)
32        {
33            $this->end_date = Carbon::now()->addMonth(1)->format('Y-m');
34        }
35
36        if(empty($this->seller_list) && empty($this->subsidiary_filter))
37        {
38            $this->seller_list = collect(User::all()->toArray());
39
40            $this->seller_list = $this->seller_list->map(function($seller) {
41                return array_merge($seller, [
42                    'check' => true
43                ]);
44            });
45        }
46
47        $this->getMonthBetween();
48
49        $sales_data = Sale::query();
50
51        $sales_data = $sales_data->select(
52            DB::raw("SUM(total) as total"),
53            DB::raw("users.name as username"),
54            DB::raw("subsidiaries.name as subsidiaryname"),
55            DB::raw("EXTRACT(YEAR FROM 'created_date') as year"),
56            DB::raw("EXTRACT(MONTH FROM 'created_date') as month"),
57            DB::raw("COUNT(total) as quantity")
58        );
59

```

Figura 103. Controlador del requerimiento 17

Fuente: Elaboración propia

RF-18: El sistema debe tener un módulo de cancelar venta en donde se gestionará las cancelaciones de ventas

En la figura 104 se muestra el modelo del RF-18:

```
app > Models > Sale.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7  use Illuminate\Database\Eloquent\SoftDeletes;
8
9  class Sale extends Model
10 {
11     use HasFactory;
12     use SoftDeletes;
13
14     protected $fillable = ['name', 'document_id', 'address', 'phone', 'user_id', 'subsidiary_id', 'total', 'created_date'];
15
16     public function user()
17     {
18         return $this->belongsTo(User::class);
19     }
20
21     public function subsidiary()
22     {
23         return $this->belongsTo(Subsidiary::class);
24     }
25
26     public function getTotalAttribute($value)
27     {
28         return round($value, 2);
29     }
30 }
31
```

Figura 104. Modelo del requerimiento 18

Fuente: Elaboración propia

En la figura 105 se muestra la vista del RF-18:

```

1 @extends('layouts.system-layout')
2 @section('styles')
3 <style>
4     *
5     {
6         box-sizing: border-box;
7     }
8     body {
9         font: 16px Arial;
10    }
11
12    /*The container must be positioned relative*/
13    .autocomplete {
14        position: relative;
15        display: inline-block;
16    }
17
18    input {
19        border: 1px solid transparent;
20        background-color: #ffffff;
21        padding: 10px;
22        font-size: 16px;
23    }
24
25    input[type="text"] {
26        background-color: #ffffff;
27        width: 100%;
28    }
29
30    input[type="submit"] {
31        background-color: #d3d3d3;
32        color: #000;
33        cursor: pointer;
34    }
35
36    .autocomplete-items {
37        position: absolute;
38        border: 1px solid #000;
39        border-bottom: none;
40        border-top: none;
41        z-index: 99;
42        /*position the autocomplete items to be the same width as the container*/
43        top: 100%;
44        left: 0;
45        right: 0;
46    }
47
48    .autocomplete-items div {
49        padding: 10px;
50        cursor: pointer;
51        background-color: #fff;
52        border-bottom: 1px solid #000;
53    }
54
55    /*when hovering an item*/
56    .autocomplete-items div:hover {
57        background-color: #d3d3d3;
58    }
59
60    /*when navigating through the items using the arrow keys*/
61    .autocomplete-active {
62        background-color: #d3d3d3;
63        color: #000;
64    }
65 </style>

```

Figura 105. Vista del requerimiento 18

Fuente: Elaboración propia

En la figura 106 se muestra el controlador del RF-18, con el nombre

```

1 class ControllerComponent extends Component
2 {
3     public $init_data;
4     public $user_id;
5     public $init_id;
6     public $basic_details;
7
8     public function render()
9     {
10         //if not null
11         if($this->init_data == null)
12         {
13             $this->init_data = Carbon::now()->format('Y-m-d');
14         }
15         //if not null
16         if($this->send_data == null)
17         {
18             $this->send_data = Carbon::now()->addDays(1)->format('Y-m-d');
19         }
20
21         return view('layouts.system-layout', [
22             'init_data' => $this->init_data,
23             'user_id' => $this->user_id,
24             'init_id' => $this->init_id,
25             'basic_details' => $this->basic_details,
26             'product_id' => $this->product_id,
27             'quantity' => $this->quantity,
28             'operation' => $this->operation,
29             'created_date' => $this->created_date,
30             'updated_date' => $this->updated_date,
31             'deleted_date' => $this->deleted_date,
32             'confirmation_modal' => $this->confirmation_modal,
33         ]);
34     }
35
36     public function delete()
37     {
38         $this->delete_id = $this->init_id;
39         $this->delete_id = $this->init_id;
40         $this->delete_id = $this->init_id;
41         $this->delete_id = $this->init_id;
42         $this->delete_id = $this->init_id;
43         $this->delete_id = $this->init_id;
44         $this->delete_id = $this->init_id;
45         $this->delete_id = $this->init_id;
46         $this->delete_id = $this->init_id;
47         $this->delete_id = $this->init_id;
48         $this->delete_id = $this->init_id;
49         $this->delete_id = $this->init_id;
50         $this->delete_id = $this->init_id;
51         $this->delete_id = $this->init_id;
52         $this->delete_id = $this->init_id;
53         $this->delete_id = $this->init_id;
54         $this->delete_id = $this->init_id;
55         $this->delete_id = $this->init_id;
56         $this->delete_id = $this->init_id;
57         $this->delete_id = $this->init_id;
58         $this->delete_id = $this->init_id;
59         $this->delete_id = $this->init_id;
60         $this->delete_id = $this->init_id;
61         $this->delete_id = $this->init_id;
62         $this->delete_id = $this->init_id;
63         $this->delete_id = $this->init_id;
64         $this->delete_id = $this->init_id;
65         $this->delete_id = $this->init_id;
66         $this->delete_id = $this->init_id;
67         $this->delete_id = $this->init_id;
68         $this->delete_id = $this->init_id;
69         $this->delete_id = $this->init_id;
70         $this->delete_id = $this->init_id;
71         $this->delete_id = $this->init_id;
72         $this->delete_id = $this->init_id;
73         $this->delete_id = $this->init_id;
74         $this->delete_id = $this->init_id;
75         $this->delete_id = $this->init_id;
76         $this->delete_id = $this->init_id;
77         $this->delete_id = $this->init_id;
78         $this->delete_id = $this->init_id;
79         $this->delete_id = $this->init_id;
80         $this->delete_id = $this->init_id;
81         $this->delete_id = $this->init_id;
82         $this->delete_id = $this->init_id;
83         $this->delete_id = $this->init_id;
84         $this->delete_id = $this->init_id;
85         $this->delete_id = $this->init_id;
86         $this->delete_id = $this->init_id;
87         $this->delete_id = $this->init_id;
88         $this->delete_id = $this->init_id;
89         $this->delete_id = $this->init_id;
90         $this->delete_id = $this->init_id;
91         $this->delete_id = $this->init_id;
92         $this->delete_id = $this->init_id;
93         $this->delete_id = $this->init_id;
94         $this->delete_id = $this->init_id;
95         $this->delete_id = $this->init_id;
96         $this->delete_id = $this->init_id;
97         $this->delete_id = $this->init_id;
98         $this->delete_id = $this->init_id;
99         $this->delete_id = $this->init_id;
100        $this->delete_id = $this->init_id;
101    }
102 }

```

Figura 106. Controlador del requerimiento 18

Fuente: Elaboración propia

RF-19: El sistema debe tener un módulo de abastecimiento en donde se podrá gestionar un nuevo abastecimiento

En la figura 107 se muestra el modelo del RF-19:

```
app > Models > Supplying.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7  use Illuminate\Database\Eloquent\SoftDeletes;
8
9  class Supplying extends Model
10 {
11     use HasFactory;
12     use SoftDeletes;
13
14     protected $fillable = ['user_id','supplier_id','created_date','subsidiary_id','total'];
15
16     public function user()
17     {
18         return $this->belongsTo(User::class);
19     }
20
21     public function supplier()
22     {
23         return $this->belongsTo(Supplier::class);
24     }
25
26     public function subsidiary()
27     {
28         return $this->belongsTo(Subsidiary::class);
29     }
30
31     public function getTotalAttribute($value)
32     {
33         return round($value, 2);
34     }
35 }
36
```

Figura 107. Modelo del requerimiento 19

Fuente: Elaboración propia

En la figura 108 se muestra la vista del RF-19:

```
resources > views > supplying/show.php
1  @extends('layouts.system-layout')
2  @section('styles')
3  <style>
4      *{
5          box-sizing: border-box;
6      }
7
8      body {
9          font: 16px Arial;
10     }
11
12     /*the container must be positioned relative*/
13     .autocomplete {
14         position: relative;
15         display: inline-block;
16     }
17
18     input {
19         border: 1px solid transparent;
20         background-color: #fff;
21         padding: 5px;
22         font-size: 16px;
23     }
24
25     input[type="text"] {
26         background-color: #fff;
27         width: 100%;
28     }
29
30     input[type="submit"] {
31         background-color: #d9d9d9;
32         color: #fff;
33         cursor: pointer;
34     }
35
36     .autocomplete-items {
37         position: absolute;
38         border: 1px solid #d9d9d9;
39         border-bottom: none;
40         border-top: none;
41         z-index: 99;
42         /*position the autocomplete items to be the same width as the container*/
43         top: 100%;
44         left: 0;
45         right: 0;
46     }
47
48     .autocomplete-items div {
49         padding: 5px;
50         cursor: pointer;
51         background-color: #fff;
52         border-bottom: 1px solid #d9d9d9;
53     }
54
55     /*when hovering an item*/
56     .autocomplete-items div:hover {
57         background-color: #e0e0e0;
58     }
59
60     /*when navigating through the items using the arrow keys*/
61     .autocomplete-active {
62         background-color: #d9d9d9;
63         color: #000000;
64     }
65 </style>
```

Figura 108. Vista del requerimiento 19

Fuente: Elaboración propia

En la figura 109 se muestra el controlador del RF-19, con el nombre

```
app > Http > Livewire > SupplyingComponent.php
19 {
34 {
54 }
55 }
56
57 public function add()
58 {
59     $checker = true;
60     $id = $this->selectedProductId;
61
62     foreach ($this->array_supplying_details as $supplying_detail)
63     {
64         if($supplying_detail["product_stock_id"] == $id)
65         {
66             $checker = false;
67         }
68     }
69
70     if($checker)
71     {
72         $product_stock = ProductStock::findOrFail($id);
73
74         $supplying_detail = [];
75         $supplying_detail['product_stock_id'] = $product_stock->id;
76         $supplying_detail['product_id'] = $product_stock->product_id;
77         $supplying_detail['name'] = $product_stock->product->name;
78         $supplying_detail['brand'] = $product_stock->product->brand;
79         $supplying_detail['purchase_price'] = 0;
80         $supplying_detail['quantity'] = 1;
81         $supplying_detail['type_currency'] = "SOL";
82         $supplying_detail['stock'] = $product_stock->stock;
83         $supplying_detail['total'] = round($supplying_detail['quantity'] * $supplying_detail['purchase_price'],2);
84         $supplying_detail['id'] = $this->counter;
85
86         $this->array_supplying_details[] = $supplying_detail;
87
88         $this->counter++;
89     }
90 }
91 }
```

Figura 109. Controlador del requerimiento 19

Fuente: Elaboración propia

RF-25: El sistema debe permitir generar un documento EXCEL con el detalle de la rotación de inventarios.

En la figura 110 se muestra el modelo del RF-25:

```

app > Models > Product.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7  use Illuminate\Database\Eloquent\SoftDeletes;
8
9  class Product extends Model
10 {
11     use HasFactory;
12     use SoftDeletes;
13
14     protected $fillable = ['name',
15         'code',
16         'description',
17         'brand',
18         'price',
19         'type_currency',
20         'img_path',
21         'category_id'];
22
23     public function category()
24     {
25         return $this->belongsTo(Category::class);
26     }
27
28     public function getPriceAttribute($value)
29     {
30         return round($value, 2);
31     }
32
33 }
34

```

Figura 110. Modelo del requerimiento 25

Fuente: Elaboración propia

En la figura 111 se muestra la vista del RF-25:

```

resources > views > rotation_reportblade.php
1  @extends('layouts.system-layout')
2  @section('styles')
3
4  <style>
5      .multiselect {
6          width: 200px;
7      }
8
9      .selectBox {
10         position: relative;
11     }
12
13     .selectBox select {
14         width: 100%;
15         font-weight: bold;
16     }
17
18     .overSelect {
19         position: absolute;
20         left: 0;
21         right: 0;
22         top: 0;
23         bottom: 0;
24     }
25
26     #checkboxes {
27         display: none;
28         border: 1px solid #dadada;
29     }
30
31     #checkboxes label {
32         display: block;
33     }
34
35     #checkboxes label:hover {
36         background-color: #1e90ff;
37     }
38 </style>
39

```

Figura 111. Vista del requerimiento 25

Fuente: Elaboración propia

En la figura 112 se muestra el controlador del RF-25, con el nombre

```

14 // app > Controllers > M SubsidaryController.php
15 // app > Controllers > M SubsidaryController.php
16 // app > Controllers > M SubsidaryController.php
17 // app > Controllers > M SubsidaryController.php
18 // app > Controllers > M SubsidaryController.php
19 // app > Controllers > M SubsidaryController.php
20 // app > Controllers > M SubsidaryController.php
21 // app > Controllers > M SubsidaryController.php
22 // app > Controllers > M SubsidaryController.php
23 // app > Controllers > M SubsidaryController.php
24 // app > Controllers > M SubsidaryController.php
25 // app > Controllers > M SubsidaryController.php
26 // app > Controllers > M SubsidaryController.php
27 // app > Controllers > M SubsidaryController.php
28 // app > Controllers > M SubsidaryController.php
29 // app > Controllers > M SubsidaryController.php
30 // app > Controllers > M SubsidaryController.php
31 // app > Controllers > M SubsidaryController.php
32 // app > Controllers > M SubsidaryController.php
33 // app > Controllers > M SubsidaryController.php
34 // app > Controllers > M SubsidaryController.php
35 // app > Controllers > M SubsidaryController.php
36 // app > Controllers > M SubsidaryController.php
37 // app > Controllers > M SubsidaryController.php
38 // app > Controllers > M SubsidaryController.php
39 // app > Controllers > M SubsidaryController.php
40 // app > Controllers > M SubsidaryController.php
41 // app > Controllers > M SubsidaryController.php
42 // app > Controllers > M SubsidaryController.php
43 // app > Controllers > M SubsidaryController.php
44 // app > Controllers > M SubsidaryController.php
45 // app > Controllers > M SubsidaryController.php
46 // app > Controllers > M SubsidaryController.php
47 // app > Controllers > M SubsidaryController.php
48 // app > Controllers > M SubsidaryController.php
49 // app > Controllers > M SubsidaryController.php
50 // app > Controllers > M SubsidaryController.php
51 // app > Controllers > M SubsidaryController.php
52 // app > Controllers > M SubsidaryController.php
53 // app > Controllers > M SubsidaryController.php
54 // app > Controllers > M SubsidaryController.php
55 // app > Controllers > M SubsidaryController.php
56 // app > Controllers > M SubsidaryController.php
57 // app > Controllers > M SubsidaryController.php
58 // app > Controllers > M SubsidaryController.php
59 // app > Controllers > M SubsidaryController.php
60 // app > Controllers > M SubsidaryController.php
61 // app > Controllers > M SubsidaryController.php
62 // app > Controllers > M SubsidaryController.php
63 // app > Controllers > M SubsidaryController.php
64 // app > Controllers > M SubsidaryController.php
65 // app > Controllers > M SubsidaryController.php
66 // app > Controllers > M SubsidaryController.php
67 // app > Controllers > M SubsidaryController.php
68 // app > Controllers > M SubsidaryController.php
69 // app > Controllers > M SubsidaryController.php
70 // app > Controllers > M SubsidaryController.php
71 // app > Controllers > M SubsidaryController.php
72 // app > Controllers > M SubsidaryController.php
73 // app > Controllers > M SubsidaryController.php
74 // app > Controllers > M SubsidaryController.php
75 // app > Controllers > M SubsidaryController.php
76 // app > Controllers > M SubsidaryController.php
77 // app > Controllers > M SubsidaryController.php
78 // app > Controllers > M SubsidaryController.php
79 // app > Controllers > M SubsidaryController.php
80 // app > Controllers > M SubsidaryController.php
81 // app > Controllers > M SubsidaryController.php
82 // app > Controllers > M SubsidaryController.php
83 // app > Controllers > M SubsidaryController.php
84 // app > Controllers > M SubsidaryController.php
85 // app > Controllers > M SubsidaryController.php
86 // app > Controllers > M SubsidaryController.php
87 // app > Controllers > M SubsidaryController.php
88 // app > Controllers > M SubsidaryController.php
89 // app > Controllers > M SubsidaryController.php
90 // app > Controllers > M SubsidaryController.php
91 // app > Controllers > M SubsidaryController.php
92 // app > Controllers > M SubsidaryController.php
93 // app > Controllers > M SubsidaryController.php
94 // app > Controllers > M SubsidaryController.php
95 // app > Controllers > M SubsidaryController.php
96 // app > Controllers > M SubsidaryController.php
97 // app > Controllers > M SubsidaryController.php
98 // app > Controllers > M SubsidaryController.php
99 // app > Controllers > M SubsidaryController.php
100 // app > Controllers > M SubsidaryController.php

```

Figura 112. Controlador del requerimiento 25

Fuente: Elaboración propia

RF-26: El sistema debe permitir generar un documento EXCEL con el detalle del porcentaje del inventario sin rotación.

En la figura 113 se muestra el modelo del RF-26:

```

1 app > Models > Product.php
2 <?php
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Factories\HasFactory;
6 use Illuminate\Database\Eloquent\Model;
7 use Illuminate\Database\Eloquent\SoftDeletes;
8
9 class Product extends Model
10 {
11     use HasFactory;
12     use SoftDeletes;
13
14     protected $fillable = ['name',
15         'code',
16         'description',
17         'brand',
18         'price',
19         'type_currency',
20         'img_path',
21         'category_id'];
22
23     public function category()
24     {
25         return $this->belongsTo(Category::class);
26     }
27
28     public function getPriceAttribute($value)
29     {
30         return round($value, 2);
31     }
32 }
33
34

```

Figura 113. Modelo del requerimiento 26

Fuente: Elaboración propia

En la figura 114 se muestra la vista del RF-26:

```

4 <style>
5     .multiselect {
6         width: 200px;
7     }
8
9     .selectBox {
10        position: relative;
11    }
12
13    .selectBox select {
14        width: 100%;
15        font-weight: bold;
16    }
17
18    .overSelect {
19        position: absolute;
20        left: 0;
21        right: 0;
22        top: 0;
23        bottom: 0;
24    }
25
26    #checkboxes {
27        display: none;
28        border: 1px solid #dadada;
29    }
30
31    #checkboxes label {
32        display: block;
33    }
34
35    #checkboxes label:hover {
36        background-color: #e90ff;
37    }
38 </style>

```

Figura 114. Vista del requerimiento 26

Fuente: Elaboración propia

En la figura 115 se muestra el controlador del RF-26, con el nombre

```

300 #def filter_m = %> dplyr::filter(m, !is.na(m$component))
301 class RotationReportStorageComponent extends Component
302 {
303   public Subsidary_filter;
304   public Subsidaries_list;
305   public Category_list;
306   public Year_selection;
307   public Month_selection = "December";
308   public filter_names = ["Year", "Category", "Month", "Year", "Month", "Year", "Month", "Category", "Category", "Month", "Month", "Month"];
309   public function render()
310   {
311     if(empty($this->year_selection))
312     {
313       $this->year_selection = 2024;
314     }
315
316     if(empty($this->subsidary_filter))
317     {
318       $this->subsidaries_list = Subsidary::all()->toArray();
319     }
320
321     if(empty($this->category_list))
322     {
323       $this->category_list = collect(Category::all()->toArray());
324
325       $this->category_list->map(function($category) {
326         return array_merge($category, [
327           'check' => true
328         ]);
329       });
330     }
331
332     $startDate = date("Y");
333     $endDate = range($startDate, $startDate+1);
334
335     $this->subsidary_filter = Auth::user()->subsidary_id;
336
337     return view('rotation-report-percentage-component', [
338       'category_list' => $this->category_list,
339       'subsidaries_list' => $this->subsidaries_list,
340       'year_start' => $startDate,
341       'month_list' => $this->filter_names
342     ]);
343   }
344 }
345
346 function obtainRotationData($user, $date) {
347   // obtener el primer día del mes
348   $inicio = date("Y-m-d", strtotime("first day of " . $date));
349
350   // obtener el último día del mes
351   $fin = date("Y-m-d", strtotime("last day of " . $date));
352
353   return $inicio . " - " . $fin;
354 }
355
356 public function gettotal()
357 {
358   $rotaciones = ["Y", "Category", "Product", "Value Inventory sin movimiento", "Value Inventory promedio", "N Rotacion Inventory"];
359
360   $year_filter = $this->year_selection . " - (" . array_search($this->month_selection, $this->filter_names) . ")";
361
362   $categories = $this->category_list->where('check', true)->pluck('id')->implode(',');
363
364   $array_value = $this->getdata($this->subsidary_filter, $year_filter, $categories);
365
366   $names = Auth::user()->name;
367
368   $fechaInicio = $this->obtainRotationData($user, array_search($this->month_selection, $this->filter_names) . " - " . $this->year_selection);
369
370   $email = new exception('El Rotacion Report, ' . $this->month_selection . ' - ' . $year_value . $names . $fechaInicio);
371   return $names . $year_value . $month_value . $names . $fechaInicio . $names . $year_value . $names . $fechaInicio;
372 }
373
374 public function getdata($subsidary_id, $date, $categories)

```

Figura 115. Controlador del requerimiento 26

Fuente: Elaboración propia

RF-27: El sistema debe permitir generar un documento EXCEL con el detalle de la duración del inventario en ese mes.

En la figura 116 se muestra el modelo del RF-27:

```
app > Models > Sale.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7  use Illuminate\Database\Eloquent\SoftDeletes;
8
9  class Sale extends Model
10 {
11     use HasFactory;
12     use SoftDeletes;
13
14     protected $fillable = ['name', 'document_id', 'address', 'phone', 'user_id', 'subsidiary_id', 'total', 'created_date'];
15
16     public function user()
17     {
18         return $this->belongsTo(User::class);
19     }
20
21     public function subsidiary()
22     {
23         return $this->belongsTo(Subsidiary::class);
24     }
25
26     public function getTotalAttribute($value)
27     {
28         return round($value, 2);
29     }
30 }
31
```

Figura 116. Modelo del requerimiento 27

Fuente: Elaboración propia

En la figura 117 se muestra la vista del RF-27:

```
4  <style>
5      .multiselect {
6          width: 200px;
7      }
8
9      .selectBox {
10         position: relative;
11     }
12
13     .selectBox select {
14         width: 100%;
15         font-weight: bold;
16     }
17
18     .overSelect {
19         position: absolute;
20         left: 0;
21         right: 0;
22         top: 0;
23         bottom: 0;
24     }
25
26     #checkboxes {
27         display: none;
28         border: 1px solid #dadada;
29     }
30
31     #checkboxes label {
32         display: block;
33     }
34
35     #checkboxes label:hover {
36         background-color: #e90ff;
37     }
38 </style>
```

Figura 117. Vista del requerimiento 27

Fuente: Elaboración propia

[illegible]

Fuente: Elaboración propia

En la figura 119 se muestra el modelo del RF-28:

```

app > Models > Sale.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7  use Illuminate\Database\Eloquent\SoftDeletes;
8
9  class Sale extends Model
10 {
11     use HasFactory;
12     use SoftDeletes;
13
14     protected $fillable = ['name', 'document_id', 'address', 'phone', 'user_id', 'subsidiary_id', 'total', 'created_date'];
15
16     public function user()
17     {
18         return $this->belongsTo(User::class);
19     }
20
21     public function subsidiary()
22     {
23         return $this->belongsTo(Subsidiary::class);
24     }
25
26     public function getTotalAttribute($value)
27     {
28         return round($value, 2);
29     }
30 }
31

```

Fuente: Elaboración propia

```

4 <style>
5     .multiselect {
6         width: 200px;
7     }
8
9     .selectBox {
10        position: relative;
11    }
12
13    .selectBox select {
14        width: 100%;
15        font-weight: bold;
16    }
17
18    .overSelect {
19        position: absolute;
20        left: 0;
21        right: 0;
22        top: 0;
23        bottom: 0;
24    }
25
26    #checkboxboxes {
27        display: none;
28        border: 1px solid #dadada;
29    }
30
31    #checkboxboxes label {
32        display: block;
33    }
34
35    #checkboxboxes label:hover {
36        background-color: #e9e9ff;
37    }
38 </style>

```

Fuente: Elaboración propia

[illegible]

Fuente: Elaboración propia

PRUEBAS DE CAJA NEGRA DEL SEGUNDO SPRINT

PRUEBA DE CAJA NEGRA						
Código	PCN07			Fecha	23/12/2024	
Tarea	Administrar ventas			Módulo	Ventas	
Descripción	Se debe validar la visualización y funcionalidad de la sección de ventas					
Caso de prueba						
Precondiciones						
Debe haber productos creados para la venta						
Pasos de la prueba						
Escenario 1: Generar una venta Dado que, me encuentro en el módulo de ventas, Cuando edito todos los campos, busco un producto y selecciono la cantidad que deseo vender y le doy click en “Crear” Entonces el sistema indica “Venta concretada” Escenario 2: Eliminar una venta Dado que, me encuentro en el módulo de Ventas realizadas, Cuando selecciono un producto y le doy click en “Borrar” Entonces el sistema indica “Venta anulada”.						
Datos de entrada			Respuesta esperada	Coincide		Respuesta del sistema
Campo	Valor	Escenario		Sí	No	
		Prueba	Generar venta	x		Se visualiza la venta en el sistema
		Prueba	Eliminar venta	x		Ya no se visualiza la venta en el sistema
Post condiciones						
No se aplica						
Defectos				Veredicto		
No existen				Pasó		
Observaciones				Probador		
No aplica				María Milagros Llauce Chapañan		

Tabla 11. Pruebas HU 07

Fuente: Elaboración propia

PRUEBA DE CAJA NEGRA						
Código		PCN08		Fecha	23/12/2024	
Tarea		Administrar abastecimiento		Módulo	Abastecimiento	
Descripción		Se debe verificar la funcionalidad de la sección de abastecimientos				
Caso de prueba						
Precondiciones						
Debe haber productos creados para la venta						
Pasos de la prueba						
Escenario 1: Abastecer stock de productos Dado que, me encuentro en el módulo de Abastecimientos Cuando selecciono un proveedor, categoría y busco un producto, por último, doy clic en “Crear” Entonces el sistema indica “Stock agregado con éxito”						
Datos de entrada			Respuesta esperada	Coincide		Respuesta del sistema
Campo	Valor	Escenario		Sí	No	
		Prueba	Abastecer productos	x		Se visualiza nuevo stock del producto agregado
Post condiciones						
No se aplica						
Defectos				Veredicto		
No existen				Pasó		
Observaciones				Probador		
No aplica				María Milagros Llauce Chapoñan		

Tabla 12. Pruebas HU 08

Fuente: Elaboración propia

PRUEBA DE CAJA NEGRA						
Código		PCN10		Fecha		23/12/2024
Tarea		Administrar reportes obligatorios		Módulo		Reportes
Descripción		Se debe comprobar que se generen los reportes obligatorios				
Caso de prueba						
Precondiciones						
No aplica						
Pasos de la prueba						
Escenario 1: Generar un Reporte de rotación de inventario mensual. Dado que, me encuentro en el módulo de rotación de inventario mensual, Cuando selecciono categoría, sucursal, año y mes, doy click en “Generar Reporte”, Entonces el sistema generará un reporte de rotación de inventario mensual.						
Escenario 2: Generar un reporte de rotación de inventario Mensual en porcentaje Dado que, me encuentro en el módulo rotación de inventario Mensual en porcentaje, Cuando selecciono la categoría, sucursal, año y mes doy clic en “Generar Excel” Entonces el sistema generará un reporte de rotación de inventario Mensual en porcentaje.						
Escenario 3: Generar un reporte de duración del inventario Mensual Dado que, me encuentro en el módulo de Reporte de duración del inventario Mensual Cuando selecciono la categoría, sucursal, año y mes, doy click en “Generar Excel” Entonces el sistema generará un reporte de duración del inventario Mensual.						
Escenario 4: Generar un reporte de Porcentaje de valor económico de inventario. Dado que, me encuentro en el módulo de Reporte de valor económico del inventario Mensual, Cuando selecciono la categoría, sucursal, año y mes, doy click en “Generar Excel” Entonces el sistema generará un reporte de Porcentaje de valor económico de inventario.						
Datos de entrada			Respuesta esperada	Coincide		Respuesta del sistema
Campo	Valor	Escenario		Sí	No	
		Prueba	Generar reporte de rotación de inventario mensual	x		Se visualiza el reporte generado
		Prueba	Generar reporte de porcentaje de inventario sin rotación mensual	x		Se visualiza el reporte generado
		Prueba	Generar reporte de la duración del inventario mensual	x		Se visualiza el reporte generado
		Prueba	Generar reporte del valor económico del inventario mensual	x		Se visualiza el reporte generado
Post condiciones						
No se aplica						
Defectos				Veredicto		
No existen				Pasó		
Observaciones				Probador		
No aplica				María Milagros Llauce Chapoñan		

Figura 122. Pruebas HU 10

Fuente: Elaboración propia

Tabla 13. Retrospectiva del sprint 2

Problema	Causas	Acciones
Se presentaron algunos problemas por tiempo	Los feriados no fueron contemplados	Para los próximos sprint se consideró también los feriados y días no laborables

Fuente: Elaboración propia

ACTAS DE PLANIFICACIÓN Y REVISIÓN DEL SPRINT 2

Acta de reunión de planificación del Sprint

Fecha: 07/10/2024

Scrum Máster: María Milagros Llaue Chapoñan

Product Owner: Olimpia Gómez Alarcón

SPRINT 2

En el presente documento se da la conformidad sobre los objetivos que se realizarán en el segundo sprint por parte del equipo de desarrollo y a su vez, informar las historias de usuario que contemplan el sprint.

En el sprint 2 se determinó lo siguiente:

HU	Nº requerimiento	Objetivos
HU7	RF - 16	El sistema debe tener un módulo de ventas en donde se podrá realizar una nueva venta
	RF - 17	El sistema debe tener un módulo de ventas en donde al realizar una venta genera un comprobante
	RF - 18	El sistema debe tener un módulo de cancelar venta en donde se gestionará las cancelaciones de ventas
HU8	RF - 19	El sistema debe tener un módulo de abastecimiento en donde se podrá gestionar un nuevo abastecimiento
HU10	RF - 25	El sistema debe permitir generar un documento EXCEL con el detalle de la rotación de inventarios.
	RF - 26	El sistema debe permitir generar un documento EXCEL con el detalle del porcentaje del inventario sin rotación.
	RF - 27	El sistema debe permitir generar un documento EXCEL con el detalle de la duración del inventario en ese mes.
	RF - 28	El sistema debe permitir generar un documento EXCEL con el detalle del valor económico del inventario.

Firma de conformidad



Olimpia Gomez Alarcon
Sub Gerente General Neo Copy Import

Figura 123. Acta de planificación sprint 2

Fuente: Elaboración propia

Acta de reunión de revisión del Sprint
Fecha: 17/10/2024
Scrum Máster: María Milagros Llauce Chapoñan
Product Owner: Olimpia Gómez Alarcón

SPRINT 2

En el presente documento se da la conformidad que la Sra. Olimpia Gomez Alarcon estuvo durante el desarrollo de cada objetivo pactado en la reunión del sprint 2.
En el sprint 2 se desarrolló lo siguiente:

HU	Nº requerimiento	Objetivos
HU7	RF - 16	El sistema debe tener un módulo de ventas en donde se podrá realizar una nueva venta
	RF - 17	El sistema debe tener un módulo de ventas en donde al realizar una venta genera un comprobante
	RF - 18	El sistema debe tener un módulo de cancelar venta en donde se gestionará las cancelaciones de ventas
HU8	RF - 19	El sistema debe tener un módulo de abastecimiento en donde se podrá gestionar un nuevo abastecimiento
HU10	RF - 25	El sistema debe permitir generar un documento EXCEL con el detalle de la rotación de inventarios.
	RF - 26	El sistema debe permitir generar un documento EXCEL con el detalle del porcentaje del inventario sin rotación.
	RF - 27	El sistema debe permitir generar un documento EXCEL con el detalle de la duración del inventario en ese mes.
	RF - 28	El sistema debe permitir generar un documento EXCEL con el detalle del valor económico del inventario.

Firma de conformidad



Olimpia Gomez Alarcon
SUB GERENTE GENERAL
RUC: 2060949314

Olimpia Gomez Alarcon
Sub Gerente General Neo Copy Import

Figura 124. Acta de revisión del sprint 2
Fuente: Elaboración propia

SPRINT 3

ANÁLISIS DEL TERCER SPRINT

En la **tabla 12** se muestran los 5 requerimientos que conforman el sprint 3.

Tabla 14. Sprint 3

HU	Nº	Requerimiento
HU3	RF - 05	El sistema debe tener un módulo de usuarios en donde se podrá gestionar la búsqueda de estos por carácter o roles.
	RF - 07	El sistema debe tener un módulo de usuarios en donde se podrá editar un usuario existente
	RF - 08	El sistema debe tener un módulo de usuarios en donde se podrá eliminar un usuario existente
HU9	RF - 23	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas realizadas
	RF - 24	El sistema debe permitir generar un documento EXCEL con el detalle de los abastecimientos realizados

Fuente: Elaboración propia

FUNCIONALIDADES DEL TERCER SPRINT

La **Tabla 13** ofrece una descripción completa de las funcionalidades que implementará el sistema. Además, ilustra cómo interactúan los usuarios con la arquitectura de software para satisfacer cada uno de los requerimientos definidos en el Sprint 3.

Tabla 15. Funcionalidades del sprint 3

R.F.	Funcionalidad
RF-05	El usuario al acceder al módulo de usuarios visualizara un listado de todos los usuarios activos que maneja el sistema. Hacer click en el combo box de roles y sucursal para poder filtrar la búsqueda.

	Adicionalmente se tiene un buscador donde se podrá realizar la búsqueda por una palabra clave.
RF-07	El usuario al acceder al módulo de usuarios visualizará un listado de todos los usuarios activos que maneja el sistema. Hacer click en el botón de editar dentro de la tabla captura el id del usuario, que se envía al controlador y envía el parámetro al modelo para que le devuelva los campos que fueron registrados de dicho usuario.
RF-08	El usuario al acceder al módulo de usuarios visualizará un listado de todos los usuarios activos que maneja el sistema. Hacer click en el botón de eliminar dentro de la tabla captura el id del usuario, se envía una solicitud al controlador con el identificador único del usuario a eliminar. El controlador, a su vez, invoca el método correspondiente en el modelo, el cual ejecuta una actualización en la base de datos para cambiar el estado del registro a inactivo.
RF-23	El usuario al acceder al módulo de reporte de ventas visualizará un listado de todas las ventas concretadas en el sistema. Hacer click en "Generar Excel" se generará un documento Excel con los datos actuales del aplicativo. Estos datos son llamados del modelo, la vista envía la solicitud al controlador, la cual este último recibe la solicitud.
RF-24	El usuario al acceder al módulo de reporte de abastecimiento visualizará un listado de los abastecimientos realizados en el sistema. Hacer click en "Generar Excel" se generará un documento Excel con los datos actuales del aplicativo. Estos datos son llamados del modelo, la vista envía la solicitud al controlador, la cual este último recibe la solicitud.

Fuente: Elaboración propia

DISEÑO DEL SPRINT 3

Prototipos del Sprint 3:

En la **Figura 127** se muestra el prototipo del requerimiento funcional 05 la cual fue aprobado por el Product Owner para la vista del módulo de usuarios en donde se podrá gestionar la búsqueda de estos por carácter o roles.

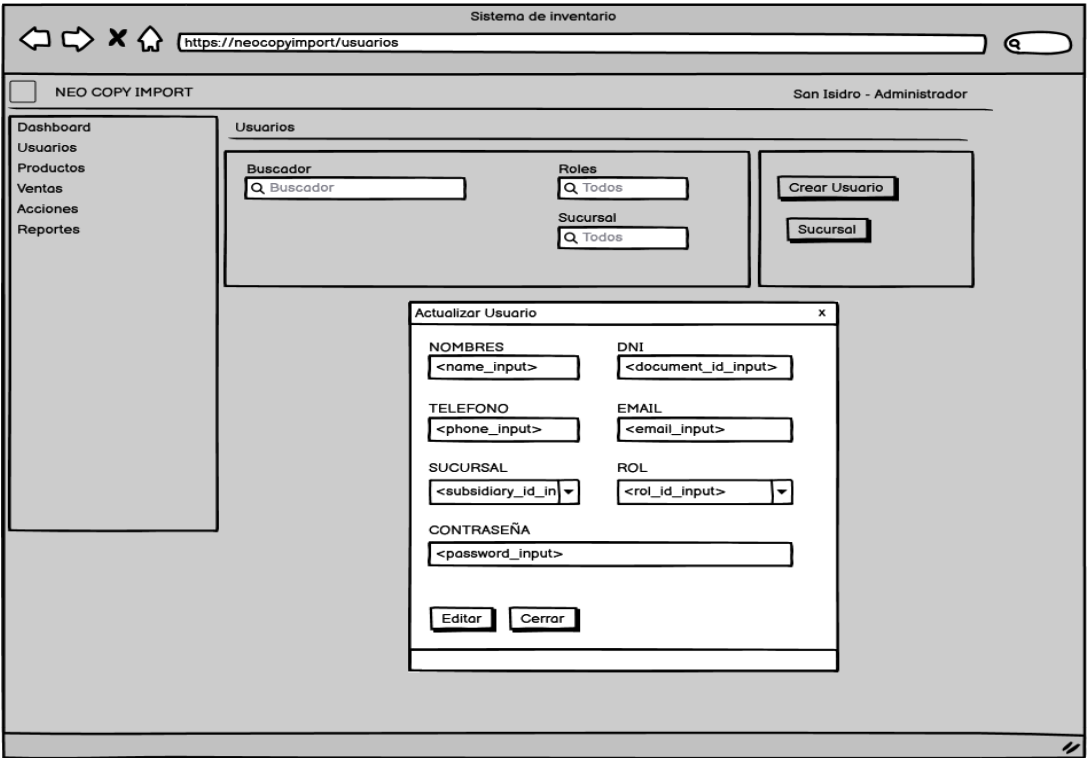


Figura 125. Prototipo para el requerimiento RF-05.

Fuente: Elaboración propia

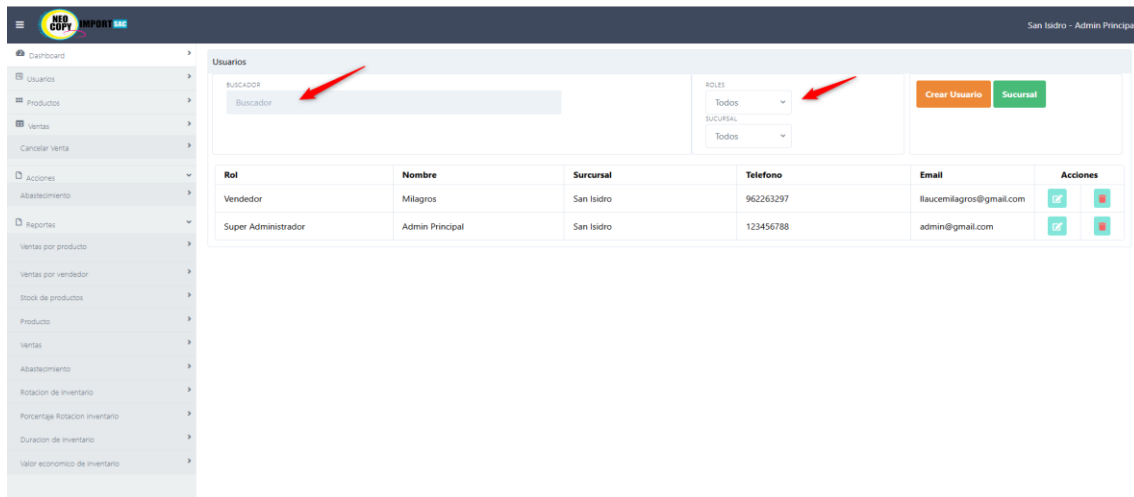


Figura 126. Búsqueda de usuario por carácter o rol

Elaboración propia

En la **Figura 129** se muestra el prototipo del requerimiento funcional 07 la cual fue aprobado por el Product Owner para la vista del módulo de usuarios en donde se podrá editar un usuario existente.

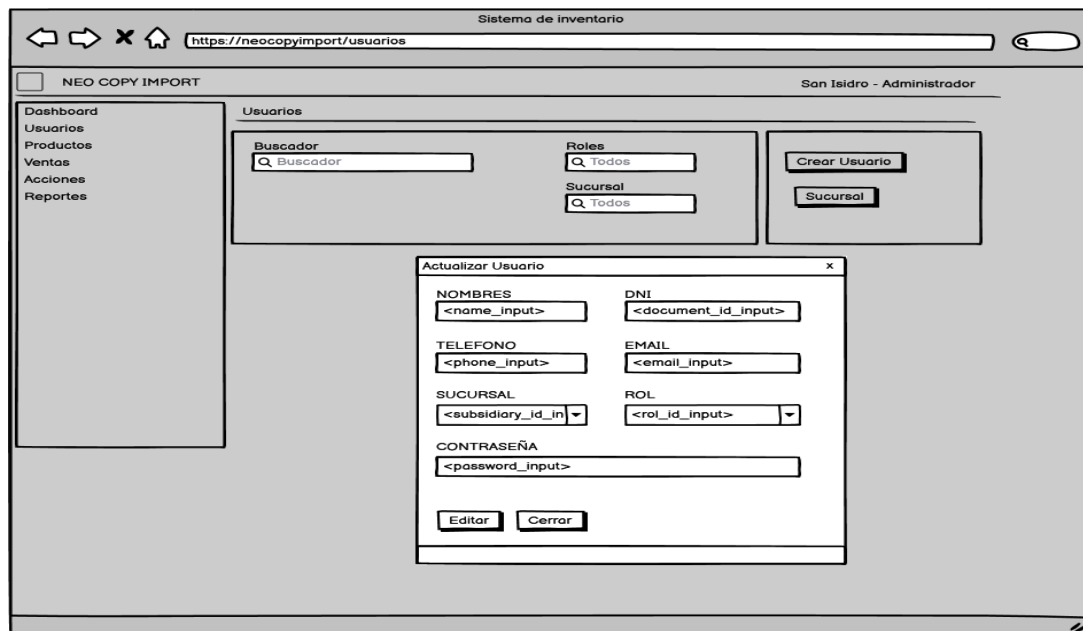


Figura 127. Prototipo para el requerimiento RF-07.

Fuente: Elaboración propia

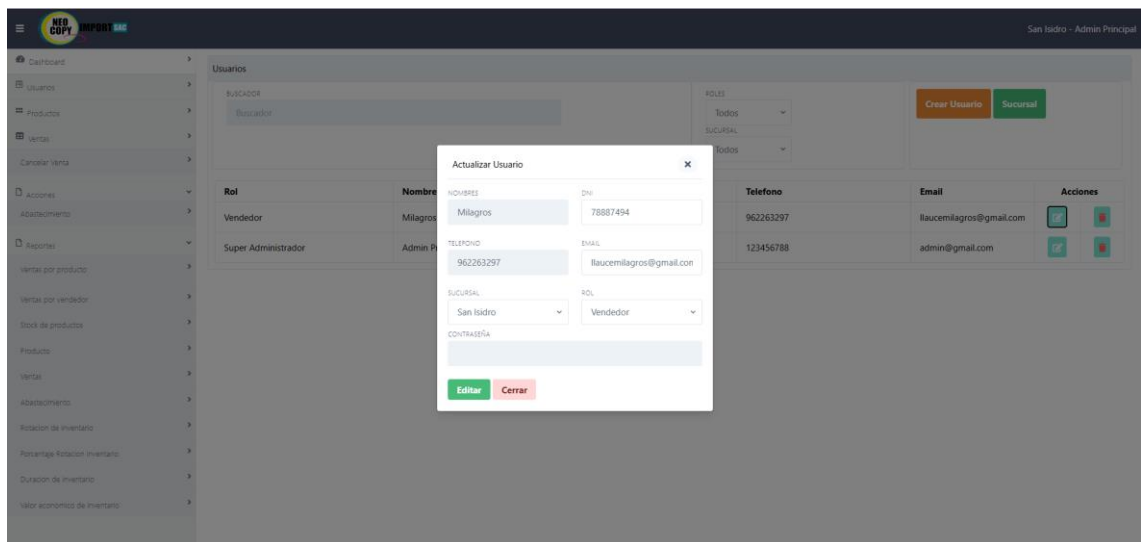


Figura 128. Editar usuario

Fuente: Elaboración propia

En la **Figura 131** se muestra el prototipo del requerimiento funcional 08 la cual fue aprobado por el Product Owner para la vista del módulo de usuarios en donde se podrá eliminar un usuario existente.

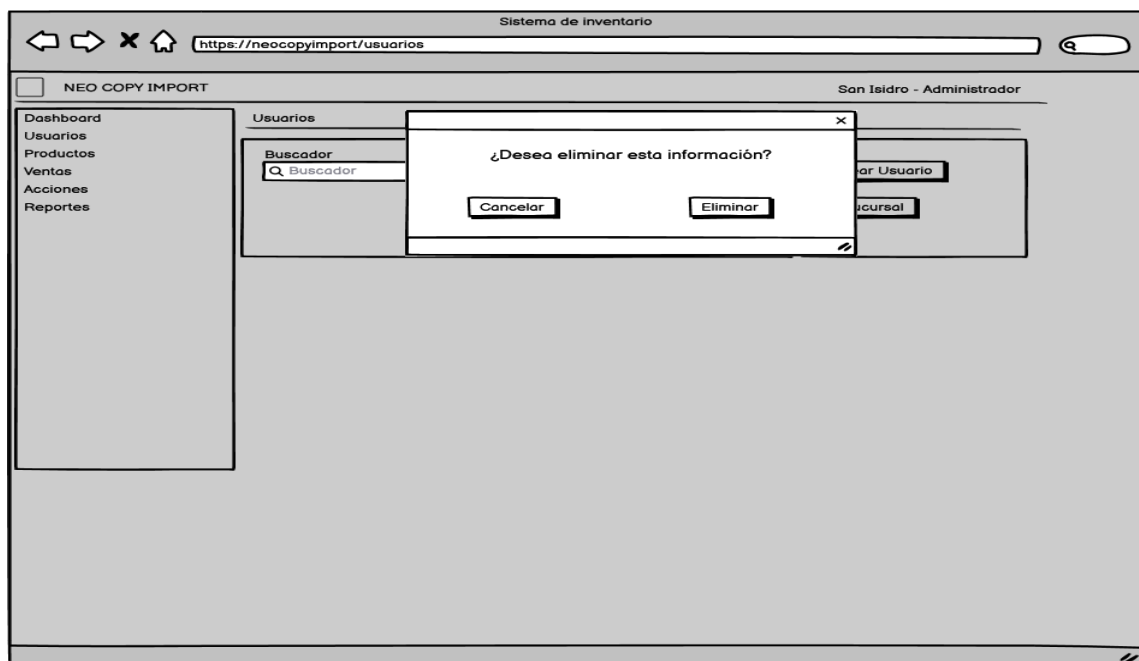


Figura 129. Prototipo para el requerimiento RF-08.

Fuente: Elaboración propia

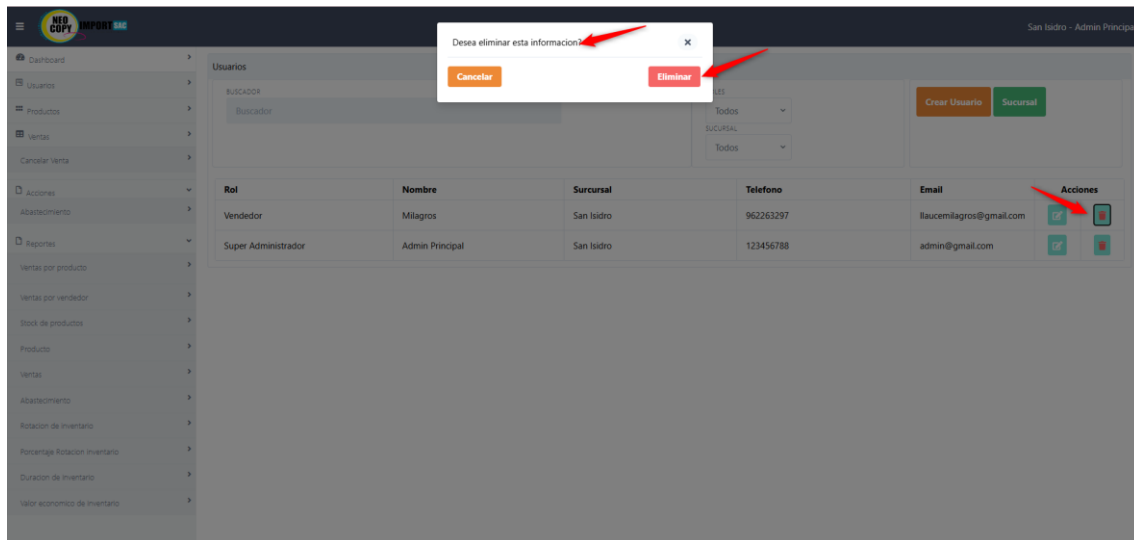


Figura 130. Eliminar usuario

Fuente: Elaboración propia

En la **Figura 133** se muestra el prototipo del requerimiento funcional 23 la cual fue aprobado por el Product Owner para la vista del módulo de reporte de ventas la cual se pueda generar un documento EXCEL con el detalle de las ventas realizadas.

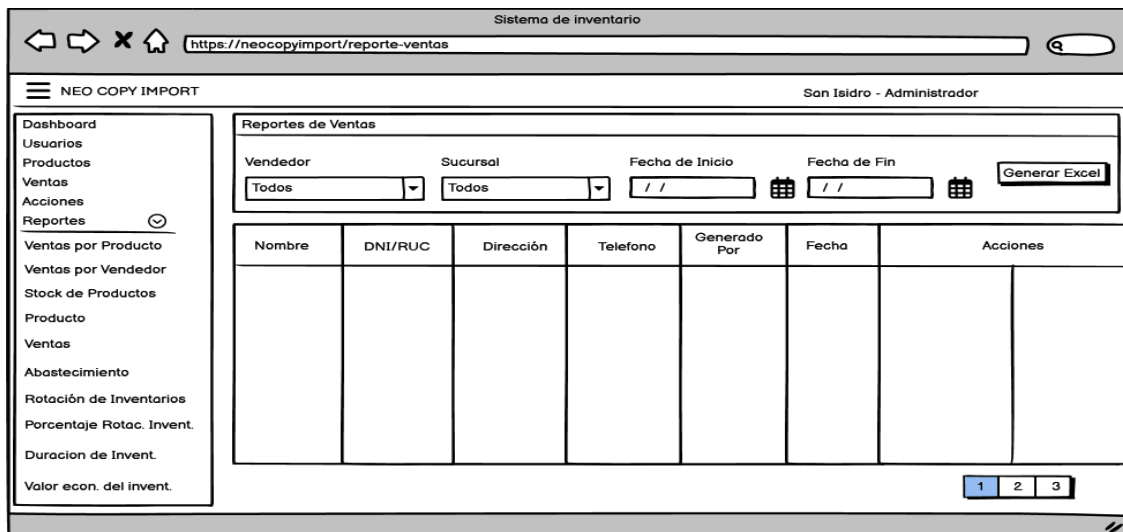


Figura 131. Prototipo para el requerimiento RF-23.

Fuente: Elaboración propia

Reportes de Ventas

VENDEDOR: Todos | Sucursal: Todos | FECHA INICIO: 09/01/2025 | FECHA FIN: 10/01/2025 | **Generar Excel**

Nombre	DNI/RUC	Direccion	Telefono	Generado por	Total	Fecha	Acciones	Acciones
--------	---------	-----------	----------	--------------	-------	-------	----------	----------

Figura 132. Reporte de ventas realizadas

Fuente: Elaboración propia

En la **Figura 135** se muestra el prototipo del requerimiento funcional 24 la cual fue aprobado por el Product Owner para la vista del módulo de abastecimiento para generar un documento EXCEL con el detalle de los abastecimientos realizados

Reportes de Abastecimiento

Proveedor: Todos | Sucursal: Todos | Fecha de Inicio: // | Fecha de Fin: // | **Generar Excel**

Proveedor	RUC/DNI	Telefono	Dirección	Total	Generado Por	Fecha	Acciones
-----------	---------	----------	-----------	-------	--------------	-------	----------

1 2 3

Figura 133. Prototipo para el requerimiento RF-24.

Fuente: Elaboración propia

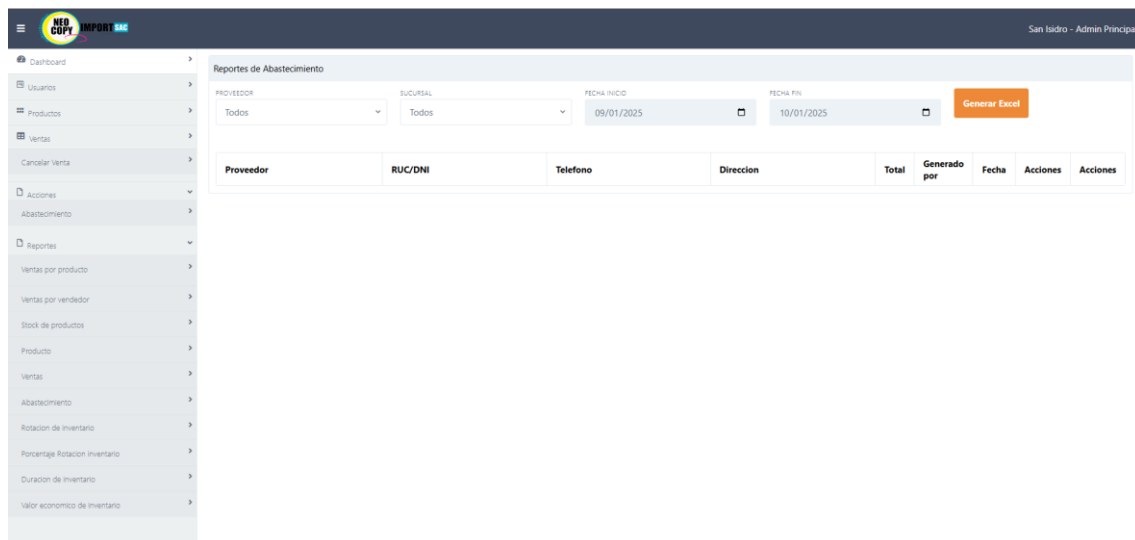


Figura 134. Reportes abastecimientos realizados

Fuente: Elaboración propia

ENTIDADES DEL TERCER SPRINT

En la Figura 139 se muestran las entidades del Sprint 3

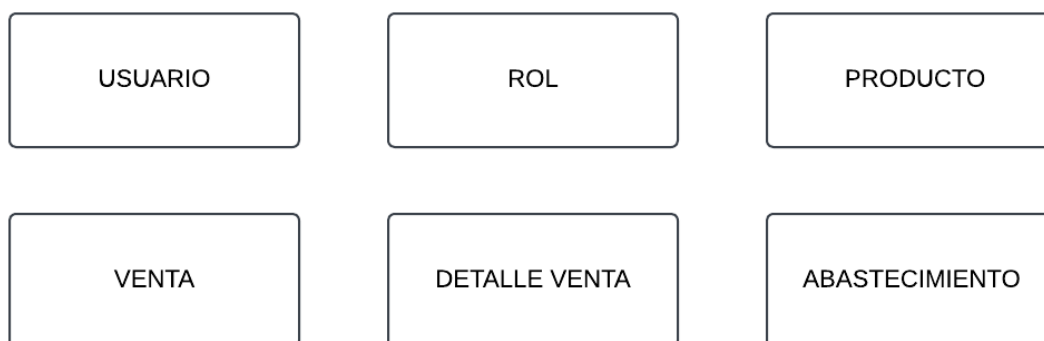


Figura 135. Entidades del Sprint 3

Fuente: Elaboración Propia

DIAGRAMA ENTIDAD/ RELACIÓN DEL TERCER SPRINT

En la Figura 138 se muestra el diagrama conceptual de la base de datos del Sprint 3

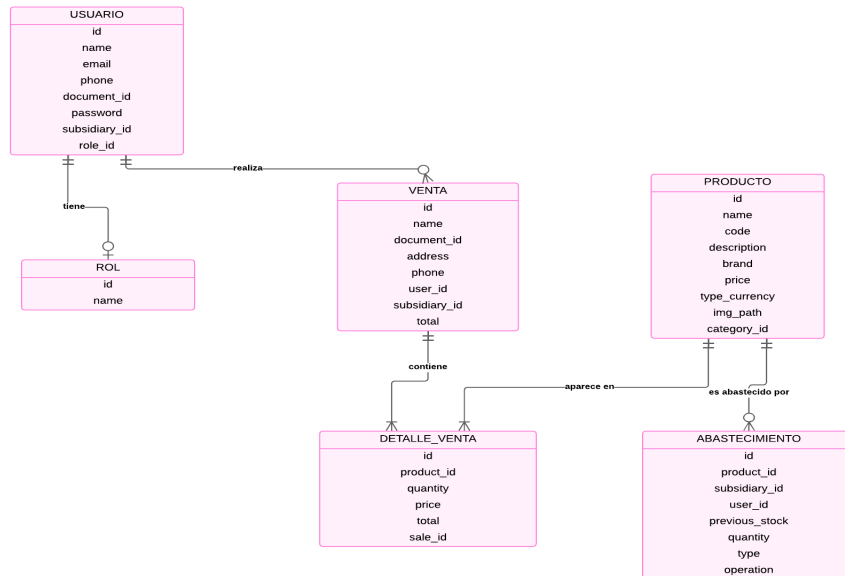


Figura 136. Diagrama Entidad / Relación del Sprint 3

Fuente: Elaboración Propia

DIAGRAMA LÓGICO DEL TERCER SPRINT

En la figura 139 se muestra el diagrama físico que representa la descripción de los datos con el mayor detalle, su estructura y las relaciones de los datos fundamentales del negocio.

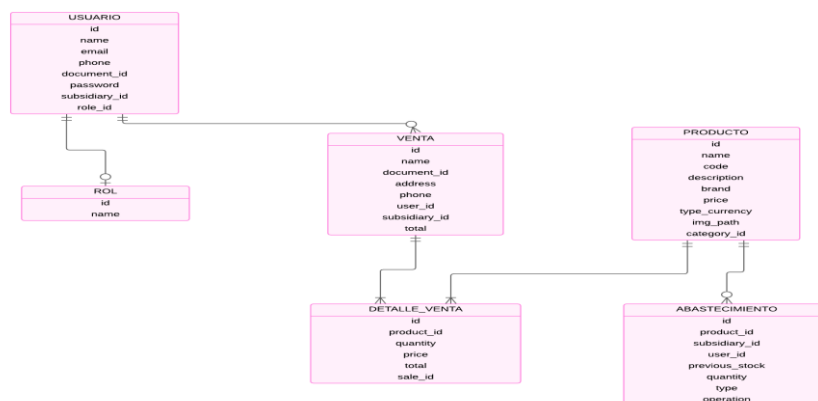


Figura 137. Diagrama Lógico del Sprint 3

Fuente: Elaboración Propia

Diagrama Físico:

En la figura 140 se muestra el diagrama físico que representa como se construirá el modelo en la base de datos.

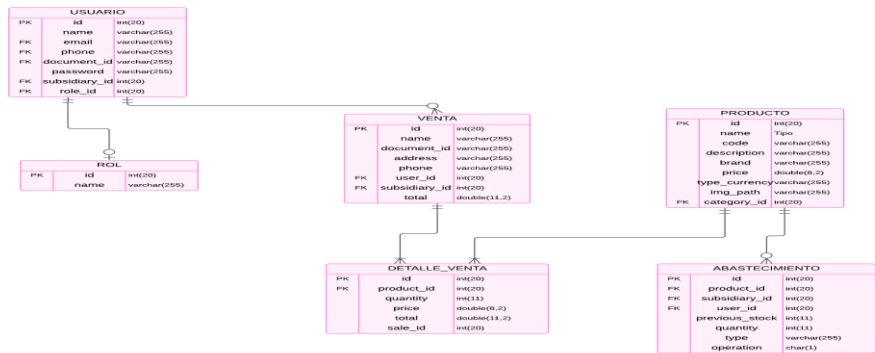


Figura 138. Diagrama Físico del Sprint 3

Fuente: Elaboración Propia

IMPLEMENTACIÓN DEL TERCER SPRINT

Diagrama de la Base de Datos:

En la Figura 141 tenemos el diseño de la base de datos en MySQL que vamos a usar para el Sprint 3.

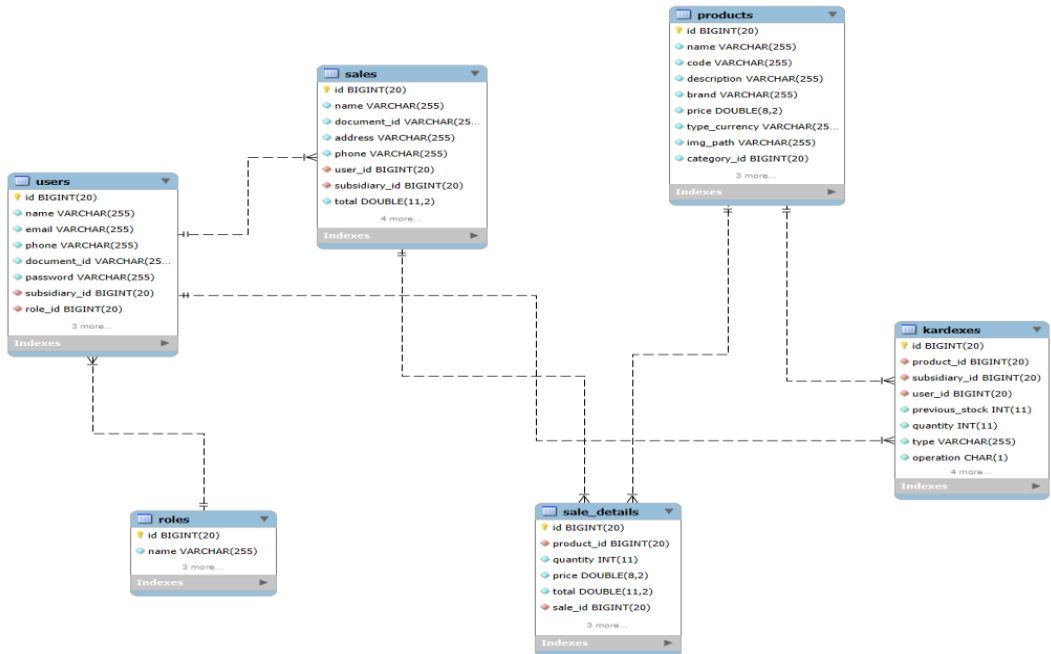


Figura 139. Diagrama de la base de datos del Sprint 3

Fuente: Elaboración Propia

RF-05: El sistema debe tener un módulo de usuarios en donde se podrá gestionar la búsqueda de estos por carácter o roles.

En la figura 142 se muestra el modelo del RF-05:

```
app > Models > User.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Foundation\Auth\User as Authenticatable;
7  use Illuminate\Notifications\Notifiable;
8  use Laravel\Sanctum\HasApiTokens;
9
10 class User extends Authenticatable
11 {
12     use Notifiable;
13     use HasFactory;
14     use HasApiTokens;
15
16     /**
17      * The attributes that are mass assignable.
18      *
19      * @var array<int, string>
20      */
21     protected $fillable = [
22         'name',
23         'phone',
24         'email',
25         'password',
26         'document_id',
27         'subsidiary_id',
28         'role_id'
29     ];
30
31     /**
32      * The attributes that should be hidden for serialization.
33      *
34      * @var array<int, string>
35      */
36     protected $hidden = [
37         'password'
38     ];
39
40     public function subsidiary()
41     {
42         return $this->belongsTo(Subsidiary::class);
43     }
44
45     public function role()
```

Figura 140. Modelo del requerimiento 05

Fuente: Elaboración propia

En la figura 145 se muestra la vista del RF-05:

```
resources > views > users.blade.php
1  @extends('layouts.system-layout')
2  @section('styles')
3
4  @endsection
5  @section('content')
6      @livewire('user-component')
7      @livewire('subsidiary-component')
8  @endsection
9
10 @section('script')
11
12
13 @endsection
14
```

Figura 141. Vista del requerimiento 05

Fuente: Elaboración propia

En la figura 144 se muestra el controlador del RF-05:

```
app > http > routes > @ UserComponent.php
13 class UserComponent extends Component
14 {
15     use WithPagination;
16
17     public $name_c, $phone_c, $email_c, $password_c, $document_id_c, $subsidiary_id_c, $role_id_c;
18     public $name_e, $phone_e, $email_e, $password_e, $document_id_e, $subsidiary_id_e, $role_id_e, $id_e;
19
20     public $delete_id;
21
22     public $test_filter;
23     public $subsidiary_filter;
24     public $role_filter;
25
26
27     public function render()
28     {
29         return view('livewire.user-component', [
30             'users' => User::when($this->test_filter, function($query){
31                 if(empty($this->test_filter))
32                 {
33                     $query->where('name', 'like', '%' . $this->test_filter . '%' );
34                 }
35             }->when($this->subsidiary_filter, function($query){
36                 if(empty($this->subsidiary_filter))
37                 {
38                     $query->where('subsidiary_id', $this->subsidiary_filter);
39                 }
40             }->when($this->role_filter, function($query){
41                 if(empty($this->role_filter))
42                 {
43                     $query->where('role_id', $this->role_filter);
44                 }
45             }->orderBy('id','desc')->paginate(10),
46             'subsidiary_list' => Subsidiary::all(),
47             'role_list' => Role::all(),
48         ]);
49     }
50
51     public function create()
52     {
53         $this->resetInputFields();
54     }
55
56     public function store()
57     {
58         $this->validate([
59             'name_c' => 'required|max:255',
60             'phone_c' => 'required|unique:users,phone|required:[0-9]{9}/',
61             'email_c' => 'required|unique:users,email|max:255',
62             'password_c' => 'required|max:255',
63             'document_id_c' => 'required|unique:users,document_id|max:255',
64             'subsidiary_id_c' => 'required',
65             'role_id_c' => 'required',
66         ]);
67
68         User::create([
69             'name' => $this->name_c,
70             'phone' => $this->phone_c,
71             'email' => $this->email_c,
72             'password' => Hash::make($this->password_c),
73             'document_id' => $this->document_id_c,
74             'subsidiary_id' => $this->subsidiary_id_c,
75             'role_id' => $this->role_id_c,
76         ]);
77     }
78 }
```

Figura 142. Controlador del requerimiento 05

Fuente: Elaboración propia

RF-07: El sistema debe tener un módulo de usuarios en donde se podrá editar un usuario existente

En la figura 145 se muestra el modelo del RF-07:

```

app > Models > User.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Foundation\Auth\User as Authenticatable;
7  use Illuminate\Notifications\Notifiable;
8  use Laravel\Sanctum\HasApiTokens;
9
10 class User extends Authenticatable
11 {
12     use Notifiable;
13     use HasFactory;
14     use HasApiTokens;
15
16     /**
17      * The attributes that are mass assignable.
18      *
19      * @var array<int, string>
20      */
21     protected $fillable = [
22         'name',
23         'phone',
24         'email',
25         'password',
26         'document_id',
27         'subsidiary_id',
28         'role_id'
29     ];
30
31     /**
32      * The attributes that should be hidden for serialization.
33      *
34      * @var array<int, string>
35      */
36     protected $hidden = [
37         'password'
38     ];
39
40     public function subsidiary()
41     {
42         return $this->belongsTo(Subsidiary::class);
43     }
44
45     public function role()

```

Figura 143. Modelo del requerimiento 07

Fuente: Elaboración propia

En la figura 146 se muestra la vista del RF-07:

```

resources > views > users.blade.php
1  @extends('layouts.system-layout')
2  @section('styles')
3
4  @endsection
5  @section('content')
6      @livewire('user-component')
7      @livewire('subsidiary-component')
8  @endsection
9
10 @section('script')
11
12
13 @endsection
14

```

Figura 144. Vista del requerimiento 07

Fuente: Elaboración propia

En la figura 147 se muestra el controlador del RF-07, con el nombre

```

14 {
15     public function edit($id)
16     {
17         $user = User::findOrFail($id);
18         $this->id_e = $id;
19         $this->name_e = $user->name;
20         $this->phone_e = $user->phone;
21         $this->email_e = $user->email;
22         $this->document_id_e = $user->document_id;
23         $this->subsidiary_id_e = $user->subsidiary_id;
24         $this->role_id_e = $user->role_id;
25
26         $this->dispatchBrowserEvent('openModal', ['modal' => 'update_user_modal', 'show' => 'confirmation_modal']);
27     }
28
29     public function update()
30     {
31         $this->validate([
32             'name_e' => 'required|max:255',
33             'phone_e' => [
34                 'required',
35                 'regex:/[0-9]{9}/',
36                 Rule::unique('users', 'phone')->ignore($this->id_e),
37             ],
38             'email_e' => [
39                 'required',
40                 'email',
41                 'max:255',
42                 Rule::unique('users', 'email')->ignore($this->id_e),
43             ],
44             'document_id_e' => [
45                 'required',
46                 'max:255',
47                 Rule::unique('users', 'document_id')->ignore($this->id_e),
48             ],
49             'subsidiary_id_e' => 'required',
50             'role_id_e' => 'required',
51         ]);
52
53         $user = User::findOrFail($this->id_e);
54
55         $user_update = [
56             'name' => $this->name_e,
57             'phone' => $this->phone_e,
58             'email' => $this->email_e,
59             'document_id' => $this->document_id_e,
60             'subsidiary_id' => $this->subsidiary_id_e,
61             'role_id' => $this->role_id_e,
62         ];
63
64         if(empty($this->password_e))
65         {
66             $user_update['password'] = Hash::make($this->password_e);
67         }
68
69         $user->update($user_update);
70
71         $this->resetInputFields();
72
73         session()->flash('message', 'Usuario Actualizado con éxito');
74
75         $this->dispatchBrowserEvent('closeModal', ['modal' => 'update_user_modal', 'show' => 'confirmation_modal']);
76     }
77 }

```

Figura 145. Controlador del requerimiento 07

Fuente: Elaboración propia

RF-08: El sistema debe tener un módulo de usuarios en donde se podrá eliminar un usuario existente

En la figura 148 se muestra el modelo del RF-08:

```

1 app > Models > User.php
2
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Factories\HasFactory;
6 use Illuminate\Foundation\Auth\User as Authenticatable;
7 use Illuminate\Notifications\Notifiable;
8 use Laravel\Sanctum\HasApiTokens;
9
10 class User extends Authenticatable
11 {
12     use Notifiable;
13     use HasFactory;
14     use HasApiTokens;
15
16     /**
17      * The attributes that are mass assignable.
18      *
19      * @var array<int, string>
20      */
21     protected $fillable = [
22         'name',
23         'phone',
24         'email',
25         'password',
26         'document_id',
27         'subsidiary_id',
28         'role_id',
29     ];
30
31     /**
32      * The attributes that should be hidden for serialization.
33      *
34      * @var array<int, string>
35      */
36     protected $hidden = [
37         'password',
38     ];
39
40     public function subsidiary()
41     {
42         return $this->belongsTo(Subsidiary::class);
43     }
44
45     public function role()
46     {
47
48     }
49 }

```

Figura 146. Modelo del requerimiento 08

Fuente: Elaboración propia

En la figura 149 se muestra la vista del RF-08:

```
resources > views > users.blade.php
1  @extends('layouts.system-layout')
2  @section('styles')
3
4  @endsection
5  @section('content')
6      @livewire('user-component')
7      @livewire('subsidiary-component')
8  @endsection
9
10 @section('script')
11
12
13 @endsection
14
```

Figura 147. Vista del requerimiento 08

Fuente: Elaboración propia

En la figura 150 se muestra el controlador del RF-08:

```
150 public function drop($id)
151 {
152     $this->delete_id = $id;
153
154     $this->dispatchBrowserEvent('openModal', ['modal' => 'delete_modal', 'show' => 'confirmation_modal']);
155 }
156
157 public function delete()
158 {
159     User::find($this->delete_id)->delete();
160     session()->flash('message', 'Usuario eliminado con éxito');
161
162     $this->dispatchBrowserEvent('closeModal', ['modal' => 'delete_modal', 'show' => 'confirmation_modal']);
163 }
164
165 public function resetInputFields()
166 {
167     $this->name_c = '';
168     $this->phone_c = '';
169     $this->email_c = '';
170     $this->password_c = '';
171     $this->document_id_c = '';
172     $this->subsidiary_id_c = '';
173     $this->role_id_c = '';
174
175     $this->name_e = '';
176     $this->phone_e = '';
177     $this->email_e = '';
178     $this->password_e = '';
179     $this->document_id_e = '';
180     $this->subsidiary_id_e = '';
181     $this->role_id_e = '';
182     $this->id_e = '';
183 }
184 }
```

Figura 148. Controlador del requerimiento 08

Fuente: Elaboración propia

RF-23: El sistema debe permitir generar un documento EXCEL con el detalle de las ventas realizadas

En la figura 151 se muestra el modelo del RF-23:

```
app > Models > Sale.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7  use Illuminate\Database\Eloquent\SoftDeletes;
8
9  class Sale extends Model
10 {
11     use HasFactory;
12     use SoftDeletes;
13
14     protected $fillable = ['name', 'document_id', 'address', 'phone', 'user_id', 'subsidiary_id', 'total', 'created_date'];
15
16     public function user()
17     {
18         return $this->belongsTo(User::class);
19     }
20
21     public function subsidiary()
22     {
23         return $this->belongsTo(Subsidiary::class);
24     }
25
26     public function getTotalAttribute($value)
27     {
28         return round($value, 2);
29     }
30 }
31
```

Figura 149. Modelo del requerimiento 23

Fuente: Elaboración propia

En la figura 152 se muestra la vista del RF-23:

```
resources > views > sale_made_report.blade.php
1  @extends('layouts.system-layout')
2  @section('styles')
3  <style>
4      .multiselect {
5          width: 200px;
6      }
7
8      .selectBox {
9          position: relative;
10     }
11
12     .selectBox select {
13         width: 100%;
14         font-weight: bold;
15     }
16
17     .overSelect {
18         position: absolute;
19         left: 0;
20         right: 0;
21         top: 0;
22         bottom: 0;
23     }
24
25     #checkboxboxes {
26         display: none;
27         border: 1px solid #dadada;
28     }
29
30     #checkboxboxes label {
31         display: block;
32     }
33
34     #checkboxboxes label:hover {
35         background-color: #1e90ff;
36     }
37 </style>
```

Figura 150. Vista del requerimiento 23

Fuente: Elaboración propia

En la figura 153 se muestra el controlador del RF-23:

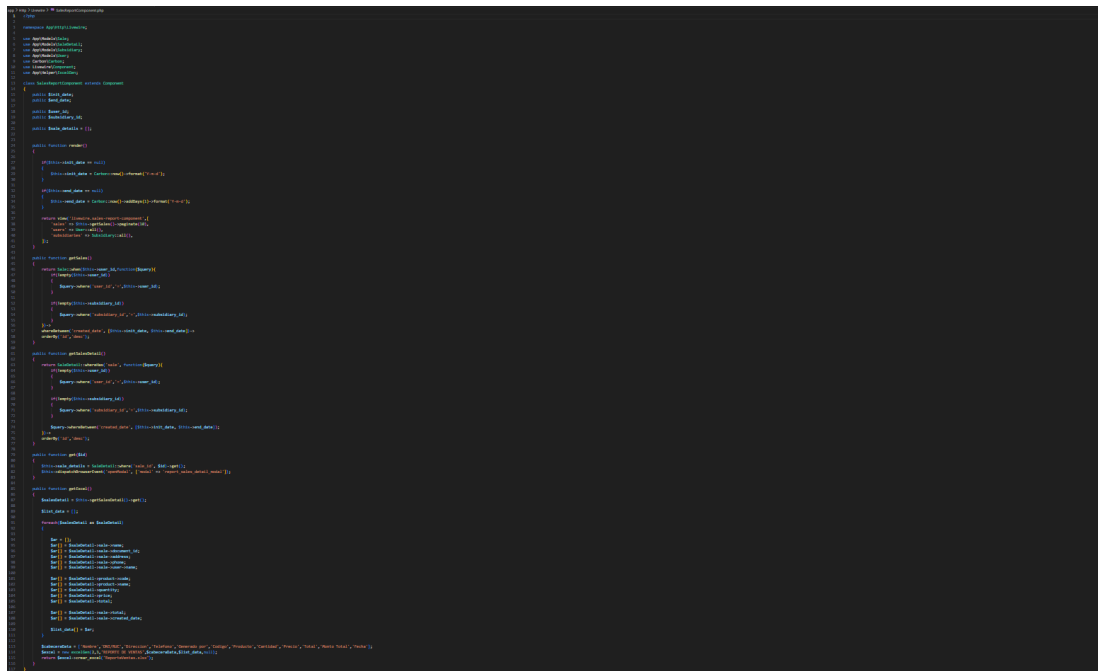


Figura 151. Controlador del requerimiento 23

Fuente: Elaboración propia

RF-24: El sistema debe permitir generar un documento EXCEL con el detalle de los abastecimientos realizados

En la figura 154 se muestra el modelo del RF-23:

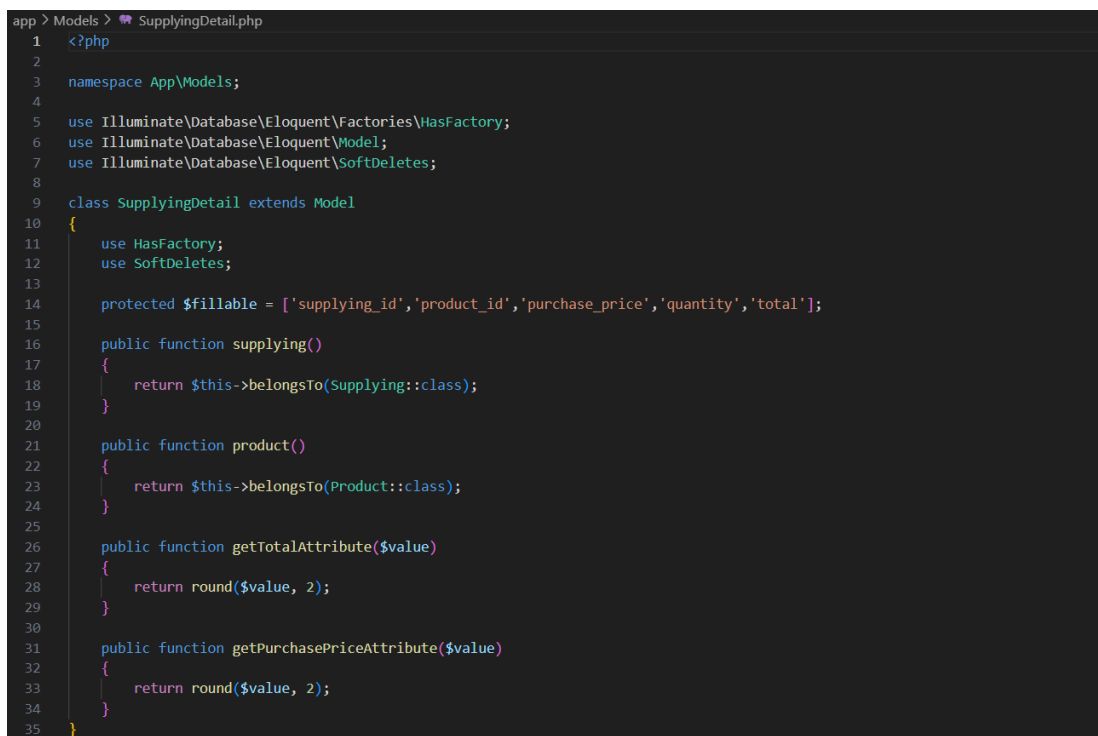


Figura 152. Modelo del requerimiento 24
Fuente: Elaboración propia

En la figura 155 se muestra la vista del RF-24:

```
resources > views > 🐘 supplyings_report.blade.php
1  @extends('layouts.system-layout')
2  @section('styles')
3
4  @endsection
5  @section('content')
6      @livewire('supplyings-report-component')
7  @endsection
8
9  @section('script')
10
11
12 @endsection
13
```

Figura 153. Vista del requerimiento 24
Fuente: Elaboración propia

En la figura 156 se muestra el controlador del RF-24:

[illegible]

Figura 154. Controlador del requerimiento 24

Fuente: Elaboración propia

PRUEBA DE CAJA NEGRA DEL TERCER SPRINT

PRUEBA DE CAJA NEGRA						
Código	PCN03			Fecha	23/12/2024	
Tarea	Administrar usuarios			Módulo	Usuarios	
Descripción	Se debe validar la visualización y funcionalidad de la sección de usuarios					
Caso de prueba						
Precondiciones						
Debe haber data generada en la base de datos						
Pasos de la prueba						
Escenario 1: Búsqueda de usuario Dado que, me encuentro en el módulo de usuarios, Cuando filtro por rol, sucursal o ingreso datos, Entonces el sistema lista un resultado.						
Escenario 2: No se especifican criterios de filtrado Dado que, me encuentro en el módulo de usuarios, Cuando no ingreso ningún dato o filtro de rol y sucursal, Entonces el sistema lista todos los usuarios existentes.						
Escenario 5: Eliminar usuarios con éxito Dado que, me encuentro en el módulo de usuarios, Cuando doy click en el botón "Borrar" y posteriormente hago click en "Eliminar" Entonces el sistema indica "Producto Eliminado con éxito"						
Escenario 6: Modificar un usuario Dado que, me encuentro en el módulo de usuarios, Cuando doy click en el botón "Modificar" y edito los campos, Entonces el sistema indica "Usuario Actualizado".						
Datos de entrada			Respuesta esperada	Coincide		Respuesta del sistema
Campo	Valor	Escenario		Sí	No	
		Prueba	Se muestra el usuario buscado	x		Facilita el usuario especificado
		Prueba	Se muestran todos los usuarios	x		Se visualizan todos los usuarios
		Prueba	Eliminar un usuario	x		El usuario ya no aparece en el sistema
		Prueba	Modificar un usuario	x		El usuario se modificó
Post condiciones						
No se aplica						
Defectos				Veredicto		
No existen				Pasó		
Observaciones				Probador		
No aplica				María Milagros Llauce Chapoñan		

Figura 155. Pruebas HU3

Fuente: Elaboración propia

PRUEBA DE CAJA NEGRA						
Código		PCN09		Fecha		23/12/2024
Tarea		Administrar reportes opcionales		Módulo		Reportes
Descripción		Se debe verificar que los reportes opcionales se generen				
Caso de prueba						
Precondiciones						
No aplica						
Pasos de la prueba						
Dado que, me encuentro en el módulo de Reporte de Abastecimiento, Cuando selecciono el proveedor, sucursal, fechas y doy clic en "Generar Excel", Entonces el sistema generará un reporte de Abastecimiento de uno o varios productos. Escenario 5: Generar un reporte de ventas realizadas. Dado que, me encuentro en el módulo de Ventas realizadas, Cuando selecciono el vendedor, la sucursal y las fechas, categoría y doy click en "Generar Excel" Entonces el sistema generará un reporte de ventas realizadas.						
Datos de entrada			Respuesta esperada	Coincide		Respuesta del sistema
Campo	Valor	Escenario		Sí	No	
		Prueba	Generar reporte de abastecimiento de un producto	x		Se visualiza el documento
		Prueba	Generar reporte de ventas realizadas	x		Se visualiza el documento
Post condiciones						
No se aplica						
Defectos				Veredicto		
No existen				Pasó		
Observaciones				Probador		
No aplica				María Milagros Llauce Chapoñan		

Figura 156. Pruebas HU9

Fuente: Elaboración propia

Tabla 16. Retrospectiva del sprint 3

Problema	Causas	Acciones
No hubo contacto frecuente con cliente	Viaje por reuniones de trabajo	Se solicitó un cronograma del tiempo disponible del cliente en las próximas semanas

Fuente: Elaboración propia

ACTAS DE PLANIFICACIÓN Y REVISIÓN DEL SPRINT 3

Acta de reunión de planificación del Sprint
Fecha: 19/10/2024
Scrum Máster: María Milagros Llauce Chapoñan
Product Owner: Olimpia Gómez Alarcón

SPRINT 3

En el presente documento se da la conformidad sobre los objetivos que se realizarán en el tercer sprint por parte del equipo de desarrollo y a su vez, informar las historias de usuario que contemplan el sprint.

En el sprint 3 se determinó lo siguiente:

HU	Nº requerimiento	Objetivos
HU3	RF - 05	El sistema debe tener un módulo de usuarios en donde se podrá gestionar la búsqueda de estos por carácter o roles.
	RF - 07	El sistema debe tener un módulo de usuarios en donde se podrá editar un usuario existente.
	RF - 08	El sistema debe tener un módulo de usuarios en donde se podrá eliminar un usuario existente.
HU9	RF - 23	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas realizadas.
	RF - 24	El sistema debe permitir generar un documento EXCEL con el detalle de los abastecimientos realizados.

Firma de conformidad



Olimpia Gomez Alarcon
SUB GERENTE GENERAL
NEO COPY IMPORT

Olimpia Gomez Alarcon
Sub Gerente General Neo Copy Import

Figura 157. Acta de planificación Sprint 3

Fuente: Elaboración propia

Acta de reunión de revisión del Sprint

Fecha: 28/10/2024

Scrum Máster: María Milagros Llauce Chapoñan

Product Owner: Olimpia Gómez Alarcón

SPRINT 3

En el presente documento se da la conformidad que la Sra. Olimpia Gomez Alarcon estuvo durante el desarrollo de cada objetivo pactado en la reunión del sprint 3.

En el sprint 3 se desarrolló lo siguiente:

HU	Nº requerimiento	Objetivos
HU3	RF - 05	El sistema debe tener un módulo de usuarios en donde se podrá gestionar la búsqueda de estos por carácter o roles.
	RF - 07	El sistema debe tener un módulo de usuarios en donde se podrá editar un usuario existente
	RF - 08	El sistema debe tener un módulo de usuarios en donde se podrá eliminar un usuario existente
HU9	RF - 23	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas realizadas
	RF - 24	El sistema debe permitir generar un documento EXCEL con el detalle de los abastecimientos realizados

Firma de conformidad



2024
Olimpia Gomez Alarcon
SUB GERENTE GENERAL
RUC/200999314

Olimpia Gomez Alarcon
Sub Gerente General Neo Copy Import

Figura 158. Acta de revisión del sprint 3

Fuente: Elaboración propia

SPRINT 4

ANÁLISIS DEL SPRINT 4

En la **tabla 15** se muestran los 4 requerimientos que conforman el sprint 4.

Tabla 17. Sprint 4

HU	Nº	Requerimiento
HU9	RF - 20	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas por producto
	RF - 21	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas por vendedor
	RF - 22	El sistema debe permitir generar un documento EXCEL con el detalle del stock de producto
HU4	RF - 09	El sistema debe tener un módulo de usuarios en donde se podrá crear una sucursal

Fuente: Elaboración propia

FUNCIONALIDADES DEL CUARTO SPRINT

La **Tabla 16** ofrece una descripción completa de las funcionalidades que implementará el sistema. Además, ilustra cómo interactúan los usuarios con la arquitectura de software para satisfacer cada uno de los requerimientos definidos en el Sprint 4.

Tabla 18. Funcionalidades del sprint 4

R.F.	Funcionalidad
RF-09	El usuario al acceder al módulo de usuarios visualizará un listado de todos los usuarios activos que maneja el sistema. Hacer click en Sucursal podrá generar en el botón "Crear" un nuevo registro, con la cual, se abrirá una ventana emergente con campos a llenar. Una vez

	completados los datos, al enviar el formulario, el controlador recibe la información y la pasa al modelo para su almacenamiento en la base de datos.
RF-20	El usuario al acceder al módulo de reporte de ventas por producto visualizará un listado de los productos vendidos en el sistema. Hacer click en “Generar Excel” se generará un documento Excel con los datos actuales del aplicativo. Estos datos son llamados del modelo, la vista envía la solicitud al controlador, la cual este último recibe la solicitud.
RF-21	El usuario al acceder al módulo de reporte de ventas por vendedor visualizará un listado de las ventas realizadas en el sistema. Hacer click en “Generar Excel” se generará un documento Excel con los datos actuales del aplicativo. Estos datos son llamados del modelo, la vista envía la solicitud al controlador, la cual este último recibe la solicitud.
RF-22	El usuario al acceder al módulo de reporte de stock de productos visualizará un listado de la cantidad de productos en el sistema. Hacer click en “Generar Excel” se generará un documento Excel con los datos actuales del aplicativo. Estos datos son llamados del modelo, la vista envía la solicitud al controlador, la cual este último recibe la solicitud.

Fuente: Elaboración propia

DISEÑO DEL CUARTO SPRINT

En la **Figura 161** se muestra el prototipo del requerimiento funcional 09 la cual fue aprobado por el Product Owner para crear una sucursal.

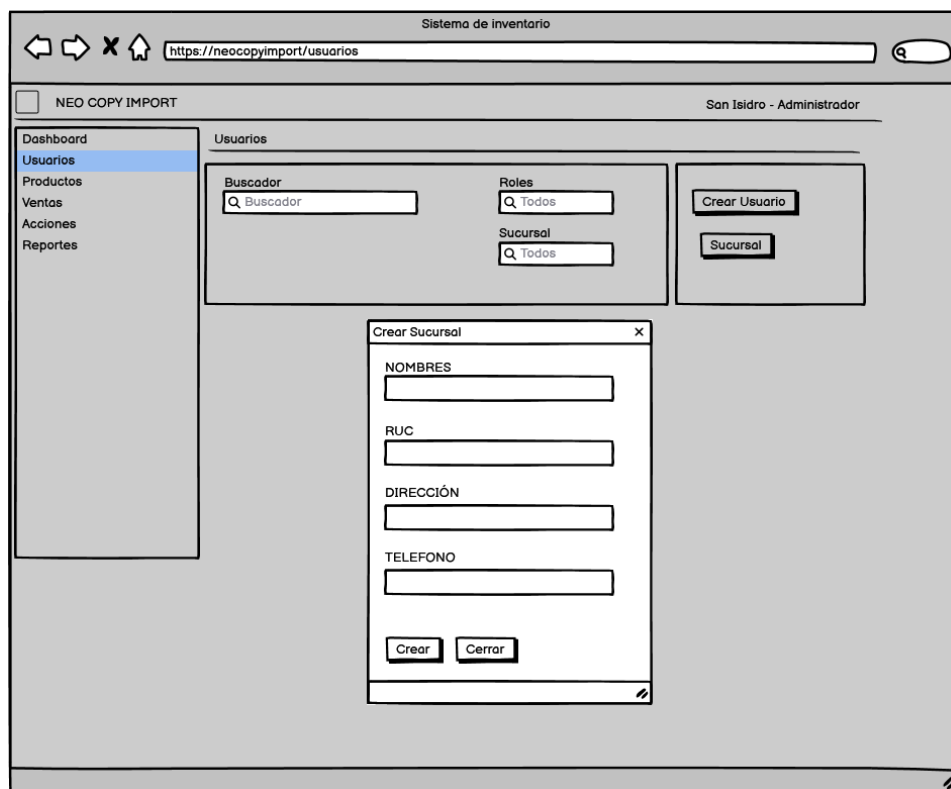


Figura 159. Prototipo para el requerimiento RF-09.

Fuente: Elaboración propia.

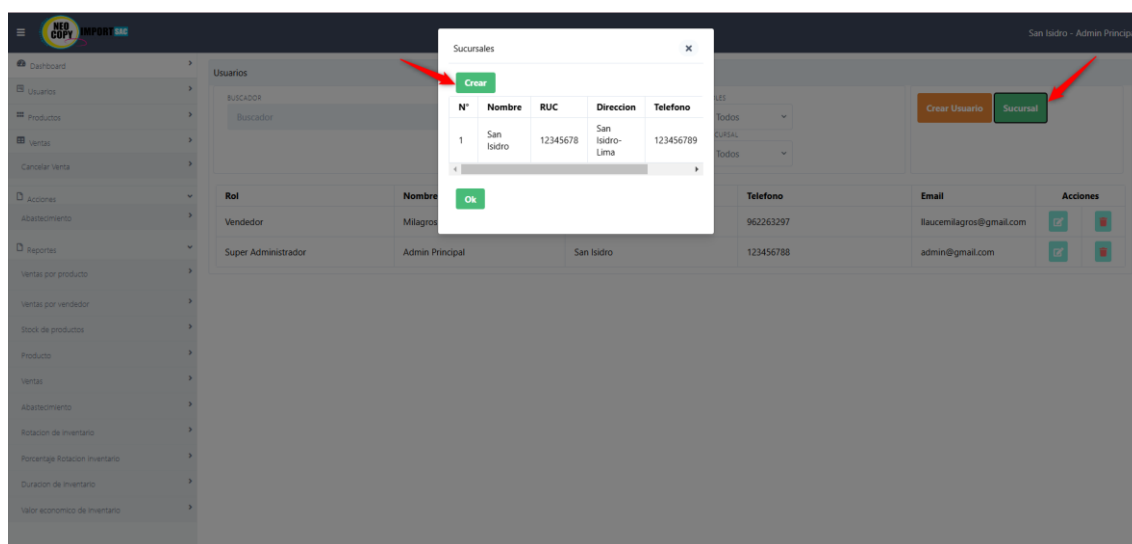


Figura 160. Crear sucursal

Fuente: Elaboración propia

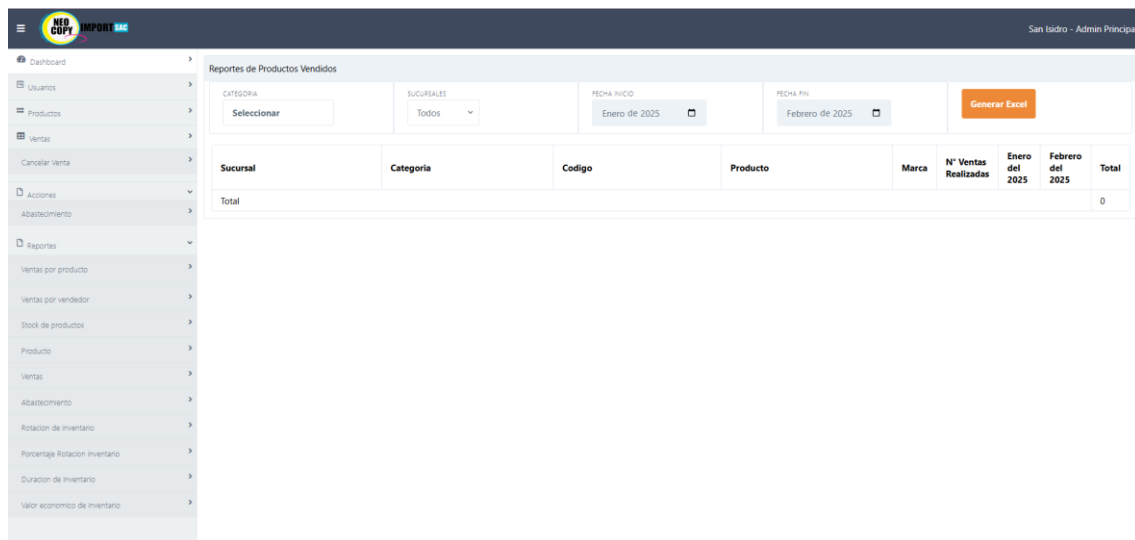


Figura 161. Ventas por producto

Fuente: Elaboración propia

En la **Figura 164** se muestra el prototipo del requerimiento funcional 20 la cual fue aprobado por el Product Owner para la vista del reporte de ventas por producto.

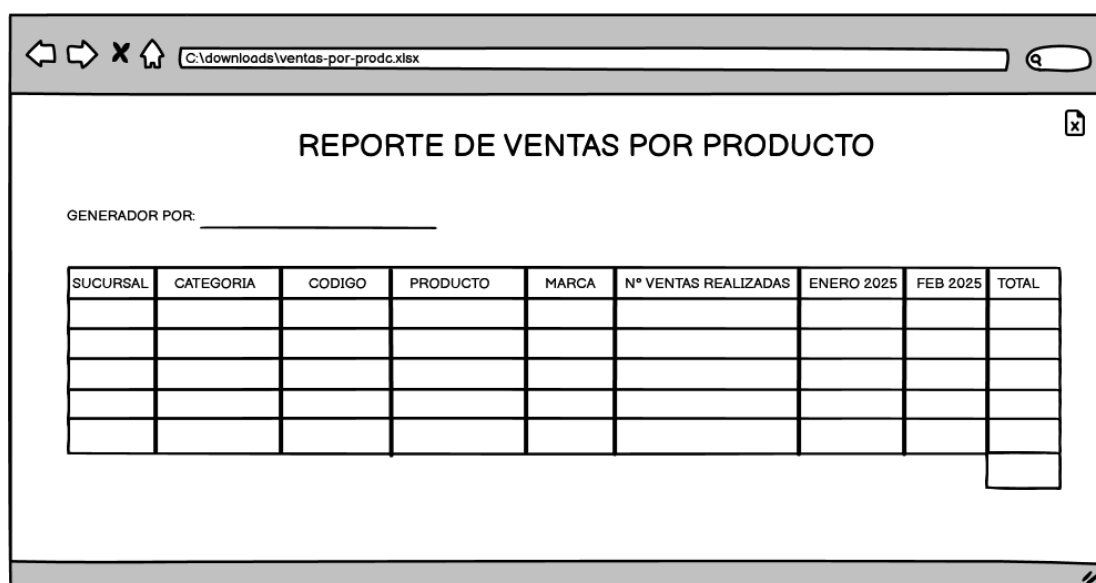


Figura 162. Prototipo para el requerimiento RF-20.

Fuente: Elaboración propia

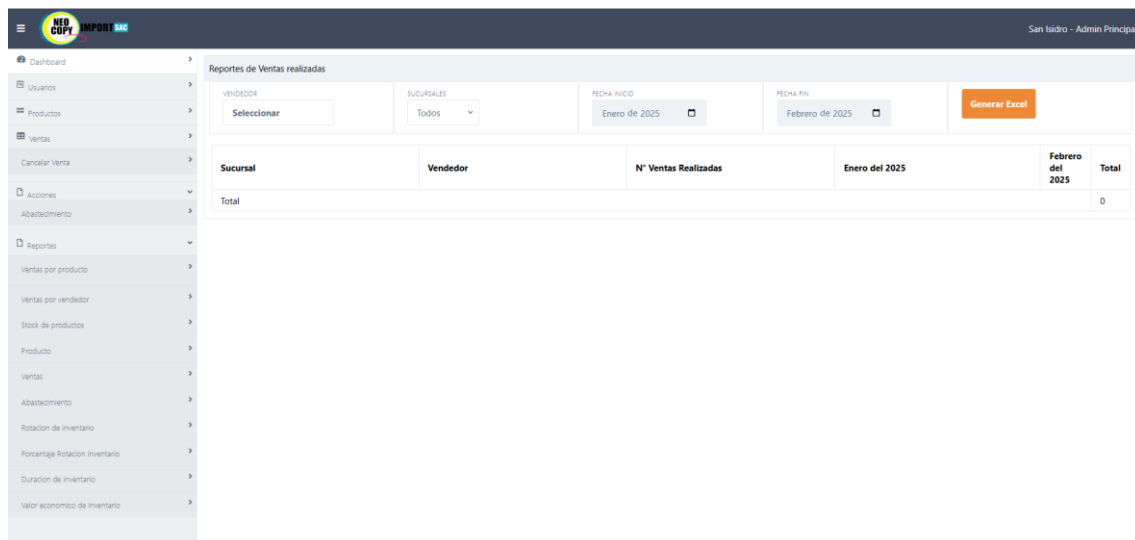


Figura 163. Reporte de ventas por vendedor

Fuente: Elaboración propia

En la **Figura 166** se muestra el prototipo del requerimiento funcional 21 la cual fue aprobado por el Product Owner para la vista del reporte de ventas por vendedor.

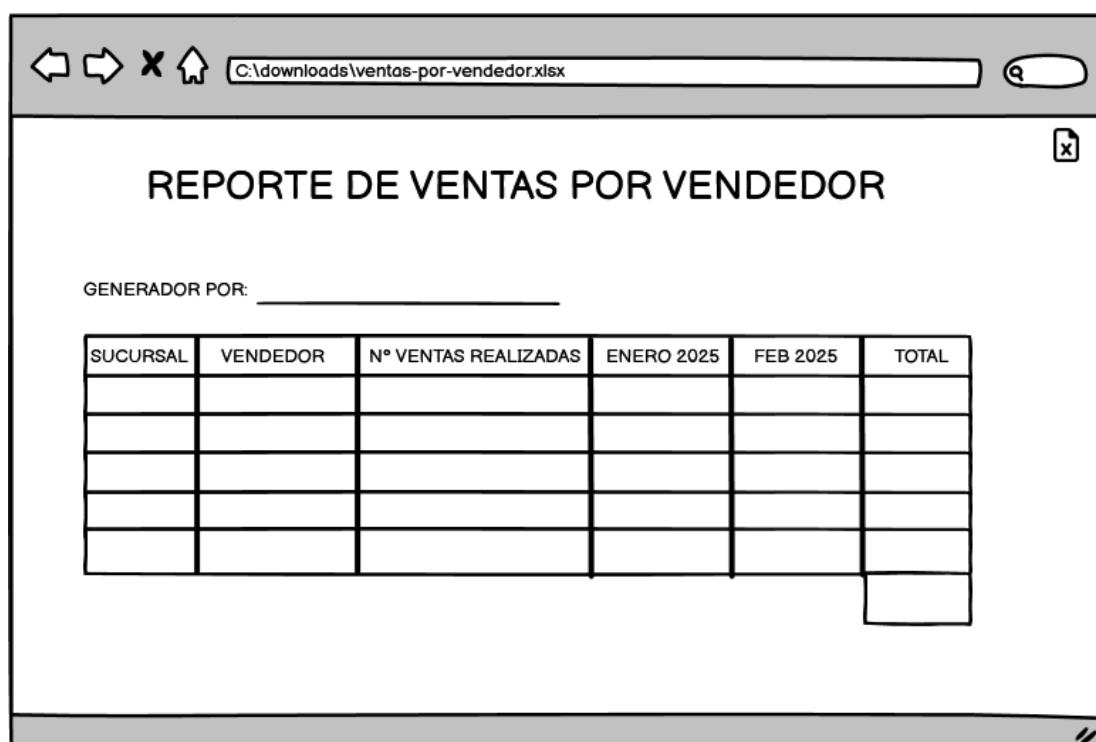


Figura 164. Prototipo para el requerimiento RF-21.

Fuente: Elaboración propia

Figura 165. Detalle stock de producto

Fuente: Elaboración propia

En la **Figura 168** se muestra el prototipo del requerimiento funcional 22 la cual fue aprobado por el Product Owner para la vista del reporte de stock de productos.

Figura 166. Prototipo para el requerimiento RF-22.

Fuente: Elaboración propia

ENTIDADES DEL CUARTO SPRINT

En la Figura 169 se muestran las entidades del Sprint 4

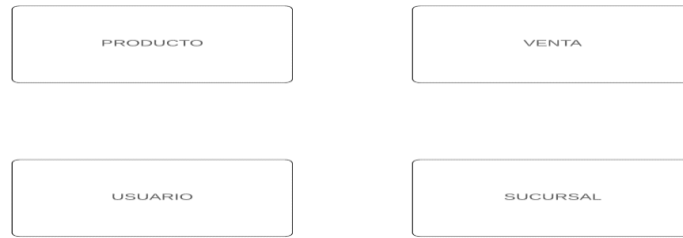


Figura 167. Entidades del Sprint 4

Fuente: Elaboración Propia

DIAGRAMA ENTIDAD/ RELACIÓN DEL CUARTO SPRINT

En la Figura 170 se muestra el diagrama conceptual de la base de datos del Sprint 4

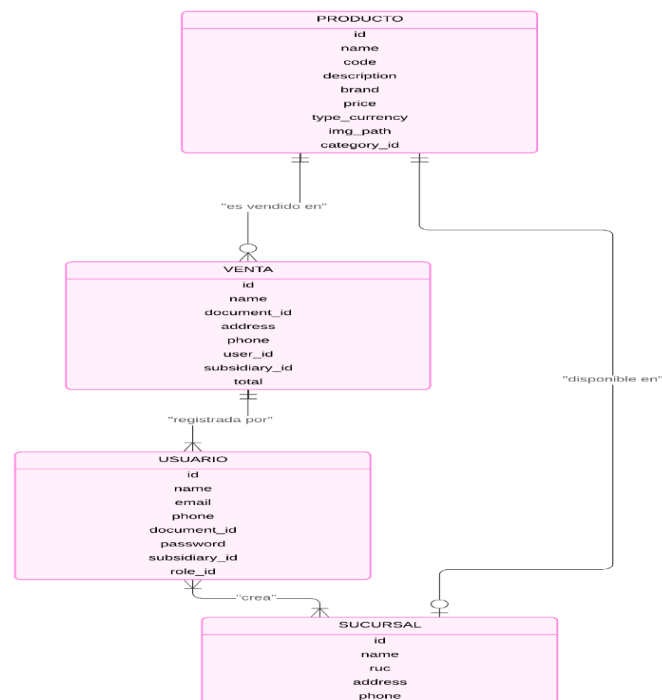


Figura 168. Diagrama Entidad / Relación del Sprint 4

Fuente: Elaboración Propia

DIAGRAMA LÓGICO DEL CUARTO SPRINT

En la figura 171 se muestra el diagrama físico que representa la descripción de los datos con el mayor detalle, su estructura y las relaciones de los datos fundamentales del negocio.

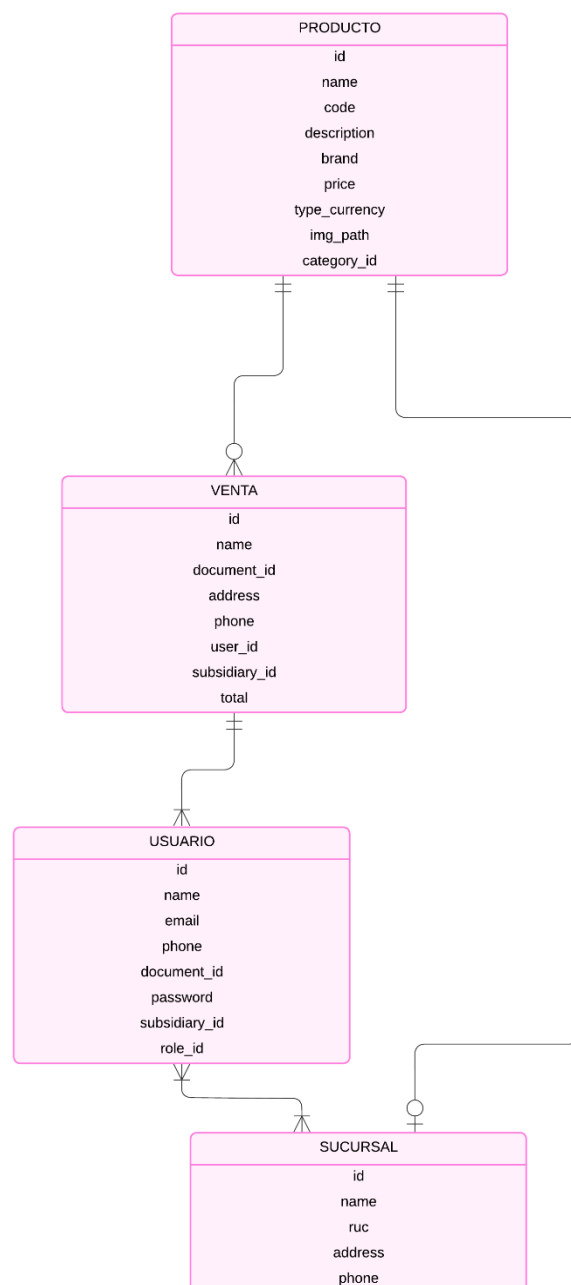


Figura 169. Diagrama Lógico del Sprint 4

Fuente: Elaboración Propia

DIAGRAMA FÍSICO DEL CUARTO SPRINT

En la figura 172 se muestra el diagrama físico que representa como se construirá el modelo en la base de datos.

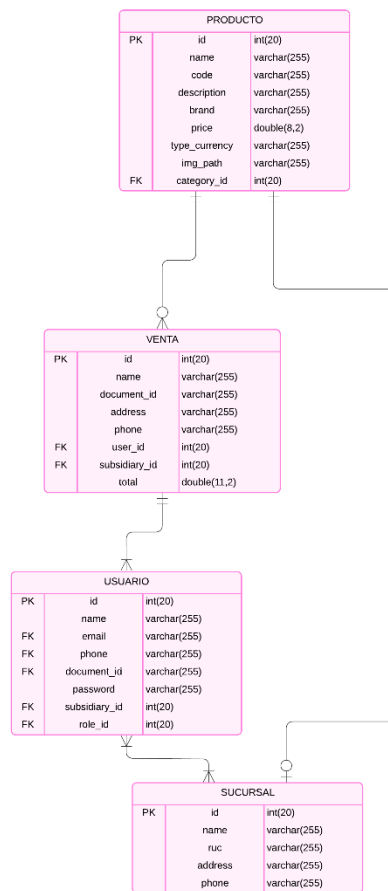


Figura 170. Diagrama Físico del Sprint 4

Fuente: Elaboración Propia

IMPLEMENTACIÓN DEL CUARTO SPRINT

Diagrama de la Base de Datos:

En la Figura 173 tenemos el diseño de la base de datos en MySQL que vamos a usar para el Sprint 4.

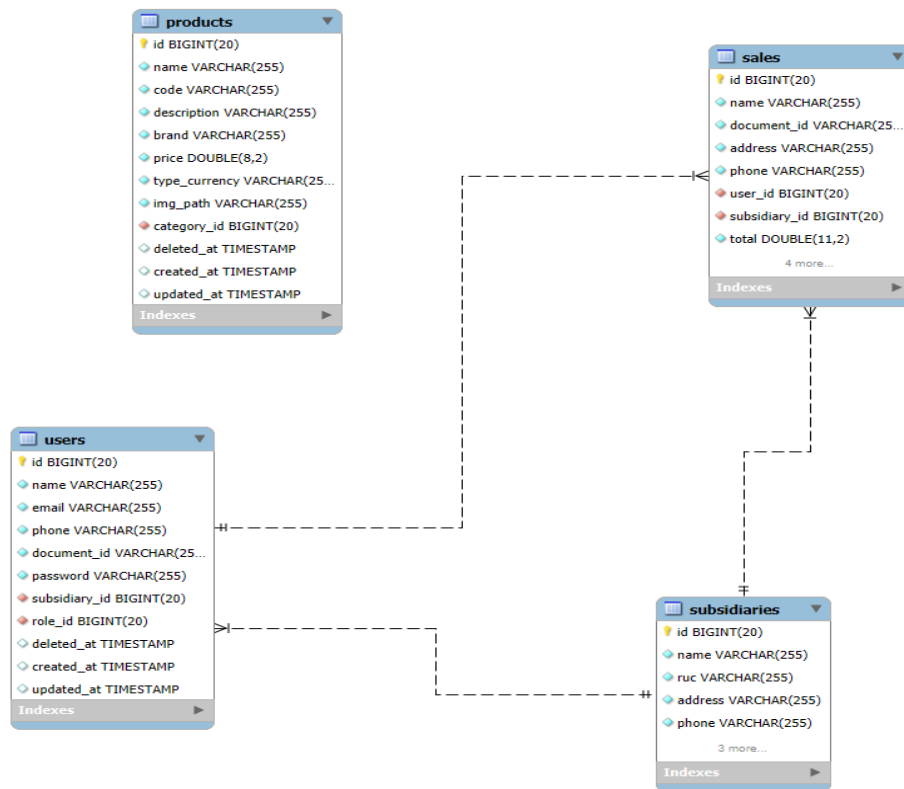


Figura 171. Diagrama de la base de datos del Sprint 4

Fuente: Elaboración Propia

RF-09: El sistema debe tener un módulo de usuarios en donde se podrá crear una sucursal

En la figura 174 se muestra el modelo del RF-09:

```

resources > views > users.blade.php
1  @extends('layouts.system-layout')
2  @section('styles')
3
4  @endsection
5  @section('content')
6      @livewire('user-component')
7      @livewire('subsidiary-component')
8  @endsection
9
10 @section('script')
11
12
13 @endsection
14
  
```

Figura 172. Modelo del requerimiento 09

Fuente: Elaboración propia

En la figura 175 se muestra la vista del RF-09:

```

app > Models > User.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Foundation\Auth\User as Authenticatable;
7  use Illuminate\Notifications\Notifiable;
8  use Laravel\Sanctum\HasApiTokens;
9
10 class User extends Authenticatable
11 {
12     use Notifiable;
13     use HasFactory;
14     use HasApiTokens;
15
16     /**
17      * The attributes that are mass assignable.
18      *
19      * @var array<int, string>
20      */
21     protected $fillable = [
22         'name',
23         'phone',
24         'email',
25         'password',
26         'document_id',
27         'subsidiary_id',
28         'role_id'
29     ];
30
31     /**
32      * The attributes that should be hidden for serialization.
33      *
34      * @var array<int, string>
35      */
36     protected $hidden = [
37         'password'
38     ];
39
40     public function subsidiary()
41     {
42         return $this->belongsTo(Subsidiary::class);
43     }
44
45     public function role()
46     {
47         return $this->belongsTo(Role::class);
48     }
49
50 }

```

Figura 173. Vista del requerimiento 09

Fuente: Elaboración propia

En la figura 176 se muestra el controlador del RF-09:

```

app > Http > Livewire > UserComponent.php
1  <?php
2
3  namespace App\Http\Livewire;
4
5  use App\Models\Role;
6  use App\Models\Subsidiary;
7  use App\Models\User;
8  use Illuminate\Support\Facades\Hash;
9  use Livewire\Component;
10 use Livewire\WithPagination;
11 use Illuminate\Validation\Rule;
12 use Illuminate\Support\Facades\Log;
13 class UserComponent extends Component
14 {
15     use WithPagination;
16
17     public $name_e, $phone_e, $email_e, $password_e, $document_id_e, $subsidiary_id_e, $role_id_e;
18     public $name_e, $phone_e, $email_e, $password_e, $document_id_e, $subsidiary_id_e, $role_id_e, $id_e;
19
20     public $delete_id;
21
22     public $text_filter;
23     public $subsidiary_filter;
24     public $role_filter;
25
26     public function render()
27     {
28         return view('livewire.user-component', [
29             'users' => User::when($this->text_filter, function($query){
30                 if(empty($this->text_filter))
31                 {
32                     $query->where('name', 'like', '%' . $this->text_filter . '%' );
33                 }
34             })->
35             when($this->subsidiary_filter, function($query){
36                 if(empty($this->subsidiary_filter))
37                 {
38                     $query->where('subsidiary_id', $this->subsidiary_filter);
39                 }
40             })->
41             when($this->role_filter, function($query){
42                 if(empty($this->role_filter))
43                 {
44                     $query->where('role_id', $this->role_filter);
45                 }
46             })->orderBy('id', 'desc')->paginate(10),
47             'subsidiary_list' => Subsidiary::all(),
48             'role_list' => Role::all(),
49         ]);
50     }
51 }

```

Figura 174. Controlador del requerimiento 09

Fuente: Elaboración propia

RF-20: El sistema debe permitir generar un documento EXCEL con el detalle de las ventas por producto

En la figura 177 se muestra el modelo del RF-20:

```
app > Models > Sale.php
1 <?php
2
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Factories\HasFactory;
6 use Illuminate\Database\Eloquent\Model;
7 use Illuminate\Database\Eloquent\SoftDeletes;
8
9 class Sale extends Model
10 {
11     use HasFactory;
12     use SoftDeletes;
13
14     protected $fillable = ['name', 'document_id', 'address', 'phone', 'user_id', 'subsidiary_id', 'total', 'created_date'];
15
16     public function user()
17     {
18         return $this->belongsTo(User::class);
19     }
20
21     public function subsidiary()
22     {
23         return $this->belongsTo(Subsidiary::class);
24     }
25
26     public function getTotalAttribute($value)
27     {
28         return round($value, 2);
29     }
30 }
31
```

Figura 175. Modelo del requerimiento 20

Fuente: Elaboración propia

En la figura 178 se muestra la vista del RF-20:

```
resources > views > sale_report.blade.php
1 @extends('layouts.system-layout')
2 @section('styles')
3 <style>
4     .multiselect {
5         width: 200px;
6     }
7
8     .selectBox {
9         position: relative;
10    }
11
12    .selectBox select {
13        width: 100%;
14        font-weight: bold;
15    }
16
17    .overSelect {
18        position: absolute;
19        left: 0;
20        right: 0;
21        top: 0;
22        bottom: 0;
23    }
24
25    #checkboxes {
26        display: none;
27        border: 1px solid #dadada;
28    }
29
30    #checkboxes label {
31        display: block;
32    }
33
34    #checkboxes label: hover {
35        background-color: #1e90ff;
36    }
37 </style>
```

Figura 176. Vista del requerimiento 20

Fuente: Elaboración propia

En la figura 179 se muestra el controlador del RF-20:

```
81 $listado_data = [];
82
83 foreach ($sales_data as $clave => $valor)
84 {
85     $val = true;
86
87     foreach ($listado_data as $clave2 => $valor2)
88     {
89         if($valor2->subsidiaryname == $valor->subsidiaryname && $valor2->username == $valor->username)
90         {
91             $val = false;
92         }
93     }
94
95     if($val)
96     {
97         $obj = new stdClass();
98         $obj->subsidiaryname = $valor->subsidiaryname;
99         $obj->username = $valor->username;
100         $obj->quantity = 0;
101         $obj->total = 0;
102         $obj->data = [];
103
104         $listado_data[] = $obj;
105
106         foreach ($listado_data as $clave2 => $valor2)
107         {
108             if($valor2->subsidiaryname == $valor->subsidiaryname && $valor2->username == $valor->username)
109             {
110                 $valor2->data[$valor->year . '-' . $valor->month] = $valor->total;
111                 $valor2->quantity += $valor->quantity;
112                 $valor2->total += $valor->total;
113             }
114         }
115     }
116     else
117     {
118         foreach ($listado_data as $clave2 => $valor2)
119         {
120             if($valor2->subsidiaryname == $valor->subsidiaryname && $valor2->username == $valor->username)
121             {
122                 $valor2->data[$valor->year . '-' . $valor->month] = $valor->total;
123                 $valor2->quantity += $valor->quantity;
124                 $valor2->total += $valor->total;
125             }
126         }
127     }
128 }
129
130 }
```

Figura 177. Controlador del requerimiento 20

Fuente: Elaboración propia

RF-21: El sistema debe permitir generar un documento EXCEL con el detalle de las ventas por vendedor

En la figura 180 se muestra el modelo del RF-21:

```
app > Models > SaleDetail.php
1 <?php
2
3 namespace App\Models;
4
5 use Illuminate\Database\Eloquent\Factories\HasFactory;
6 use Illuminate\Database\Eloquent\Model;
7 use Illuminate\Database\Eloquent\SoftDeletes;
8
9 class SaleDetail extends Model
10 {
11     use HasFactory;
12     use SoftDeletes;
13
14     protected $fillable = ['product_id','quantity','price','total','sale_id'];
15
16     public function product()
17     {
18         return $this->belongsTo(Product::class);
19     }
20
21     public function sale()
22     {
23         return $this->belongsTo(Sale::class);
24     }
25
26     public function getTotalAttribute($value)
27     {
28         return round($value, 2);
29     }
30
31     public function getPriceAttribute($value)
32     {
33         return round($value, 2);
34     }
35 }
36
```

Figura 178. Modelo del requerimiento 21

Fuente: Elaboración propia

En la figura 181 se muestra la vista del RF-21:

```
resources > views > sales/sales.php
3 <style>
4
5
6
7
8
9 body {
10   font: 16px Arial;
11 }
12
13 /*the container must be positioned relative*/
14 .autocomplete {
15   position: relative;
16   display: inline-block;
17 }
18
19 input {
20   border: 1px solid transparent;
21   background-color: #f1f1f1;
22   padding: 10px;
23   font-size: 16px;
24 }
25
26 input[type=text] {
27   background-color: #f1f1f1;
28   width: 100%;
29 }
30
31 input[type=submit] {
32   background-color: DodgerBlue;
33   color: #fff;
34   cursor: pointer;
35 }
36
37 .autocomplete-items {
38   position: absolute;
39   border: 1px solid #d4d4d4;
40   border-bottom: none;
41   border-top: none;
42   z-index: 99;
43   /*position the autocomplete items to be the same width as the container*/
44   top: 100%;
45   left: 0;
46   right: 0;
47 }
48
49 .autocomplete-items div {
50   padding: 10px;
51   cursor: pointer;
52   background-color: #fff;
53   border-bottom: 1px solid #d4d4d4;
54 }
55
56 /*when hovering an item*/
57 .autocomplete-items div:hover {
58   background-color: #e9e9e9;
59 }
```

Figura 179. Vista del requerimiento 21

Fuente: Elaboración propia

En la figura 182 se muestra el controlador del RF-21:

```
1 public function getSalesDetail()
2 {
3     return SalesDetail::whereHas('sale', function($query){
4         if(!empty($this->user_id))
5         {
6             $query->where('user_id','=', $this->user_id);
7         }
8
9         if(!empty($this->subsidiary_id))
10        {
11            $query->where('subsidiary_id','=', $this->subsidiary_id);
12        }
13
14        $query->whereBetween('created_date', [$this->init_date, $this->end_date]);
15    })->
16    orderBy('id','desc');
17 }
18
19 public function get($id)
20 {
21     $this->sale_details = SalesDetail::where('sale_id', $id)->get();
22     $this->dispatchBrowserEvent('openModal', ['modal' => 'report_sales_detail_modal']);
23 }
24
25 public function getExcel()
26 {
27     $salesDetail = $this->getSalesDetail()->get();
28     $list_data = [];
29
30     foreach($salesDetail as $saleDetail)
31     {
32         $sr = [];
33         $sr[] = $saleDetail->sale->name;
34         $sr[] = $saleDetail->sale->document_id;
35         $sr[] = $saleDetail->sale->address;
36         $sr[] = $saleDetail->sale->phone;
37         $sr[] = $saleDetail->sale->user->name;
38
39         $sr[] = $saleDetail->product->code;
40         $sr[] = $saleDetail->product->name;
41         $sr[] = $saleDetail->quantity;
42         $sr[] = $saleDetail->price;
43         $sr[] = $saleDetail->total;
44
45         $sr[] = $saleDetail->sale->total;
46         $sr[] = $saleDetail->sale->created_date;
47
48         $list_data[] = $sr;
49     }
50 }
```

Figura 180. Controlador del requerimiento 21

Fuente: Elaboración propia

RF-22: El sistema debe permitir generar un documento EXCEL con el detalle del stock de producto

En la figura 183 se muestra el modelo del RF-22:

```
app > Models > ProductStock.php
1  <?php
2
3  namespace App\Models;
4
5  use Illuminate\Database\Eloquent\Factories\HasFactory;
6  use Illuminate\Database\Eloquent\Model;
7  use Illuminate\Database\Eloquent\SoftDeletes;
8
9  class ProductStock extends Model
10 {
11     use HasFactory;
12     use SoftDeletes;
13
14     protected $fillable = ['product_id','stock','subsidiary_id'];
15
16     public function product()
17     {
18         return $this->belongsTo(Product::class);
19     }
20
21     public function subsidiary()
22     {
23         return $this->belongsTo(Subsidiary::class);
24     }
25 }
26
```

Figura 181. Modelo del requerimiento 22

Fuente: Elaboración propia

En la figura 184 se muestra la vista del RF-22:

```
resources > views > product_stock_reportblade.php
1 @extends('layouts.system-layout')
2 @section('styles')
3 <style>
4     * {
5         box-sizing: border-box;
6     }
7
8     body {
9         font: 16px Arial;
10    }
11
12    /*The container must be positioned relative*/
13    .autocomplete {
14        position: relative;
15        display: inline-block;
16    }
17
18    input {
19        border: 1px solid transparent;
20        background-color: #fff;
21        padding: 10px;
22        font-size: 16px;
23    }
24
25    input[type="text"] {
26        background-color: #fff;
27        width: 100%;
28    }
29
30    input[type="submit"] {
31        background-color: #007bff;
32        color: #fff;
33        cursor: pointer;
34    }
35
36    .autocomplete-items {
37        position: absolute;
38        border: 1px solid #ddd;
39        border-bottom: none;
40        border-top: none;
41        z-index: 99;
42        /*position the autocomplete items to be the same width as the container*/
43        top: 100%;
44        left: 0;
45        right: 0;
46    }
47
48    .autocomplete-items div {
49        padding: 10px;
50        cursor: pointer;
51        background-color: #fff;
52        border-bottom: 1px solid #ddd;
53    }
54
55    /*when hovering an item*/
56    .autocomplete-items div:hover {
57        background-color: #e6e6fa;
58    }
59
60    /*when navigating through the items using the arrow keys*/
61    .autocomplete-active {
62        background-color: #007bff;
63        color: #fff;
64    }
65 </style>
```

Figura 182. Vista del requerimiento 22

Fuente: Elaboración propia

En la figura 185 se muestra el controlador del RF-09:

```
class ProductRequirementController extends Controller {
  public $view_filter;
  public $category_filter;
  public $init_date;
  public $end_date;
  public $product_id;
  public $selectedProductId;

  public function render() {
    if($this->init_date == null) {
      $this->init_date = (Carbon::now()-format('Y-m-d'));
    }
    if($this->end_date == null) {
      $this->end_date = (Carbon::now()+addDays(1))-format('Y-m-d');
    }
    $product_query = Product::query();
    if(!empty($this->category_filter)) {
      $product_query = $product_query->where('category_id', $this->category_filter);
    }
    return view('includes/product-stock-report-component', [
      'viewname' => $this->getarden(),
      'category' => $category_id,
      'products' => $product_query->orderBy('id', 'desc')->paginate(),
      'subsidiary' => $subsidiary::find(Auth::user()->subsidiary_id)
    ]);
  }

  public function getarden() {
    return $arden->where('subsidiary_id', '=', Auth::user()->subsidiary_id)
      ->where('product_id', '=', $this->product_id)
      ->whereBetween('created_date', [$this->init_date, $this->end_date])
      ->orderBy('product_id', 'desc');
  }

  public function add() {
    $id = $this->selectedProductId;
    $this->product_id = $id;
  }

  public function getarden() {
    $ardenes = $this->getarden()->get();
    $list_data = [];
    foreach($ardenes as $arden) {
      $ar = [
        $arden->product->category->name,
        $arden->product->stock,
        $arden->product->name,
        $arden->product->name,
        $arden->product->name,
        $arden->previous_stock,
        $arden->quantity,
        ($arden->operation == '+') ? $arden->previous_stock + $arden->quantity : $arden->previous_stock - $arden->quantity,
        $arden->operation,
        $arden->created_date,
        $arden->created_date
      ];
      $list_data[] = $ar;
    }
  }
}
```

Figura 183. Controlador del requerimiento 22

Fuente: Elaboración propia

PRUEBA CAJA NEGRA DEL CUARTO SPRINT

PRUEBA DE CAJA NEGRA						
Código	PCN09			Fecha	23/12/2024	
Tarea	Administrar reportes opcionales			Módulo	Reportes	
Descripción	Se debe verificar que los reportes opcionales se generen					
Caso de prueba						
Precondiciones						
No aplica						
Pasos de la prueba						
Escenario 2: Generar un reporte de ventas realizadas por vendedor Dado que, me encuentro en el módulo de Reportes de Ventas realizadas, Cuando selecciono el vendedor, sucursal y las fechas, doy click en "Generar Excel" Entonces el sistema generará un reporte de las ventas realizadas por vendedor filtrado.						
Escenario 3: Generar un reporte de stock de productos. Dado que, me encuentro en el módulo de Stock de productos Cuando selecciono la sucursal y las fechas, categoría, y busco el producto y doy click en "Generar Excel" Entonces el sistema generará un reporte de stock de productos filtrado.						
Escenario 6: Generar un reporte de ventas realizadas por producto Dado que, me encuentro en el módulo de Reportes de Productos Vendidos, Cuando selecciono la categoría, sucursal y las fechas, doy click en "Generar Excel" Entonces el sistema generará un reporte de las ventas realizadas por producto filtrado.						
Datos de entrada			Respuesta esperada	Coincide		Respuesta del sistema
Campo	Valor	Escenario		Sí	No	
		Prueba	Generar reporte de ventas por vendedor	x		Se visualiza el documento
		Prueba	Generar reporte de stock de productos	x		Se visualiza el documento
		Prueba	Generar reporte de ventas realizadas por producto	x		Se visualiza el documento
Post condiciones						
No se aplica						
Defectos				Veredicto		
No existen				Pasó		
Observaciones				Probador		
No aplica				María Milagros Llauce Chapoñan		

Figura 184. Pruebas HU9

Fuente: Elaboración propia

PRUEBA DE CAJA NEGRA						
Código	PCN04			Fecha	23/12/2024	
Tarea				Módulo		
Descripción	Se debe verificar la funcionalidad de agregar una nueva sucursal					
Caso de prueba						
Precondiciones						
No aplica						
Pasos de la prueba						
Escenario 2: Agregar una sucursal con éxito Dado que, me encuentro en el módulo de usuarios, Cuando doy clic en "Sucursal" y en "Crear" e ingreso data en los campos, Entonces en el sistema figura un pop-up: "Sucursal agregada con éxito"						
Datos de entrada			Respuesta esperada	Coincide		Respuesta del sistema
Campo	Valor	Escenario		Sí	No	
		Prueba	Agregar sucursal	x		Se visualiza la sucursal agregada
Post condiciones						
No se aplica						
Defectos				Veredicto		
No existen				Pasó		
Observaciones				Probador		
No aplica				María Milagros Llauce Chapoñan		

Figura 185. Pruebas HU

Fuente: Elaboración propia

Tabla 19. Retrospectiva del sprint 4

Problema	Causas	Acciones
Subida a la nube de la web para la empresa	Cliente consideró primero la implementación beta sin subir a la nube	Se demostró que el sistema web funciona con normalidad

Fuente: Elaboración propia

ACTAS DE PLANIFICACIÓN Y REVISIÓN DEL SPRINT 4

Acta de reunión de planificación del Sprint

Fecha: 02/11/2024

Scrum Máster: María Milagros Llauce Chapoñan

Product Owner: Olimpia Gómez Alarcón

SPRINT 4

En el presente documento se da la conformidad sobre los objetivos que se realizarán en el cuarto sprint por parte del equipo de desarrollo y a su vez, informar las historias de usuario que contemplan el sprint.

En el sprint 4 se determinó lo siguiente:

HU	Nº	Requerimiento
HU9	RF - 20	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas por producto.
	RF - 21	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas por vendedor.
	RF - 22	El sistema debe permitir generar un documento EXCEL con el detalle del stock de producto.
HU4	RF - 09	El sistema debe tener un módulo de usuarios en donde se podrá crear una sucursal.

Firma de conformidad



Olimpia Gomez Alarcon
Sub Gerente General Neo Copy Import

Figura 186. Acta de planificación del sprint 4

Fuente: Elaboración propia

Acta de reunión de revisión del Sprint

Fecha: 14/11/2024

Scrum Máster: María Milagros Llauce Chapoñan

Product Owner: Olimpia Gómez Alarcón

SPRINT 4

En el presente documento se da la conformidad que la Sra. Olimpia Gomez Alarcon estuvo durante el desarrollo de cada objetivo pactado en la reunión del sprint 4.

En el sprint 4 se desarrolló lo siguiente:

HU	Nº requerimiento	Objetivos
HU9	RF - 20	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas por producto
	RF - 21	El sistema debe permitir generar un documento EXCEL con el detalle de las ventas por vendedor
	RF - 22	El sistema debe permitir generar un documento EXCEL con el detalle del stock de producto
HU4	RF - 09	El sistema debe tener un módulo de usuarios en donde se podrá crear una sucursal

Firma de conformidad



Olimpia Gomez Alarcon
Sub Gerente General Neo Copy Import

Figura 187. Acta de revisión del sprint 4

Fuente: Elaboración propia